

# Block 4: Implementation and Applications of AI

Short explanations, worked examples and self-check questions for every unit, with every idea also explained in Malayalam.

ഓരോ ആശയവും ലളിതമായ ഇംഗ്ലീഷിലും മലയാളത്തിലും, ഉദാഹരണങ്ങളോടെ.

## UNIT 10

### Intelligent Agents

ചുറ്റുപാട് sensors വഴി അറിഞ്ഞ് actuators വഴി പ്രവർത്തിക്കുന്ന "agent" എന്താണ്, അത് എപ്പോഴാണ് rational ആകുന്നത് എന്നതാണ് ഈ unit. PEAS, environment-ന്റെ

തരങ്ങൾ, simple reflex മുതൽ learning agent വരെയുള്ള agent-കളുടെ ഘടന, agent-കൾ പഠിക്കുന്ന വഴികൾ എന്നിവ ഉദാഹരണങ്ങൾ സഹിതം നമ്മൾ കാണുന്നു.

What is an agent? Percept, percept sequence, software agent

Two approaches to defining an agent, and agent properties

Rational agents and the performance measure

Rationality vs. omniscience; autonomy and learning

Example: the boundary-following robot

Task environment and PEAS: the automated road transport driver

PEAS for more agents

Environment types 1: observable, static/dynamic, deterministic/stochastic

Environment types 2: episodic/sequential, discrete/continuous

Environment types 3: single-agent vs. multi-agent

Classifying task environments (the unit's five examples and more)

Structure of an agent: architecture + program; table-driven agents

Simple reflex (SR) agents

Model-based reflex agents

Goal-based agents

Utility-based agents

Learning agents and their four components

Forms of learning: rote, instruction, analogy, induction, deduction

Supervised, unsupervised and reinforcement learning

## What is an agent? Percept, percept sequence, software agent

**Agent:** An entity that perceives its environment through sensors and acts upon that environment through actuators.

**Agent:** Sensors വഴി തന്റെ environment-നെ perceive ചെയ്യുകയും actuators വഴി ആ environment-ൽ പ്രവർത്തിക്കുകയും ചെയ്യുന്ന ഒരു entity.

**Percept:** The input an agent receives through its sensors in a unit of time.

**Percept:** ഒരു യൂണിറ്റ് സമയത്തിൽ agent-ന് sensors വഴി ലഭിക്കുന്ന input.

**Percept sequence:** The history of all percepts the agent has received so far.

**Percept sequence:** Agent-ന് ഇതുവരെ ലഭിച്ച എല്ലാ percepts-ന്റെയും ചരിത്രം (history).

An **agent** is an entity that acts, generally on behalf of someone else. More precisely, an agent **perceives its environment through sensors and acts on the environment through actuators**. Some experts also require an agent to be autonomous and goal-directed. A **percept** is the input the agent receives through its sensors over a unit of time; a **percept sequence** is the history of percepts, long enough for the agent to decide on an action.

### മലയാളത്തിൽ

Agent എന്നാൽ നമുക്കുവേണ്ടി പണിയെടുക്കുന്ന ഒരാൾ, ഒരു ഗൾഫ് visa agent-നെപ്പോലെ. AI-യിൽ agent ചുറ്റുപാട് sensors (camera, mic, thermometer) വഴി അറിയുന്നു, actuators (wheels, speaker, screen) വഴി പ്രവർത്തിക്കുന്നു. ഒരു നിമിഷം കിട്ടുന്ന input ആണ് percept; ഇതുവരെ കിട്ടിയ percept-കളുടെ മുഴുവൻ നിരയാണ് percept sequence.

Agent: perceives through sensors, acts through actuators

Percept = input received through sensors in a unit of time

Percept sequence = the history (sequence) of percepts received so far

Agent function: percept sequence  $\rightarrow$  action

John McCarthy and his colleagues at MIT gave one of the first definitions: a **software agent** is a system which, when given a goal, carries out the details of the appropriate computer operations, and when it gets stuck, asks humans for advice, evaluates that advice and acts suitably. A computer agent is software that additionally:

- (i) has autonomous control (operates under its own control)
- (ii) is perceptive (can perceive its environment)
- (iii) persists over a long period of time
- (iv) adapts to changes in the environment
- (v) is capable of taking over others' goals

### EXAMPLE

Identify the sensors, actuators and a percept for a room air-conditioner with auto mode.

Sensors: temperature sensor, humidity sensor, remote-control receiver. Actuators: compressor switch, fan motor, louvre motor, display. A percept: "room at 29 °C, humidity 80%, remote says 24 °C". A percept sequence: the readings of the last 10 minutes, which show whether the room is cooling fast enough.

## Two approaches to defining an agent, and agent properties

**Ascription approach:** Defines an agent by how a person perceives it: an entity that acts on someone's behalf to perform a delegated task.

**Ascription approach:** ഒരാൾ agent-നെ എങ്ങനെ കാണുന്നു എന്നതിനെ അടിസ്ഥാനമാക്കിയുള്ള നിർവ്വചനം: മറ്റൊരാൾക്കുവേണ്ടി ഏൽപ്പിക്കപ്പെട്ട task ചെയ്യുന്ന entity.

**Description approach:** Defines an agent by its properties: an entity that functions continuously and autonomously in an environment, possibly with other agents.

**Description approach:** Properties അടിസ്ഥാനമാക്കിയുള്ള നിർവ്വചനം: ഒരു environment-ൽ, ഒരുപക്ഷേ മറ്റ് agents-നോടൊപ്പം, തുടർച്ചയായും autonomous ആയും പ്രവർത്തിക്കുന്ന entity.

Because the idea is young, experts define "agent" in two related ways. The **ascription approach** treats an agent as the way a person perceives it ("agent is that agent does"): it acts on someone's behalf on a delegated task and must know both the domain and its user. A football-playing agent and an email-managing agent differ in expertise but both model their user. The **description approach** defines an agent by its properties: an entity that functions **continuously and autonomously** in an environment, possibly with other agents, adapts without constant human guidance, learns from experience, and communicates and cooperates with other agents.

#### മലയാളത്തിൽ

ആദ്യ രീതിയിൽ agent-നെ "അത് ചെയ്യുന്ന ജോലി" നോക്കി നിർവചിക്കുന്നു: നമ്മൾ ഒരു electrician-നെയും plumber-നെയും വ്യത്യസ്ത ജോലിക്ക് വിളിക്കുന്നതുപോലെ. രണ്ടാമത്തെ രീതിയിൽ agent-ന് വേണ്ട ഗുണങ്ങളുടെ list നോക്കുന്നു: സ്വയം പ്രവർത്തിക്കണം, ദീർഘകാലം നിലനിൽക്കണം, മാറ്റങ്ങളോട് പൊരുത്തപ്പെടണം, മറ്റ് agent-കളുമായി സംസാരിക്കണം.

| Property                   | Meaning                                                           | Example (food-delivery app agent)                 |
|----------------------------|-------------------------------------------------------------------|---------------------------------------------------|
| Reactivity                 | Senses the environment and acts accordingly                       | Re-routes the rider when a road is blocked        |
| Autonomy                   | Moves towards its goal, changing strategy without much human help | Reassigns an order when a rider cancels           |
| Communicating ability      | Talks to other agents and humans                                  | Messaging restaurant and customer                 |
| Coexisting by cooperating  | Works with other agents for a common goal                         | Coordinates with the payment agent                |
| Adapting to new situations | Learns, changes and adapts                                        | Learns that monsoon evenings need more riders     |
| Drawing inferences         | Concludes useful facts not directly given                         | Infers "customer is vegetarian" from past orders  |
| Temporal continuity        | Works over long periods                                           | Runs 24 × 7                                       |
| Personality                | Simulates the person it acts for                                  | Replies in the user's preferred language and tone |
| Mobility                   | Moves from one environment to another                             | Same agent runs on phone, web and smartwatch      |

**Exam tip:** "List the properties of an agent" is a common 5-mark question. Memorise the nine names (Reactivity, Autonomy, Communication, Cooperation, Adaptation, Inference, Temporal continuity, Personality, Mobility) and give one line each.

## Rational agents and the performance measure

**Rational agent:** An agent that, for each percept sequence, selects the action expected to maximise its performance measure, given its knowledge.

**Rational agent:** ഓരോ percept sequence-നും, തനിക്കുള്ള knowledge വെച്ച്, performance measure പരമാവധിയാക്കുമെന്ന് പ്രതീക്ഷിക്കുന്ന action തിരഞ്ഞെടുക്കുന്ന agent.

**Performance measure:** An objective, measurable criterion, fixed by the designer, for judging how successful an agent's behaviour is.

**Performance measure:** Agent-ന്റെ പ്രവർത്തനം എത്രത്തോളം വിജയകരമാണെന്ന് വിലയിരുത്താൻ designer നിശ്ചയിക്കുന്ന, objective ആയതും അളക്കാവുന്നതുമായ മാനദണ്ഡം.

A **rational agent** acts so as to achieve the best outcome when outcomes are certain, and the **best expected outcome** when they are uncertain. Correct logical inference is only one route to rationality: a reflex like pulling your hand from a hot vessel is rational without any reasoning. When a perfectly correct action is impossible, taking the best approximate action under the circumstances is also rational. Rationality is judged by a **performance measure**.

### മലയാളത്തിൽ

Rational എന്നാൽ "ഉള്ള അറിവ് വെച്ച് ഏറ്റവും നല്ലത് ചെയ്യുക" എന്നാണ്, എല്ലായ്പ്പോഴും ജയിക്കുക എന്നല്ല. ചൂടുള്ള ചട്ടിയിൽ കൈ തട്ടിയാൽ ആലോചിക്കാതെ കൈ വലിക്കുന്നതും rational തന്നെ. ഒരു agent നന്നായി ചെയ്തോ എന്ന് അളക്കാൻ ഒരു performance measure വേണം, അത് designer നേരത്തേ തീരുമാനിക്കണം.

A good performance measure is: objective; measurable/observable; decided by the designer for the problem(s) the agent is built for

Rationality depends on: (1) the performance measure, (2) the agent's knowledge of the environment, (3) the length of the percept sequence, (4) the set of available actions

Rational agent: for each percept sequence, choose the action expected to maximise the performance measure, given its knowledge

### EXAMPLE

A KSRTC bus-scheduling agent is judged by "number of buses run per day". Is this a good performance measure?

It is objective and measurable, but it rewards running empty buses at 3 a.m. A better measure combines passengers carried, on-time percentage and fuel cost per km. The choice of measure changes what "rational" means: the agent does what the measure rewards.

**Exam tip:** The unit says a rational agent takes the action "which would correct its performance measure"; read this as maximise its expected performance measure.

## Rationality vs. omniscience; autonomy and learning

**Omniscience:** Knowing everything, including the actual outcome of every action in advance; this is impossible in practice, so a rational agent is not always successful.

**Omniscience:** ഓരോ action-ന്റെയും യഥാർത്ഥ ഫലം ഉൾപ്പെടെ എല്ലാം മുൻകൂട്ടി അറിയൽ; ഇത് പ്രായോഗികമായി അസാധ്യമാണ്, അതിനാൽ rational agent എപ്പോഴും വിജയിക്കണമെന്നില്ല.

**Autonomy:** The extent to which an agent relies on its own percepts rather than on the designer's prior knowledge.

**Autonomy:** Designer-ന്റെ മുൻകൂട്ടിയുള്ള knowledge-നെക്കാൾ agent സ്വന്തം percepts-നെ ആശ്രയിക്കുന്ന അളവ്.

**Learning:** Updating knowledge from experience and environmental change so as to act better in future.

**Learning:** ഭാവിയിൽ മെച്ചമായി പ്രവർത്തിക്കാൻ അനുഭവത്തിൽ നിന്നും environment-ലെ മാറ്റങ്ങളിൽ നിന്നും knowledge പുതുക്കൽ.

**Rationality** tries to maximise the outcome from current input, environmental conditions, available actions and past experience. **Omniscience** means knowing everything, including the actual future outcome of each action, which is practically impossible. So a rational agent is not always successful. **Autonomy** means relying on the agent's own percepts rather than on the designer's prior knowledge; greater autonomy makes an agent more flexible. **Learning** means updating knowledge from experience and environmental change so as to act better in future.

#### മലയാളത്തിൽ

നാളെ എന്തു സംഭവിക്കും എന്ന് അറിയാവുന്നത് omniscience ആണ്; അത് ആർക്കും ഇല്ല. കാലാവസ്ഥാ പ്രവചനം നോക്കി കൂടാതെ പോയി, പെട്ടെന്ന് മഴ പെയ്തു: അത് irrational അല്ല, ഭാവി അറിയാത്തതു കൊണ്ടാണ്. തുടക്കത്തിൽ agent designer-ന്റെ അറിവ് ഉപയോഗിക്കണം; അനുഭവം കൂടുന്നതോടെ സ്വന്തം percept-കൾ വെച്ച് പഠിച്ച് കൂടുതൽ autonomous ആകണം, പുതിയ driver ആദ്യം instructor പറയുന്നത് കേട്ട് പിന്നീട് സ്വയം ഓടിക്കുന്നതുപോലെ.

#### EXAMPLE (THE UNIT'S SOHAN SCENARIO)

Sohan walks to a nearby grocery shop. At a crossing, police chasing a terrorist fire, and a bullet hits Sohan. Was Sohan irrational?

No. Nothing in his percepts or knowledge suggested danger; he could not know the future. He was not omniscient, but his action (walking to the shop by the usual road) was rational given what he knew.

#### EXAMPLE

Why should an agent not have full autonomy on day one, yet become autonomous later?

On day one it has no experience of the environment, so the designer's built-in knowledge is its best guide. That prior knowledge may be incomplete or wrong, and the environment changes, so as experience grows the agent should learn and let its own percepts dominate. Only in rare, fully static environments is complete built-in knowledge enough.

## Example: the boundary-following robot

**Boundary-following robot:** A simple agent in a 2-D grid room that senses its eight neighbouring cells and moves along the boundaries of walls and objects.

**Boundary-following robot:** ഒരു 2-D grid മുറിയിൽ, ചുറ്റുമുള്ള എട്ട് cells sense ചെയ്ത്, ഭിത്തികളുടെയും വസ്തുക്കളുടെയും boundary-യിലൂടെ നീങ്ങുന്ന ലളിതമായ agent.

**Sense vector:** The robot's percept  $\langle c_1, \dots, c_8 \rangle$ , where  $c_i = 1$  if neighbouring cell  $i$  is occupied (or does not exist) and 0 if it is free.

**Sense vector:** Robot-ന്റെ percept ആയ  $\langle c_1, \dots, c_8 \rangle$ ; അയൽ cell  $i$  occupied ആണെങ്കിൽ (അല്ലെങ്കിൽ നിലവിലില്ലെങ്കിൽ)  $c_i = 1$ , free ആണെങ്കിൽ 0.

A problem for an agent is defined by (i) its perception capabilities, (ii) the actions it can perform, and (iii) its environment. The robot lives in a walled room viewed as a 2-D grid of cells, with some immovable objects. It



occupies one cell at a time and senses its eight neighbours  $c1 \dots c8$  (clockwise from top-left). Each  $c_i$  is 1 if occupied, 0 if free; a neighbour that does not exist (beyond a wall) is treated as **occupied**. Its only action is to move into a free neighbouring cell; moving into an occupied cell has no effect.

#### മലയാളത്തിൽ

മതിലിനോട് ചേർന്ന് വീടിന് ചുറ്റും നടക്കുന്ന ഒരു robot. അതിന് ചുറ്റുമുള്ള 8 കള്ളികൾ മാത്രമേ കാണാനാവൂ: ഒഴിഞ്ഞതാണെങ്കിൽ 0, വസ്തുവോ മതിലോ ഉണ്ടെങ്കിൽ 1. ഈ 8 bit ആണ് sense vector. അതിൽ നിന്ന് 4 features കണക്കാക്കി, features നോക്കി ഏത് action എടുക്കണമെന്ന് rules തീരുമാനിക്കുന്നു.

|    |       |    |
|----|-------|----|
| c1 | c2    | c3 |
| c8 | robot | c4 |
| c7 | c6    | c5 |

Sense vector =  $\langle c1, c2, c3, c4, c5, c6, c7, c8 \rangle$ ,  $c_i \in \{0, 1\}$ ;  $2^8 = 256$  possible percepts

$f1 = 1$  if  $c2 = 1$  OR  $c3 = 0$ , else 0

$f2 = 1$  if  $c4 = 0$  OR  $c5 = 1$ , else 0

$f3 = 1$  if  $c6 = 1$  OR  $c7 = 1$ , else 0

$f4 = 1$  if  $c8 = 1$  OR  $c1 = 1$ , else 0

Action 1:  $f1 = f2 = f3 = f4 = 0 \rightarrow$  move up to the right-most free cell (else try Action 3)

Action 2:  $f1 = 1$  AND  $f2 = 0 \rightarrow$  move to the topmost free cell on the right (else stop)

Action 3:  $f2 = 1$  AND  $f3 = 0 \rightarrow$  move down to the left-most free cell (else try Action 2)

Action 4:  $f3 = 1$  AND  $f4 = 0 \rightarrow$  move to the bottom-most free cell on the left (else try Action 1)

**Perception phase:** read  $c1 \dots c8$  and compute the features. **Action phase:** a predefined function maps the features to a move; a default move to any free neighbour is used when no rule applies. Performance measures: number of times the robot reverses its course (fewer is better), the maximum of the average distance from the walls (smaller is better), boundary distance covered in a given time (more is better).

#### EXAMPLE (SENSE VECTOR)

The robot stands in the top-right corner of the room; the cells to its left, below-left and below are free. Find the sense vector.

$c1, c2, c3$  are above the top wall: they do not exist, so 1.  $c4, c5$  are beyond the right wall: 1.  $c6$  (below),  $c7$  (below-left),  $c8$  (left) are free: 0.

Sense vector =  $\langle 1, 1, 1, 1, 1, 0, 0, 0 \rangle$ .

### EXAMPLE (UNIT'S POSITION Y)

Y is next to the left wall with nothing else around it. Check the unit's vector  $\langle 1, 0, 0, 0, 0, 0, 1, 1 \rangle$ .

The left-hand neighbours c1 (top-left), c7 (bottom-left) and c8 (left) are beyond the wall, so they are 1; the rest are free.  $\langle 1, 0, 0, 0, 0, 0, 1, 1 \rangle$  is correct.

### EXAMPLE (FEATURES → ACTION)

Compute the features and the action fired for each sense vector.

| Sense vector                                                   | f1 f2 f3<br>f4 | Rule fired                                                      |
|----------------------------------------------------------------|----------------|-----------------------------------------------------------------|
| $\langle 1, 1, 1, 1, 1, 0, 0, 0 \rangle$ (top-right corner)    | 1 1 0 1        | Action 3 (f2 = 1, f3 = 0): move down to left-most free cell, c7 |
| $\langle 0, 0, 0, 1, 0, 0, 0, 0 \rangle$ (object on the right) | 1 0 0 0        | Action 2 (f1 = 1, f2 = 0)                                       |
| $\langle 0, 0, 1, 1, 0, 0, 0, 0 \rangle$                       | 0 0 0 0        | Action 1                                                        |
| $\langle 0, 0, 0, 0, 0, 1, 0, 0 \rangle$ (object below)        | 1 1 1 0        | Action 4 (f3 = 1, f4 = 0)                                       |
| $\langle 1, 0, 0, 0, 0, 0, 1, 1 \rangle$ (position Y)          | 1 1 1 1        | none: default move                                              |

Working for row 1: f1: c2 = 1 → 1. f2: c4 = 0? no; c5 = 1? yes → 1. f3: c6 = 0, c7 = 0 → 0. f4: c8 = 0 but c1 = 1 → 1.

Row 5: f1 = 1 (c3 = 0), f2 = 1 (c4 = 0), f3 = 1 (c7 = 1), f4 = 1 (c8 = 1). No rule matches (1,1,1,1), so the default action runs.

**Exam tip:** The unit prints the sense vector at position X as  $\langle 0, 0, 0, 0, 0, 0, 0, 1, 1 \rangle$ , which has nine entries; a sense vector always has exactly eight (c1 ... c8). The unit's four rules are only illustrative and do not cover every feature combination (e.g. 1,1,1,1), which is why the default action exists. The unit's Exercise 1 depends on a figure that is not reproducible here; solve it the same way: missing neighbours = 1, objects = 1, free = 0.

## Task environment and PEAS: the automated road transport driver

**Task environment:** Everything involved in the problem that the agent is meant to solve.

**Task environment:** Agent പരിഹരിക്കേണ്ട problem-ൽ ഉൾപ്പെടുന്ന എല്ലാം.

**PEAS:** The specification of a task environment by its Performance measure, Environment, Actuators and Sensors.

**PEAS:** Performance measure, Environment, Actuators, Sensors എന്നിവ വഴി ഒരു task environment-നെ വിവരിക്കുന്ന രീതി.

The **task environment** is everything involved in the problem that the agent is meant to solve. The first step in designing an agent is to specify it as completely as possible using **PEAS**: Performance measure (expected performance), Environment (the world around the agent), Actuators (how it acts), Sensors (how it gathers information). The unit's main example is an automated public road transport driver (bus, taxi or auto driver), far more complex than the boundary robot.

### മലയാളത്തിൽ

ഒരു agent ഉണ്ടാക്കുന്നതിനു മുമ്പ് PEAS എന്ന നാല് ചോദ്യങ്ങൾക്ക് ഉത്തരം എഴുതണം: എങ്ങനെ വിലയിരുത്തും (P), ഏത് ലോകത്ത് (E), എന്തുകൊണ്ട് പ്രവർത്തിക്കും (A), എന്തുകൊണ്ട് അറിയും (S). ഒരു KSRTC bus driver-നെ robot ആക്കണമെങ്കിൽ ആദ്യം ഇവ നാലും വ്യക്തമാക്കണം.

| PEAS        | Automated public road transport driver (unit's example)                                                                                                                                                 |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Performance | Maximise passenger safety and comfort; reach the correct destination; minimise travel time; obey traffic rules; cause minimum disturbance to other agents; minimise cost                                |
| Environment | Roads from 12-lane expressways to bumpy rural roads; left-hand or right-hand traffic rules; known and unknown places; all kinds of passengers; heavy vehicles, cars, three-wheelers, even bullock carts |
| Actuators   | Steering, brakes, gears, accelerator; display screen; communication devices                                                                                                                             |
| Sensors     | Odometer, speedometer, engine sensors, GPS, distance sensors to other vehicles; sensing traffic, road condition and turns ahead                                                                         |

**Exam tip:** In PEAS answers, keep "Environment" as things outside the agent (roads, traffic, weather, passengers) and "Sensors/Actuators" as devices on the agent. Mixing "camera" into Environment or "road" into Sensors loses marks.

## PEAS for more agents

Writing PEAS is a skill best learnt by practice. Below are five agents that IGNOU-style questions resemble. Notice that the performance measure is always something measurable, and that the same physical device (a camera) is a sensor, never part of the environment.

### മലയാളത്തിൽ

ഓരോ agent-നും PEAS table ഉണ്ടാക്കി പരിശീലിക്കുക. KSEB smart meter, Malayalam chatbot പോലെ നമ്മുടെ നാട്ടിലെ ഉദാഹരണങ്ങൾ ഓർക്കാൻ എളുപ്പമാണ്. Performance-ൽ എപ്പോഴും അളക്കാവുന്ന കാര്യങ്ങൾ എഴുതുക: "നല്ല സേവനം" എന്നല്ല, "പരാതികളുടെ എണ്ണം കുറയ്ക്കുക" എന്ന്.

| Agent                                              | Performance                                                                                                  | Environment                                                                         | Actuators                                                                        | Sensors                                                                    |
|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Self-driving car                                   | Safety (no accidents), legal driving, trip time, fuel/charge used, passenger comfort                         | Roads, other vehicles, pedestrians, animals, signals, weather, potholes             | Steering, accelerator, brake, indicators, horn, passenger display                | Cameras, LiDAR, radar, GPS, IMU, speedometer, ultrasonic parking sensors   |
| KSEB smart-meter agent (home energy)               | Accurate readings, lower monthly bill, less load in peak hours, fast tamper/leak detection, no missed alerts | Home wiring and appliances, residents, KSEB grid, time-of-day tariff, power cuts    | Relays to switch loads (pump, water heater), SMS/app alerts, data upload to KSEB | Current and voltage sensors, clock, tariff signal from KSEB, tamper sensor |
| Malayalam customer-care chatbot (e.g. bus enquiry) | Correct answers, queries solved without a human, response time, user rating, correct Malayalam               | Users typing Malayalam or Manglish, timetable/booking database, human support staff | Text replies on screen, speech output, booking API calls, hand-off to a human    | Keyboard/text input, microphone with speech recogniser                     |
| Vacuum cleaner (two-square world)                  | Number of clean squares per time step; fewer moves (energy)                                                  | Squares A and B, dirt                                                               | Wheels (Left, Right), suction (Suck)                                             | Location sensor, dirt sensor                                               |
| Chess-playing agent                                | Win (1), draw ( $\frac{1}{2}$ ), loss (0); moves within time control                                         | Chess board, opponent, clock                                                        | Display move on screen or robotic arm                                            | Board input (keyboard/camera), clock                                       |

### EXAMPLE

Write PEAS for a medical diagnosis system (a typical IGNOU question).

P: correct diagnosis, healthy patient, minimum cost and tests, no lawsuits. E: patient, hospital, doctors and staff. A: screen display of questions, tests, diagnoses, treatments, referrals. S: keyboard entry of symptoms, findings and patient answers; lab reports.

## Environment types 1: observable, static/dynamic, deterministic/stochastic

**Fully observable:** The sensors give the complete relevant state of the environment at every moment; otherwise it is partially observable.

**Fully observable:** ഓരോ നിമിഷവും sensors environment-ന്റെ പ്രസക്തമായ പൂർണ്ണ state നൽകുന്നു; അല്ലെങ്കിൽ അത് partially observable ആണ്.

**Static / dynamic:** A static environment does not change while the agent is deciding; a dynamic one changes over time.

**Static / dynamic:** Agent തീരുമാനമെടുക്കുന്നതിനിടെ static environment മാറുന്നില്ല; dynamic environment സമയത്തിനനുസരിച്ച് മാറുന്നു.

**Deterministic:** The next state is completely determined by the current state and the agent's action; otherwise the environment is stochastic.

**Deterministic:** അടുത്ത state നിലവിലെ state-ഉം agent-ന്റെ action-ഉം ചേർന്ന് പൂർണ്ണമായി നിർണ്ണയിക്കുന്നു; അല്ലെങ്കിൽ environment stochastic ആണ്.

**Fully observable:** the sensors give the complete relevant state of the world at every moment, so no internal state is needed; rare in reality. **Partially observable:** only some aspects can be sensed (limited sensors, noise). The boundary robot sees only its 8 neighbours; the road driver cannot see potholes a kilometre ahead or other drivers' intentions: both are partially observable.

**Static:** the environment does not change while the agent is deciding (crossword puzzle, boundary robot).

**Dynamic:** it changes on its own or through the agent's actions (road traffic). **Semi-dynamic:** the environment itself does not change with time, but the agent's performance score does (chess played with a clock).

**Deterministic:** the current state and the agent's action completely determine the next state (boundary robot). Otherwise **stochastic** (road driver: traffic is unpredictable). The judgement is made from the agent's point of view. If the only uncertainty is the other player's moves, the environment is called **strategic** (tic-tac-toe, chess).

#### മലയാളത്തിൽ

Fully observable: ക്യാരംസ് board പോലെ എല്ലാം മുന്നിൽ കാണാം. Partially observable: ചിട്ടകളിൽ എതിരാളിയുടെ കൈയിലെ ചീട്ട് കാണാനാവില്ല. Static: crossword ആലോചിക്കുന്നതിനിടെ ചോദ്യം മാറില്ല. Dynamic: MG Road-ലെ traffic നമ്മൾ ആലോചിക്കുമ്പോഴും മാറിക്കൊണ്ടിരിക്കും. Deterministic: switch ഇട്ടാൽ fan കറങ്ങും എന്ന് ഉറപ്പ്. Stochastic: ബസ് സ്റ്റോപ്പിൽ ബസ് എപ്പോൾ വരുമെന്ന് ഉറപ്പില്ല.

Fully observable  $\Rightarrow$  sensors show the complete relevant state (no memory needed)

Static: no change while deciding; Dynamic: changes over time; Semi-dynamic: world fixed, score changes with time

Deterministic: next state =  $f(\text{current state, action})$ ; Stochastic: next state uncertain; Strategic: deterministic except for other agents' actions

#### EXAMPLE

Classify the KSEB smart-meter agent on these three dimensions.

Partially observable: it sees total current, not which appliance is on or what the residents plan. Dynamic: consumption and grid voltage change whether or not the agent acts. Stochastic: when residents switch on the geyser or a power cut occurs cannot be predicted from the current state and action.

**Exam tip:** The unit defines semi-dynamic as "the environment does not change by itself, but an agent's performance changes the environment". The standard meaning (Russell & Norvig, the unit's reference) is: the environment does not change with time, but the agent's performance score does, e.g. chess with a clock. Use this definition.

## Environment types 2: episodic/sequential, discrete/continuous

**Episodic:** The agent's experience is divided into independent episodes, so an action in one episode does not affect others; otherwise it is sequential.

**Episodic:** Agent-ന്റെ അനുഭവം സ്വതന്ത്രമായ episodes ആയി വിഭജിക്കപ്പെടുന്നു, അതിനാൽ ഒരു episode-ലെ action മറ്റുള്ളവയെ ബാധിക്കുന്നില്ല; അല്ലെങ്കിൽ അത് sequential ആണ്.

**Discrete:** The environment has a finite number of distinct states, percepts and actions; if these vary smoothly, it is continuous.

**Discrete:** Environment-ന് പരിമിതമായ എണ്ണം വ്യത്യസ്ത states, percepts, actions മാത്രമേ ഉള്ളൂ; ഇവ തുടർച്ചയായി മാറുന്നുവെങ്കിൽ അത് continuous ആണ്.

**Episodic:** the agent's experience splits into independent episodes; the action in one episode does not depend on, or affect, other episodes. A ticket checker at a cinema hall, or a robot checking bottle seals on an assembly line, works episodically. **Sequential:** previous actions affect the current decision and the current action affects future ones (road driver, chess). Episodic is simpler because the agent need not think ahead.

**Discrete:** a finite number of distinct states, percepts and actions, with time in slots (chess, boundary robot).

**Continuous:** states and time vary smoothly (driving: speed, position, steering angle). A driver's sensors may sample at discrete intervals, but so often that the data is treated as continuous.

### മലയാളത്തിൽ

**Episodic:** റേഷൻ കടയിൽ ഓരോ കാർഡും വെച്ചേറെ പരിശോധിക്കുന്നു, മുമ്പത്തെ ആളിന്റെ കാർഡ് ഇപ്പോഴത്തേതിനെ ബാധിക്കില്ല. **Sequential:** ചെസ്സിൽ ഇപ്പോഴത്തെ ഒരു നീക്കം കളി മുഴുവൻ മാറ്റും. **Discrete:** ചെസ്സ് board-ൽ 64 കളികൾ മാത്രം. **Continuous:** വണ്ടിയുടെ speed 40.0, 40.1, 40.15 ... എന്നിങ്ങനെ തുടർച്ചയായി മാറുന്നു.

### EXAMPLE

Is a spam filter episodic or sequential? Discrete or continuous?

**Episodic:** each email is classified on its own; marking one mail as spam does not change the correct label of the next (ignoring learning). **Discrete:** the percept is a finite text and the action set is {spam, not spam}.



**Exam tip:** The unit ties discrete/continuous only to how time is treated, but its own chess argument uses the finite number of states. Mention both: the distinction applies to states, time, percepts and actions. Also note the unit's list in 10.4.3 names five properties but then discusses six (discrete vs. continuous is the sixth).

## Environment types 3: single-agent vs. multi-agent

**Single-agent:** The agent is the only active entity in the environment.

**Single-agent:** Environment-ൽ സജീവമായ ഒരേയൊരു entity ആ agent മാത്രമാണ്.

**Multi-agent:** Other agents also act in the environment; it is cooperative if they share a common goal and competitive if they compete.

**Multi-agent:** മറ്റ് agents-ഉം environment-ൽ പ്രവർത്തിക്കുന്നു; അവർക്ക് പൊതുവായ goal ആണെങ്കിൽ cooperative, പരസ്പരം മത്സരിക്കുന്നുവെങ്കിൽ competitive.

In a **single-agent** environment the agent is the only active entity and need not synchronise with anyone (boundary robot, crossword solver). In a **multi-agent** environment other agents exist, and the agent must track their behaviour. If all agents work towards a common goal it is **cooperative**; if they compete (tic-tac-toe, chess, where each player predicts the other's moves) it is **competitive**. Two key issues: deciding whether another entity is a passive object or an agent (normally, an agent if it optimises a performance measure that affects ours; there is no fixed rule), and deciding how agents will communicate.

### മലയാളത്തിൽ

Crossword തനിയെ ചെയ്യുന്നത് single-agent. വള്ളംകളിയിൽ ഒരേ വള്ളത്തിലെ തുഴക്കാർ cooperative multi-agent; രണ്ട് വള്ളങ്ങൾ തമ്മിൽ competitive. റോഡിലെ വണ്ടികൾ രണ്ടും ചേർന്നതാണ്: അപകടം ഒഴിവാക്കാൻ സഹകരിക്കുന്നു, parking space-ന് വേണ്ടി മത്സരിക്കുന്നു. വഴിയിലെ ഒരു കല്ല് agent അല്ല, വെറും object; മറ്റൊരു driver സ്വന്തം ലക്ഷ്യം നേടാൻ ശ്രമിക്കുന്നതുകൊണ്ട് agent ആണ്.

### EXAMPLE

Classify: (a) a fleet of warehouse robots sharing shelves, (b) an online auction bidding agent, (c) self-driving cars at a junction.

(a) Cooperative multi-agent: all robots share the goal of fast order picking; they must communicate to avoid collisions. (b) Competitive multi-agent: each bidder wants the item at the lowest price. (c) Partly cooperative (avoid collisions, follow signals) and partly competitive (each wants to pass first).

**Exam tip:** For medical diagnosis the unit chooses single-agent "to keep things simple", but says it becomes multi-agent if patients or staff are treated as agents maximising their own measure (e.g. profitability). Say this when asked.

## Classifying task environments (the unit's five examples and more)

The unit classifies five task environments. Crossword: single-agent, static, sequential, fully observable, deterministic, discrete. Medical diagnosis: single-agent (by choice), partially observable, stochastic, partly episodic (each patient) and partly sequential (treatment of one patient over time), continuous. Tic-tac-toe: multi-agent competitive, fully observable, strategic, semi-dynamic, discrete, sequential. Chess: same as tic-tac-toe, but strictly partially observable because some rules need the game history. Automobile driver: multi-agent, partially observable, dynamic, stochastic, sequential, continuous; the hardest.

### മലയാളത്തിൽ

Exam-ൽ ഒരു agent തന്നെ ആറ് property-കളും ചോദിക്കും. ഓരോ വരിയും ഒരു കാരണം സഹിതം എഴുതിയാൽ മുഴുവൻ മാർക്ക് കിട്ടും. താഴെയുള്ള table ഒന്ന് കാണാതെ പഠിക്കുക; driver ഏറ്റവും ബുദ്ധിമുട്ടുള്ളതും crossword ഏറ്റവും എളുപ്പമുള്ളതും ആണെന്ന് ഓർക്കുക.

| Task environment                     | Observable                            | Agents                   | Deterministic? | Episodic?                | Static?                            | Discrete?  |
|--------------------------------------|---------------------------------------|--------------------------|----------------|--------------------------|------------------------------------|------------|
| Crossword puzzle                     | Fully                                 | Single                   | Deterministic  | Sequential               | Static                             | Discrete   |
| Medical diagnosis                    | Partially                             | Single (or multi)        | Stochastic     | Episodic + sequential    | Dynamic                            | Continuous |
| Tic-tac-toe                          | Fully                                 | Multi, competitive       | Strategic      | Sequential               | Static (semi-dynamic with a clock) | Discrete   |
| Chess (with clock)                   | Partially (unit) / Fully (board view) | Multi, competitive       | Strategic      | Sequential               | Semi-dynamic                       | Discrete   |
| Automobile driver / self-driving car | Partially                             | Multi (coop. + comp.)    | Stochastic     | Sequential               | Dynamic                            | Continuous |
| KSEB smart-meter agent               | Partially                             | Single                   | Stochastic     | Sequential               | Dynamic                            | Continuous |
| Malayalam chatbot                    | Partially                             | Multi (user is an agent) | Stochastic     | Sequential within a chat | Dynamic                            | Discrete   |
| Vacuum cleaner (two squares)         | Partially (senses own square)         | Single                   | Deterministic  | Sequential               | Static                             | Discrete   |

#### EXAMPLE

Justify each entry for the Malayalam chatbot.

Partially observable: the user's real intent and context are hidden; Manglish spelling makes it worse. Multi-agent: the user is another agent with goals. Stochastic: the next user message cannot be predicted from the bot's reply. Sequential: an earlier question ("Thrissur-il ninnu?") changes how "next bus eppol?" must be answered. Dynamic: the user may type again or leave while the bot is still processing. Discrete: messages and replies are finite strings.

**Exam tip:** The unit calls tic-tac-toe semi-dynamic because "the performance score changes with time"; that holds only when moves are timed. Without a clock, tic-tac-toe is static. For chess, the unit says "strictly partially observable" because castling, en passant and draw-by-repetition need history not visible on the board; many textbooks call chess fully observable. Write the unit's answer and give the reason.

## Structure of an agent: architecture + program; table-driven agents

**Architecture:** The hardware device, with sensors and actuators, on which the agent runs (Agent = Architecture + Program).

**Architecture:** Agent പ്രവർത്തിക്കുന്ന, sensors-ഉം actuators-ഉം ഉള്ള hardware device (Agent = Architecture + Program).

**Agent program:** The program that maps percepts to actions.

**Agent program:** Percepts-നെ actions-ലേക്ക് map ചെയ്യുന്ന program.

**Table-driven agent:** An agent that looks up its action for every possible percept sequence in a stored table.

**Table-driven agent:** സാധ്യമായ ഓരോ percept sequence-നുമുള്ള action ഒരു സൂക്ഷിച്ചുവെച്ച table-ൽ നോക്കി കണ്ടെത്തുന്ന agent.

An agent has two parts: the **architecture**, the hardware device with sensors and actuators on which the agent runs, and the **agent program**, which maps percepts to actions. They must match: if the boundary robot loses its right-side sensors, its program must change. The **table-driven** approach stores the action for every possible percept sequence in a table. It works in principle, but the table grows explosively, so compact agent programs are far more efficient.

### മലയാളത്തിൽ

Agent = architecture + program. Architecture ശരീരം പോലെ (sensors, motors), program തലച്ചോറ് പോലെ. ഓരോ സാധ്യതയ്ക്കും ഉത്തരം ഒരു വലിയ table-ൽ എഴുതിവെക്കാം, പക്ഷേ അത് കാണാപ്പാഠം പഠിക്കുന്നതുപോലെയാണ്: ചോദ്യങ്ങൾ കൂടുമ്പോൾ table താങ്ങാനാവാത്തത്ര വലുതാകും.

Agent = Architecture + Program

Table-driven agent: table entries =  $|P| + |P|^2 + \dots + |P|^T$  ( $|P|$  = number of possible percepts,  $T$  = lifetime in steps)

### EXAMPLE

The two-square vacuum cleaner has 4 possible percepts: (A, Clean), (A, Dirty), (B, Clean), (B, Dirty). How many table entries does a table-driven agent need for a lifetime of 3 steps? For the boundary robot, for just 2 steps?

Vacuum:  $4 + 4^2 + 4^3 = 4 + 16 + 64 = 84$  entries.

Boundary robot:  $|P| = 2^8 = 256$ , so  $256 + 256^2 = 256 + 65\,536 = 65\,792$  entries for only two steps. A 4-rule program does the same job, which is why agent programs beat tables.

## Simple reflex (SR) agents

**Definition:** A simple reflex agent has no internal state and selects actions by condition-action rules on the current percept only.

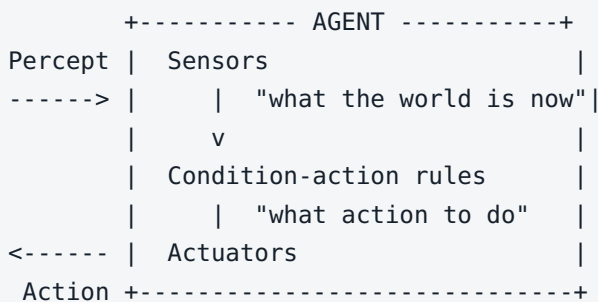
Simple reflex agent-ന് internal state ഇല്ല; നിലവിലെ percept-നെ മാത്രം ആശ്രയിച്ച് condition-action rules വഴി അത് action തിരഞ്ഞെടുക്കുന്നു.

A **simple reflex agent** has no internal state: it remembers nothing and reacts only to the current percept, using **condition-action rules**. One of the earliest was **Machina Speculatrix**, a wheeled device with motor, photocells and vacuum tubes that moved towards dimmer light and avoided bright light. The boundary-following robot is an SR agent, and a car's emergency braking is SR behaviour. SR agents are simple but have very limited intelligence: they cannot use past experience, do not learn, and work correctly only if the environment is **fully observable**.

### മലയാളത്തിൽ

SR agent "കണ്ടാൽ ഉടൻ ചെയ്യുക" എന്ന നിയമം മാത്രം പാലിക്കുന്നു: ഓർമ്മയില്ല.

Automatic street light ഇരുട്ടായാൽ കത്തും, വെളിച്ചം വന്നാൽ അണയും; ഇന്നലെ എന്തുണ്ടായി എന്ന് അതിന് അറിയില്ല. പക്ഷേ എല്ലാം ഒറ്റനോട്ടത്തിൽ കാണാൻ പറ്റാത്ത ഇടത് ഇത് തെറ്റും.



IF condition (on current percept) THEN action

e.g. IF a human suddenly comes in front of the car THEN apply brakes immediately

#### EXAMPLE (VACUUM-WORLD TRACE, VERIFIED BY SIMULATION)

Two squares A and B, both dirty; the agent starts in A. SR program: IF Dirty THEN Suck; ELSE IF in A THEN Right; ELSE Left. Performance: +1 for each clean square after each step. Trace 6 steps.

| Step | Percept    | Action | After : location , A , B | Points (running total) |
|------|------------|--------|--------------------------|------------------------|
| 1    | (A, Dirty) | Suck   | A, Clean, Dirty          | 1 (1)                  |
| 2    | (A, Clean) | Right  | B, Clean, Dirty          | 1 (2)                  |
| 3    | (B, Dirty) | Suck   | B, Clean, Clean          | 2 (4)                  |
| 4    | (B, Clean) | Left   | A, Clean, Clean          | 2 (6)                  |
| 5    | (A, Clean) | Right  | B, Clean, Clean          | 2 (8)                  |
| 6    | (B, Clean) | Left   | A, Clean, Clean          | 2 (10)                 |

Both squares are clean after step 3, but the SR agent keeps shuttling Left/Right forever: it cannot remember that A is already clean. If each move costs 1 point, its score over 6 steps is 10 – 4 moves = 6.

## Model-based reflex agents

**Definition:** A model-based reflex agent keeps an internal state, updated using a model of how the world evolves and how its actions affect it, to handle partial observability.

Model-based reflex agent ഒരു internal state സൂക്ഷിക്കുന്നു; world എങ്ങനെ മാറുന്നു, സ്വന്തം actions അതിനെ എങ്ങനെ ബാധിക്കുന്നു എന്ന model ഉപയോഗിച്ച് അത് പുതുക്കി partial observability കൈകാര്യം ചെയ്യുന്നു.

To handle **partially observable** environments, the agent keeps an **internal state**, a function of the percept sequence so far, that reflects the parts of the world it cannot currently see. To update that state it needs a **model** of the environment encoded in its program, containing knowledge of (i) how the world evolves independently of the agent, and (ii) how the agent's own actions affect the world. Reflex rules plus this model give a model-based reflex agent.

#### മലയാളത്തിൽ

Rear-view mirror-ൽ ഒരു വണ്ടി കണ്ടു, പിന്നെ അത് blind spot-ൽ മറഞ്ഞു. കാണുന്നില്ലെങ്കിലും അത് അവിടെ ഉണ്ടെന്ന് നമ്മൾ ഓർക്കുന്നു: ഇതാണ് internal state. "വണ്ടികൾ മുന്നോട്ട് നീങ്ങും", "ഞാൻ steering തിരിച്ചാൽ lane മാറും" എന്ന അറിവാണു് model.

```
Percept --> Sensors --> [ current state of the world ]
                        ^
                        | uses: internal state,
                        | how the world evolves,
                        | effect of my actions
                        v
                        Condition-action rules --> Actuators --> Action
```

```
state ← UPDATE-STATE(state, last action, percept, model)
rule ← RULE-MATCH(state, rules)
action ← rule.action
```

#### EXAMPLE (VACUUM WORLD WITH MEMORY, VERIFIED BY SIMULATION)

Same start (A and B dirty, agent in A). The agent remembers the status of each square it has seen or cleaned, and does NoOp once it knows both are clean.

| Step | Percept    | Internal state after update       | Action | Points (total) |
|------|------------|-----------------------------------|--------|----------------|
| 1    | (A, Dirty) | A = Clean (after Suck), B = ?     | Suck   | 1 (1)          |
| 2    | (A, Clean) | A = Clean, B = ?                  | Right  | 1 (2)          |
| 3    | (B, Dirty) | A = Clean, B = Clean (after Suck) | Suck   | 2 (4)          |
| 4    | (B, Clean) | A = Clean, B = Clean              | NoOp   | 2 (6)          |
| 5    | (B, Clean) | A = Clean, B = Clean              | NoOp   | 2 (8)          |
| 6    | (B, Clean) | A = Clean, B = Clean              | NoOp   | 2 (10)         |

Clean-square score is the same (10), but with a move cost of 1 it scores  $10 - 1 = 9$  against the SR agent's 6. Memory of the unseen square made it better. (If dirt could reappear, the model would also need "how the world evolves", e.g. re-check every few steps.)

## Goal-based agents

**Definition:** A goal-based agent chooses actions using goal information along with the current state, by predicting which actions will lead to the goal.

Goal-based agent നിലവിലെ state-നൊപ്പം goal information-ഉം ഉപയോഗിച്ച്, ഏത് actions goal-ലേക്ക് എത്തിക്കുമെന്ന് പ്രവചിച്ച് action തിരഞ്ഞെടുക്കുന്നു.

Knowing the current state is sometimes not enough: driving from city A to city B, knowing you are now at C does not tell you which fork to take; you must also know the **goal**, B. A **goal-based agent** chooses actions using goal information plus the current state. It looks into the future ("what will happen if I do this?"), can retract steps that led away from the goal, and reasons in an if-then-else way. It may be slower than an SR agent, but it is more flexible: when the goal changes, only the goal needs changing, not all the condition-action rules.

### മലയാളത്തിൽ

Kochi-യിൽ നിന്ന് Kozhikode-ലേക്ക് പോകുമ്പോൾ Thrissur-ൽ എത്തിയെന്ന് അറിഞ്ഞാൽ പോരാ, എവിടേക്കാണ് പോകേണ്ടത് എന്നും അറിയണം; അപ്പോഴേ ഏത് വഴി തിരിയണം എന്ന് തീരുമാനിക്കാൻ പറ്റും. Goal-based agent ഓരോ action-ന്റെയും ഫലം മുൻകൂട്ടി ആലോചിക്കുന്നു. Destination മാറ്റിയാൽ Google Maps പുതിയ വഴി തരുന്നതുപോലെ, goal മാറ്റിയാൽ മതി.

```
Percept --> [ current state ] --> "what will the state be if I do action X?"
                                     |
                                     [ Goals ] -----> choose X that reaches the goal -->
Action
```

### EXAMPLE (THE UNIT'S BRAKING EXAMPLE)

A driver agent has the goal "do not hit the vehicle in front". The vehicle ahead slows down. How does a goal-based agent differ from an SR agent?

SR: IF car ahead brakes THEN brake hard. Goal-based: it predicts that braking hard may cause the car behind to hit it, which also violates safety, so it brakes gradually, keeping both gaps safe. It chose the action by predicting outcomes against its goal.



### EXAMPLE

A goal-based route agent is at a fork: left leads to Kozhikode in 180 km, right leads to Palakkad. Goal: Kozhikode. What does it do, and what can it not decide?

It takes the left fork because that reaches the goal. If there were two roads to Kozhikode (180 km and 195 km), both reach the goal, so a pure goal-based agent has no way to prefer the shorter one. This is the gap utility-based agents fill.

## Utility-based agents

**Definition:** A utility-based agent uses a utility function that maps each state to a real number (its degree of success) and chooses the action leading to the highest-utility state.

Utility-based agent ഓരോ state-നെയും ഒരു real number-ലേക്ക് (വിജയത്തിന്റെ അളവ്) map ചെയ്യുന്ന utility function ഉപയോഗിച്ച്, ഏറ്റവും ഉയർന്ന utility ഉള്ള state-ലേക്ക് നയിക്കുന്ന action തിരഞ്ഞെടുക്കുന്നു.

A goal-based agent only knows "goal reached or not". It cannot handle **conflicting goals**, has no notion of **partial success**, and cannot choose the faster or cheaper of several ways to reach a goal. A **utility-based agent** uses a **utility function** that maps each world state to a number measuring its degree of success ("happiness"). Among the reachable next states it chooses the one with the highest utility. With conflicting or multiple goals it picks the one with higher utility or least cost.

### മലയാളത്തിൽ

രണ്ട് ബസുകളും Kozhikode-ൽ എത്തും; ഒന്ന് fast passenger, ഒന്ന് ordinary. Goal-based agent-ന് രണ്ടും ഒരുപോലെ. Utility-based agent സമയം, കൂലി, സൗകര്യം എന്നിവയ്ക്ക് number കൊടുത്ത് ഏറ്റവും കൂടിയ score ഉള്ളത് തിരഞ്ഞെടുക്കുന്നു.

Utility function  $U$ : state  $\rightarrow$  real number (degree of success)

Choose action  $a$  that leads to the reachable state  $s$  with maximum  $U(s)$

### EXAMPLE (KSEB SMART-METER AGENT)

The agent must fill the water tank tonight by running a 1 kW pump for 2 hours. Assume a time-of-day tariff of ₹6.25/unit at 7 p.m. (peak) and ₹4.50/unit at 11 p.m. (off-peak). The household rates the inconvenience of water arriving late at ₹2. Utility  $U = -(\text{electricity cost}) - (\text{inconvenience cost})$ . Which slot is chosen?

Energy = 1 kW × 2 h = 2 units.

7 p.m.: cost =  $2 \times 6.25 = ₹12.50$ , inconvenience 0 →  $U = -12.50$ .

11 p.m.: cost =  $2 \times 4.50 = ₹9.00$ , inconvenience ₹2 →  $U = -11.00$ .

$-11.00 > -12.50$ , so the agent runs the pump at 11 p.m. A goal-based agent sees both as "tank full" and could not choose. If the family rated inconvenience at ₹4,  $U(11 \text{ p.m.}) = -13.00$  and 7 p.m. would win: the utility function captures preferences.

## Learning agents and their four components

**Definition:** A learning agent starts with some initial knowledge and improves its performance from experience; its four components are the learning component, performance element, critic and problem generator.

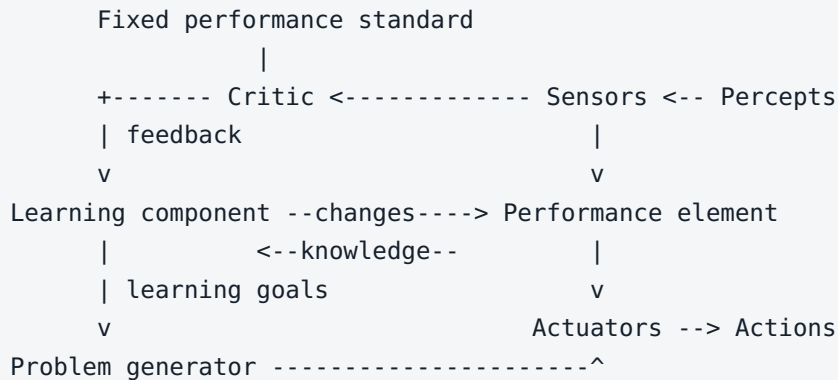
Learning agent കറച്ച് initial knowledge-ൽ തുടങ്ങി അനുഭവത്തിൽ നിന്ന് performance മെച്ചപ്പെടുത്തുന്നു; learning component, performance element, critic, problem generator എന്നിവയാണ് അതിന്റെ നാല് components.

All knowledge cannot be coded in advance, especially in real, dynamic, multi-agent environments (a pedestrian steps out even on a green signal). Success in unknown environments depends on the ability to **learn**: start with some initial knowledge and improve it from experience. A learning agent has four components:

- (i) Learning component: uses percepts and feedback to gradually improve performance
- (ii) Performance component (element): selects all actions, from percepts + knowledge from the learning component
- (iii) Critic: judges how well the agent is doing against a fixed performance standard; drives changes to the performance component
- (iv) Problem generator: suggests new, possibly sub-optimal actions to gain fresh experience (exploration)

#### മലയാളത്തിൽ

ഡ്രൈവിംഗ് പഠിക്കുന്ന ഒരാളെ ഓർക്കുക. വണ്ടി ഓടിക്കുന്നത് performance element. അടുത്തിരിക്കുന്ന instructor "clutch നേരത്തേ വിട്ടു" എന്ന് പറയുന്നത് critic. ആ അഭിപ്രായം വെച്ച് ശീലം മാറ്റുന്നത് learning component. "ഇന്ന് ഒരു കയറ്റമുള്ള റോഡിൽ ശ്രമിച്ചുനോക്കാം" എന്ന് പുതിയ അനുഭവം നിർദ്ദേശിക്കുന്നത് problem generator.



#### EXAMPLE

Map the four components for the Malayalam chatbot.

Performance element: the dialogue model that writes replies. Critic: compares user ratings and "was the query resolved?" against the target (e.g. 90% resolved). Learning component: retrains on chats the critic flagged, e.g. learns that "vandi" in Manglish means "bus" in this context. Problem generator: occasionally tries a new phrasing or asks a clarifying question to find out whether it works better.

**Exam tip:** The learning component and the performance element are designed together: learning is useless unless the performance element can turn new knowledge into better actions. The problem generator's sub-optimal actions look bad in the short run but may be better in the long run. Also, 10.5 lists only four agent categories, but 10.5.5 adds learning agents as a fifth; answer "five types".

## Forms of learning: rote, instruction, analogy, induction, deduction

**Rote learning:** Knowledge is copied directly into the knowledge base, with the least inference.

**Rote learning:** ഏറ്റവും കുറഞ്ഞ inference-ഓടെ knowledge നേരിട്ട് knowledge base-ലേക്ക് പകർത്തുന്നു.

**Learning by induction:** Drawing a general conclusion from many positive examples; usually right, but it can be refuted.

**Learning by induction:** ധാരാളം positive examples-ൽ നിന്ന് ഒരു പൊതു നിഗമനത്തിൽ എത്തൽ; സാധാരണയായി ശരിയാണ്, പക്ഷേ തെറ്റാണെന്ന് തെളിയിക്കപ്പെടാം.

**Learning by deduction:** Irrefutable inference that is always correct whenever its premises are correct.

**Learning by deduction:** Premises ശരിയാണെങ്കിൽ എപ്പോഴും ശരിയാകുന്ന, നിഷേധിക്കാനാവാത്ത inference.

Learning taxonomies can be built on the type of knowledge learnt (concepts, problem solving, game playing), the representation (predicate calculus, rules, frames) or the application area (diagnosis, scheduling, prediction). A classification independent of representation gives five methods, in increasing amount of inference:

| Method                       | Idea                                                                         | Example                                                                                     |
|------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Rote learning                | Knowledge copied directly into the knowledge base; least inference           | Memorising multiplication tables                                                            |
| Learning through instruction | Knowledge from a teacher, integrated into existing knowledge; more inference | A pupil learning from a teacher                                                             |
| Learning by analogy          | New concepts from known similar ones                                         | Solving exercises after worked examples; driving a lorry after learning on a car            |
| Learning by induction        | General conclusion from many positive examples; usually right but refutable  | After seeing many cows: "a cow has four legs, two horns"; a three-legged cow refutes it     |
| Learning by deduction        | Irrefutable inference: correct whenever the premises are correct             | Mathematics: all squares are rectangles, ABCD is a square $\Rightarrow$ ABCD is a rectangle |

### മലയാളത്തിൽ

ഗുണനഷ്ടിക കാണാതെ പഠിക്കുന്നത് rote. ടീച്ചർ പറഞ്ഞുതന്നത് instruction. Two-wheeler ഓടിച്ച അറിവ് വെച്ച് scooty ഓടിക്കുന്നത് analogy. കണ്ട കാക്കകളെല്ലാം കറുപ്പാണ്, അതുകൊണ്ട് എല്ലാ കാക്കയും കറുപ്പ് എന്ന് പറയുന്നത് induction: ഒരു വെളുത്ത കാക്കയെ കണ്ടാൽ തെറ്റും. കണക്കിലെ proof പോലെ premises ശരിയെങ്കിൽ ഉത്തരം ഉറപ്പായും ശരിയാകുന്നത് deduction.

Inductive learning is the most important for designing an agent's learning component. Learning in an agent depends on the subject matter (concepts, problem solving, games), the representation used (predicate calculus, frames, scripts) and the critic's feedback.

### EXAMPLE

Name the form of learning: (a) a spell-checker stores a new word the user adds; (b) a smart meter sees 30 months where the bill rose in April and concludes "bills peak in summer"; (c) from "every KSEB bill above 250 units is charged at the higher slab" and "this bill is 300 units", it concludes the higher slab applies.

(a) Rote learning. (b) Induction (could fail in a cool year). (c) Deduction (certain, given the premises).

## Supervised, unsupervised and reinforcement learning

**Supervised learning:** Learning a function from example input-output pairs.

**Supervised learning:** ഉദാഹരണ input-output pairs-ൽ നിന്ന് ഒരു function പഠിക്കൽ.

**Unsupervised learning:** Learning relevant properties or patterns from data when no input-output pairs are given.

**Unsupervised learning:** Input-output pairs നൽകാതെ data-യിൽ നിന്ന് പ്രസക്തമായ properties അല്ലെങ്കിൽ patterns സ്വയം കണ്ടെത്തൽ.

**Reinforcement learning:** Learning an action policy that maximises total (discounted) reward from occasional positive or negative rewards.

**Reinforcement learning:** ഇടയ്ക്കിടെ ലഭിക്കുന്ന positive അല്ലെങ്കിൽ negative rewards-ൽ നിന്ന്, മൊത്തം (discounted) reward പരമാവധിയാക്കുന്ന action policy പഠിക്കൽ.

Learning based on feedback is of three kinds. **Supervised learning** learns a function from example input-output pairs (e.g. learning a mapping from state conditions to actions, or how the world evolves). In **unsupervised learning** no input-output pairs are given; the system must find relevant properties on its own. In **reinforcement learning** the task may not be statable and may be ongoing; the user occasionally gives positive or negative rewards, and the agent learns an **action policy** to maximise total reward. For never-ending tasks the total may be

infinite, so **future rewards are discounted**. When rewards depend on a sequence of earlier actions, it is **delayed-reinforcement learning**.

#### മലയാളത്തിൽ

Supervised: ടീച്ചർ ഉത്തരം സഹിതം ചോദ്യങ്ങൾ തരുന്നു. Unsupervised: ഉത്തരമില്ലാതെ, കുട്ടി സ്വയം patterns കണ്ടെത്തുന്നു, ഉദാഹരണത്തിന് മീൻ ചന്തയിലെ മീനുകളെ പേരിടാതെ സാമ്യം നോക്കി കൂട്ടങ്ങളാക്കുന്നത്. Reinforcement: പട്ടിക്കുട്ടിയെ പരിശീലിപ്പിക്കുന്നതുപോലെ, നല്ലത് ചെയ്താൽ biscuit, തെറ്റിയാൽ ശകാരം. ഇപ്പോൾ കിട്ടുന്ന reward-ന് പിന്നീട് കിട്ടുന്നതിനേക്കാൾ വില കൂടുതലാണ്: അതാണ് discounting.

Discounted return =  $r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$ ,  $0 \leq \gamma < 1$

Constant reward  $r$  forever:  $r / (1 - \gamma)$

#### EXAMPLE

A driving agent gets reward 10 every step forever. (a) Why is plain "total reward" useless? (b) Find the discounted return with  $\gamma = 0.9$ . (c) Should it prefer reward 5 now or 8 three steps later?

(a)  $10 + 10 + 10 + \dots = \infty$ , so every policy that earns forever looks equally good.

(b)  $10 / (1 - 0.9) = 10 / 0.1 = 100$ .

(c) 8 three steps later is worth  $8 \times 0.9^3 = 8 \times 0.729 = 5.832 > 5$ , so it waits. With  $\gamma = 0.8$ :  $8 \times 0.512 = 4.096 < 5$ , so it takes 5 now.

#### EXAMPLE

Classify: (a) training a chatbot on 10 000 Malayalam questions each labelled with its correct intent; (b) grouping KSEB consumers by usage pattern with no labels; (c) a chess agent that only learns "won" or "lost" at the end of each game.

(a) Supervised. (b) Unsupervised (clustering). (c) Reinforcement, and specifically delayed reinforcement, because the reward depends on the whole sequence of moves.

**Exam tip:** The unit's example of unsupervised learning, "finding the shortest path between two cities in an unknown country", is really exploration/search rather than unsupervised learning. In an answer, prefer a clustering example (grouping customers or documents without labels) and mention the unit's example only if asked for it.

## Quick recap

- Agent: perceives through sensors, acts through actuators; percept = one input, percept sequence = full history.

- Software agent attributes: autonomous, perceptive, persistent, adaptive, can take over others' goals.
- Nine properties: reactivity, autonomy, communication, cooperation, adaptation, inference, temporal continuity, personality, mobility.
- Rational = maximise expected performance given percepts, knowledge and available actions; not omniscient, not always successful.
- Rationality depends on performance measure, knowledge of environment, percept sequence, action set.
- Autonomy grows with experience; learning corrects incomplete or wrong built-in knowledge.
- Boundary robot: 8-bit sense vector (missing cells = 1), 4 features, rule-based actions.
- PEAS = Performance, Environment, Actuators, Sensors: the first step in agent design.
- Environment dimensions: fully/partially observable, static/dynamic (semi-dynamic), deterministic/stochastic (strategic), episodic/sequential, single/multi-agent (cooperative/competitive), discrete/continuous.
- Hardest: automobile driver (partially, multi, stochastic, sequential, dynamic, continuous). Easiest: crossword.
- Agent = architecture + program; table-driven agents need  $|P| + \dots + |P|^T$  entries.
- Agent types: simple reflex (current percept only) → model-based (internal state + model) → goal-based (predict, reach goal) → utility-based (degree of success) → learning (critic, learning component, performance element, problem generator).
- Forms of learning: rote, instruction, analogy, induction (refutable), deduction (irrefutable).
- Feedback-based learning: supervised (labelled pairs), unsupervised (no labels), reinforcement (rewards, discounted; delayed reinforcement).

## Check yourself

**Q. 1. Define an agent, a percept and a percept sequence. List any five properties an agent is expected to have.**

An agent perceives its environment through sensors and acts on it through actuators, usually on someone's behalf. A percept is the input received through sensors in a unit of time; a percept sequence is the history of percepts, long enough for the agent to decide on an action.

Properties (any five): reactivity (sense and respond), autonomy (pursue goal without much human help), communicating ability, coexistence by cooperating, adaptation to new situations, drawing inferences, temporal continuity, personality, mobility.

**Q. 2. Differentiate between rationality and omniscience with an example. On what four factors does rationality depend?**

Rationality means choosing the action expected to maximise the performance measure from current percepts, knowledge, available actions and past experience. Omniscience means knowing everything, including the actual outcome of actions in the future, which is impossible. Sohan, hit by a stray bullet on his way to the shop, was not omniscient but was not irrational: nothing he knew signalled danger.

Rationality depends on (1) the performance measure, (2) the agent's knowledge of the environment, (3) the percept sequence to date, (4) the set of actions available.

**Q. 3. A boundary-following robot is in the bottom-left corner of the room. The cells above, above-right and to the right of it are free. Write its sense vector, compute  $f_1 \dots f_4$  using the unit's rules, and state which action fires.**

c1 (top-left), c7 (bottom-left), c8 (left) are beyond the left wall; c5 (bottom-right), c6 (bottom), c7 are beyond the floor wall: all 1. c2 (above), c3 (above-right), c4 (right) are free: 0.

Sense vector  $\langle 1, 0, 0, 0, 1, 1, 1, 1 \rangle$ .

$f_1: c_3 = 0 \rightarrow 1$ .  $f_2: c_4 = 0 \rightarrow 1$ .  $f_3: c_6 = 1 \rightarrow 1$ .  $f_4: c_8 = 1 \rightarrow 1$ . Features (1, 1, 1, 1).

Action 1 needs all 0; Action 2 needs  $f_2 = 0$ ; Action 3 needs  $f_3 = 0$ ; Action 4 needs  $f_4 = 0$ . None fires, so the robot takes the default action (move to any free neighbouring cell).

**Q. 4. Give the PEAS description of an automated taxi driver (or a self-driving car).**

P: safety of passengers, comfort, reaching the correct destination, minimum trip time, obeying traffic rules, minimum disturbance to others, minimum cost.

E: roads of all kinds (expressways to rural roads), traffic rules, other vehicles (lorries, cars, autos, bullock carts), pedestrians, passengers, weather.

A: steering, accelerator, brake, gears, indicators/horn, display screen, communication devices.

S: cameras, radar/LiDAR or distance sensors, GPS, speedometer, odometer, engine sensors.



**Q. 5. Classify the task environment of (a) a crossword-solving agent, (b) a medical diagnosis agent, (c) a chess-playing agent, on all six dimensions, with reasons.**

(a) Crossword: fully observable (whole grid and clues visible), single-agent, deterministic (next grid fixed by the word written), sequential (a filled word constrains later words), static (nothing changes while thinking), discrete.

(b) Medical diagnosis: partially observable (internal condition not directly visible), single-agent (multi if staff/patients are treated as agents), stochastic (response to treatment uncertain), episodic across patients but sequential for one patient, dynamic (patient's condition changes), continuous.

(c) Chess: strictly partially observable per the unit (castling, en passant, repetition need history; many books say fully), multi-agent competitive, strategic (deterministic except the opponent's moves), sequential, semi-dynamic with a clock, discrete.

**Q. 6. Why can a simple reflex agent fail in a partially observable environment? How does a model-based reflex agent overcome this? Illustrate with the vacuum world.**

An SR agent acts only on the current percept, so anything it cannot currently sense is ignored. In the two-square vacuum world its sensor sees only its own square. After cleaning both squares it cannot remember that the other square is clean, so it keeps moving Left and Right forever (4 moves in 6 steps, 3 of them wasted after step 3).

A model-based reflex agent keeps an internal state updated from the percept sequence using a model of how the world evolves and how its actions affect it. It remembers "A = Clean, B = Clean" and does NoOp from step 4 on, scoring 9 instead of 6 when each move costs 1.

**Q. 7. Compare goal-based and utility-based agents. When is a utility-based agent necessary?**

A goal-based agent predicts the results of actions and chooses those that reach its goal; its judgement is binary (goal reached or not). It is flexible, because changing the goal changes behaviour without rewriting rules.

A utility-based agent maps each state to a number (degree of success) and picks the action leading to the highest-utility state. It is necessary when (i) goals conflict (speed vs. safety), (ii) partial success matters, or (iii) there are several ways to reach the goal and the cheaper or faster one must be chosen, e.g. two routes to Kozhikode of 180 km and 195 km.

**Q. 8. Draw and explain the components of a learning agent.**

(i) Performance element: chooses actions from percepts and knowledge. (ii) Critic: compares the agent's behaviour with a fixed performance standard and gives feedback. (iii) Learning component: uses the feedback to improve the performance element's knowledge. (iv) Problem generator: suggests new, possibly sub-optimal actions so the agent gains fresh experience. Diagram: sensors feed both the critic and the performance element; critic → feedback → learning component → changes → performance element → actuators (the performance element sends knowledge back to the learning component); learning component → learning goals → problem generator → performance element.

Example: a driver agent meeting a vehicle coming the wrong way on a one-way road has no rule for it; the problem generator suggests a new action, and the critic's judgement of the result is learned.

**Q. 9. Explain the five forms of learning with one example each. Which one is irrefutable?**

Rote: copying knowledge directly (multiplication tables). Instruction: learning from a teacher and integrating it with existing knowledge. Analogy: new concept from a similar known one (driving a heavy vehicle after a light one). Induction: general conclusion from many examples (all cows have four legs), can be refuted by a counter-example. Deduction: conclusion that must be true if the premises are true (mathematical proof).

Deduction is irrefutable; induction is the most used by humans and the most important for agents' learning components.

**Q. 10. Distinguish supervised, unsupervised and reinforcement learning. What is discounting of rewards and why is it needed? What is delayed reinforcement learning?**

Supervised: learns a function from given input-output pairs. Unsupervised: no output labels; the system finds structure on its own (e.g. clustering consumers). Reinforcement: the agent receives occasional positive or negative rewards and learns a policy to maximise total reward.

For ongoing tasks the total reward can be infinite, so future rewards are multiplied by  $\gamma^t$  ( $0 \leq \gamma < 1$ ), making near rewards count more: a reward of 10 per step forever becomes  $10 / (1 - 0.9) = 100$  with  $\gamma = 0.9$ . Delayed reinforcement learning is learning a policy when the reward depends on a whole sequence of earlier actions, e.g. win/loss only at the end of a chess game.

## UNIT 11

# AI Programming with Python

AI പ്രോഗ്രാമുകൾ Python-ൽ എങ്ങനെ എഴുതാം എന്നതാണ് ഈ unit. NumPy, Pandas, Matplotlib, Scikit-learn, TensorFlow, PyTorch എന്നീ libraries ആദ്യം പഠിക്കുന്നു, പിന്നെ N-Queen, Minimax, Alpha-Beta, BFS, DFS, Best First, A\*, AO\*, IDA\* എന്നിവയുടെ Python code-ഉം model evaluation-ഉം hyperparameter tuning-ഉം നോക്കുന്നു.

Why Python for AI

Python libraries for AI: an overview

NumPy: creating arrays and array properties

NumPy: indexing, slicing, vectorised operations, statistics

NumPy: matrices, random numbers, broadcasting, normalisation

Pandas: Series, DataFrame and reading files

Pandas: exploring, selecting and cleaning data

Pandas: transforming, combining and analysing data

Matplotlib: visualising data

Scikit-learn: the ML workflow and Linear Regression

Scikit-learn: Decision Tree and K-Means

TensorFlow and Keras

PyTorch

N-Queen problem (backtracking)

Min-Max algorithm for Tic-Tac-Toe

Alpha-Beta pruning

The sample graph and Breadth First Search

Depth First Search

Best First Search

A\* algorithm

AO\* algorithm

Iterative Deepening A\* (IDA\*)

Model evaluation metrics

Hyperparameter tuning

## Why Python for AI

**Definition:** Python is a high-level, general-purpose programming language with simple, readable syntax and a large set of libraries, which makes it the main language for AI and ML.

Python ലളിതവും വായിക്കാൻ എളുപ്പമുള്ളതുമായ syntax-ഉം ധാരാളം libraries-ഉം ഉള്ള ഒരു high-level, general-purpose programming language ആണ്; അതുകൊണ്ടാണ് ഇത് AI, ML എന്നിവയുടെ പ്രധാന language ആയത്.

Python is a high-level, general-purpose, beginner-friendly language with readable syntax. It became the main language for AI and ML for four reasons: (1) **easy syntax**, so complex algorithms are quick to write and debug; (2) a large set of **libraries** (NumPy, Pandas, NLTK, OpenCV, scikit-learn, TensorFlow, PyTorch ...); (3) a huge **community** with documentation, forums and courses; (4) it is **platform independent**, so the same program runs on Windows, Linux or macOS without change.

Early AI used rule-based systems and search; modern AI is mostly machine learning, which learns patterns from data. Python supports both styles, so one language covers expert systems, state-space search and deep learning.

#### മലയാളത്തിൽ

Python-ന്റെ code വായിക്കാൻ ഏതാണ്ട് English പോലെയാണ്, അതുകൊണ്ട് algorithm-ൽ ശ്രദ്ധിക്കാം, syntax-ൽ കുടുങ്ങില്ല. ഒരു നല്ല അടുക്കള പോലെയാണ് Python: NumPy, Pandas പോലുള്ള libraries ആണ് അവിടെ റെഡിയായി വെച്ചിരിക്കുന്ന പാത്രങ്ങളും മസാലകളും, എല്ലാം ആദ്യം മുതൽ ഉണ്ടാക്കേണ്ട. Rule-based AI മുതൽ deep learning വരെ ഒരേ language-ൽ ചെയ്യാം എന്നതാണ് ഏറ്റവും വലിയ ഗുണം.

| Application area                  | How Python is used                                                   |
|-----------------------------------|----------------------------------------------------------------------|
| Pathfinding in robotics and games | State-space search: A*, BFS, DFS                                     |
| Puzzle solving                    | 8-puzzle, Sudoku, N-Queens by state-space search                     |
| Planning in education systems     | Intelligent tutors search the space of teaching actions              |
| Machine / deep learning           | Analytics, recommendation systems, fraud detection                   |
| Natural Language Processing       | NLTK, spaCy, transformers: sentiment analysis, chatbots, translation |
| Computer vision                   | OpenCV + DL frameworks: face recognition, object detection           |
| Robotics and automation           | Robotic process automation (RPA), autonomous systems                 |
| Reinforcement learning            | Game-playing agents, robot control, decision making                  |

**Exam tip:** "Why is Python popular for AI?" is a common 4–5 mark question. Write the four points (easy syntax, rich libraries, large community, platform independence) and name one library for each AI area.

## Python libraries for AI: an overview

**Definition:** A library is a collection of pre-written, reusable modules and functions that a program imports to perform a task without writing the code from scratch.

Code ആദ്യം മുതൽ എഴുതാതെ ഒരു task ചെയ്യാൻ program import ചെയ്യുന്ന, മുൻകൂട്ടി എഴുതിയ, വീണ്ടും ഉപയോഗിക്കാവുന്ന modules-ന്റെയും functions-ന്റെയും ശേഖരമാണ് library.

Each AI task has its own set of libraries. You install a library once (`pip install numpy`; in Anaconda the common ones are pre-installed) and then import it, usually with a short alias such as `np` or `pd`. The alias is just a nickname; any name works, but the standard ones make code easy for others to read.

### മലയാളത്തിൽ

ഓരോ ജോലിക്കും ഓരോ library ഉണ്ട്: കണക്കിന് NumPy, table data-ക്ക് Pandas, graph വരക്കാൻ Matplotlib, ML-ന് scikit-learn, deep learning-ന് TensorFlow/PyTorch. `import numpy as np` എന്നെഴുതുമ്പോൾ `np` ഒരു ചെല്ലപ്പേര് മാത്രമാണ്, "ഉണ്ണികൃഷ്ണൻ" എന്നതിന് "ഉണ്ണി" എന്ന് വിളിക്കുന്ന പോലെ.

| Task                          | Libraries                                         |
|-------------------------------|---------------------------------------------------|
| Numerical computing           | NumPy                                             |
| Data handling / preprocessing | Pandas                                            |
| Visualisation                 | Matplotlib                                        |
| Rule-based expert systems     | Pyke, Experta (knowledge base + inference engine) |
| Graphs and state-space search | NetworkX (paths, A*, IDA*, minimax game trees)    |
| Classical machine learning    | scikit-learn                                      |
| Deep learning                 | TensorFlow (with Keras), PyTorch                  |
| NLP                           | NLTK, spaCy, Hugging Face transformers            |
| Computer vision               | OpenCV                                            |
| Reinforcement learning        | Stable Baselines3, Ray RLlib                      |

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
```

**Exam tip:** The unit says scikit-learn offers "reinforcement learning". It does not: scikit-learn covers classification, regression, clustering, dimensionality reduction and model selection. For reinforcement learning use Stable Baselines3 or Ray RLLib, as the same paragraph later says.

## NumPy: creating arrays and array properties

**NumPy (Numerical Python):** A Python library for numerical computing that provides the fast ndarray together with mathematical, random-number and linear-algebra functions.

**NumPy (Numerical Python):** വേഗതയേറിയ ndarray-ഉം mathematical, random-number, linear-algebra functions-ഉം നൽകുന്ന, numerical computing-ന്റെ Python library.

**ndarray:** NumPy's N-dimensional array of elements of the same type, described by its shape, ndim, dtype and size.

**ndarray:** ഒരേ type ഉള്ള elements അടങ്ങിയ NumPy-യുടെ N-dimensional array; shape, ndim, dtype, size എന്നിവ അതിനെ വിവരിക്കുന്നു.

**NumPy** (Numerical Python) provides the ndarray, a fast array for numbers, plus maths, random numbers and linear algebra. It is the base of Pandas, SciPy, TensorFlow and PyTorch. AI data is mostly 2-D (tables, matrices) or 3-D (images, sequences). Arrays can be made from lists or with built-in functions such as zeros, ones and random.rand. Every array has a shape, ndim, dtype and size.

### മലയാളത്തിൽ

NumPy array എന്നാൽ ഒരേ type-ലുള്ള numbers നിരത്തി വെച്ച ഒരു പെട്ടി. Python list-നെക്കാൾ വളരെ വേഗം കൂടുതലാണ്, കാരണം എല്ലാ elements-ഉം ഒരേ type ആണ്, memory-യിൽ അടുത്തടുത്ത് ഇരിക്കുന്നു. shape പറയും എത്ര row, എത്ര column; ndim എത്ര dimension; dtype ഏത് തരം number; size ആകെ എത്ര elements.

`np.array(list)` – array from a Python list

`np.zeros((r, c))`, `np.ones((r, c))`, `np.random.rand(r, c)`

`a.shape` = (rows, columns); `a.ndim` = number of axes; `a.dtype` = element type; `a.size` = rows × columns

### EXAMPLE: CREATING ARRAYS

```
import numpy as np

arr = np.array([1, 2, 3, 4])
print("1-D array:", arr)
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
print("2-D array:\n", arr2d)
print("zeros:\n", np.zeros((3, 3)))
print("ones:\n", np.ones((2, 2)))
np.random.seed(0)
print("random:\n", np.random.rand(3, 3))
```

```
1-D array: [1 2 3 4]
2-D array:
[[1 2 3]
 [4 5 6]]
zeros:
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
ones:
[[1. 1.]
 [1. 1.]]
random:
[[0.5488135  0.71518937 0.60276338]
 [0.54488318 0.4236548  0.64589411]
 [0.43758721 0.891773   0.96366276]]
```

Line by line: `np.array([1,2,3,4])` turns a list into a 1-D array (printed without commas). A list of lists gives a 2-D array with 2 rows and 3 columns. `np.zeros((3,3))` and `np.ones((2,2))` take the shape as a tuple and fill it with 0.0 or 1.0 (the dot shows these are floats). `np.random.seed(0)` fixes the random generator so you get the same "random" numbers every run; `np.random.rand(3,3)` gives a 3×3 array of values in [0, 1).

ആദ്യത്തെ line list-നെ array ആക്കുന്നു. List-നുള്ളിൽ list കൊടുത്താൽ rows-ഉം columns-ഉം ഉള്ള matrix കിട്ടും. zeros, ones ഒരു shape കൊടുത്താൽ ആ വലിപ്പത്തിൽ 0 അല്ലെങ്കിൽ 1 നിറച്ച matrix തരും. seed(0) വെച്ചാൽ ഓരോ തവണ run ചെയ്യുമ്പോഴും ഒരേ random numbers കിട്ടും, അതുകൊണ്ടാണ് ഇവിടെ output ഉറപ്പിച്ച് കാണിക്കാൻ പറ്റുന്നത്.

### EXAMPLE: ARRAY PROPERTIES

```
import numpy as np

a2d = np.array([[1, 2, 3], [4, 5, 6]])
print("Shape (rows, columns):", a2d.shape)
print("Number of dimensions:", a2d.ndim)
print("Data type:", a2d.dtype)
print("Total number of elements:", a2d.size)
```

```
Shape (rows, columns): (2, 3)
Number of dimensions: 2
Data type: int64
Total number of elements: 6
```

The array has 2 rows and 3 columns, so shape is (2, 3), ndim is 2 and size is  $2 \times 3 = 6$ . Whole numbers are stored as 64-bit integers (int64) on Linux and macOS.

2 rows, 3 columns ഉള്ളതുകൊണ്ട് shape (2, 3), dimension 2, ആകെ elements 6. Integer ആയതുകൊണ്ട് dtype int64 ആണ്.

**Exam tip:** The unit's properties program prints `arr2d.dtype`, a variable from the previous program, instead of `a2d.dtype`. Run on its own it raises `NameError`; use `a2d.dtype` as above. Also, dtype can show `int32` on older Windows NumPy, so do not treat `int64` as the only right answer.

## NumPy: indexing, slicing, vectorised operations, statistics

**Indexing:** Selecting a single element of an array by its position, starting from 0 (negative indices count from the end).

**Indexing:** ഒരു array-യിലെ ഒരു element-നെ അതിന്റെ position വെച്ച് തിരഞ്ഞെടുക്കൽ; position 0-ൽ തുടങ്ങുന്നു (negative indices അവസാനം മുതൽ എണ്ണുന്നു).

**Slicing:** Selecting a range of elements with `a[start:stop]`, where the stop index is not included.

**Slicing:** `a[start:stop]` ഉപയോഗിച്ച് ഒരു കൂട്ടം elements തിരഞ്ഞെടുക്കൽ; stop index ഉൾപ്പെടുന്നില്ല.

**Vectorised operation:** An operation applied to every element of an array at once, without an explicit loop.

**Vectorised operation:** ഒരു explicit loop ഇല്ലാതെ array-യിലെ എല്ലാ elements-നും ഒറ്റയടിക്ക് പ്രയോഗിക്കുന്ന operation.



**Indexing** picks one element (`a[0]`, `a[-1]`, `b[row, col]`). **Slicing** picks a range `a[start:stop]`, where `stop` is not included. **Vectorised operations** apply arithmetic to every element at once, with no loop, which is why NumPy is fast on large AI datasets. Functions like `sum`, `mean`, `std`, `max`, `min` summarise data; the `axis` argument chooses column-wise (`axis=0`) or row-wise (`axis=1`).

#### മലയാളത്തിൽ

Slicing ഒരു train-ലെ bogies എണ്ണുന്ന പോലെയാണ്: `c[1:4]` എന്നാൽ index 1 മുതൽ 3 വരെ, 4 ഉൾപ്പെടില്ല. Vectorised operation എന്നാൽ for loop എഴുതാതെ മുഴുവൻ array-യിലും ഒറ്റയടിക്ക് കണക്ക് ചെയ്യൽ: `x * 2` എല്ലാ element-നെയും ഇരട്ടിയാക്കും. `axis=0` മുകളിൽ നിന്ന് താഴേക്ക് (column), `axis=1` ഇടത് നിന്ന് വലത്തേക്ക് (row).

`a[i]` – element at index `i` (`a[-1]` = last)

`a[start:stop]` – indices start ... stop - 1

`b[i, j]` – row `i`, column `j`; `d[0:2, 1:3]` – rows 0-1, columns 1-2

`np.sum(m, axis=0)` → column sums; `np.sum(m, axis=1)` → row sums

## EXAMPLE: INDEXING AND SLICING

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
print("First element:", a[0])
print("Last element:", a[-1])
print("Index 1 to 3:", a[1:4])
print("First three:", a[:3])
print("Index 2 to end:", a[2:])
d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Row 0, Column 1:", d[0, 1])
print("First two rows, last two columns:\n", d[0:2, 1:3])
```

```
First element: 10
Last element: 50
Index 1 to 3: [20 30 40]
First three: [10 20 30]
Index 2 to end: [30 40 50]
Row 0, Column 1: 2
First two rows, last two columns:
[[2 3]
 [5 6]]
```

Indices start at 0, so `a[0]` is 10 and `a[-1]` counts from the end. `a[1:4]` gives indices 1, 2, 3. A missing start means "from the beginning", a missing stop means "to the end". For a 2-D array, `d[0, 1]` is row 0, column 1 = 2, and `d[0:2, 1:3]` takes rows 0–1 and columns 1–2.

Index 0-ൽ നിന്നാണ് തുടങ്ങുന്നത്, -1 എന്നാൽ അവസാനത്തേത്. `[ :3]` തുടക്കം മുതൽ 3 എണ്ണം, `[2:]` index 2 മുതൽ അവസാനം വരെ. 2-D array-യിൽ കോമയ്ക്ക് മുമ്പ് row, ശേഷം column. `d[0:2, 1:3]` ആദ്യ രണ്ട് rows-ൽ നിന്ന് അവസാന രണ്ട് columns മുറിച്ചെടുക്കുന്നു.

### EXAMPLE: VECTORISED ARITHMETIC

```
import numpy as np

x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
print("x + y :", x + y)
print("x - y :", x - y)
print("x * y :", x * y)
print("x / y :", x / y)
print("x * 2 :", x * 2)
print("x ** 2:", x ** 2)
```

```
x + y : [5 7 9]
x - y : [-3 -3 -3]
x * y : [ 4 10 18]
x / y : [0.25 0.4 0.5 ]
x * 2 : [2 4 6]
x ** 2: [1 4 9]
```

Each operator works element by element: 1+4, 2+5, 3+6 and so on.  $x * y$  is element-wise multiplication ( $1 \times 4$ ,  $2 \times 5$ ,  $3 \times 6$ ), not matrix multiplication. Division always gives floats. A scalar (2) is applied to every element.

ഓരോ operator-ഉം ഒരേ സ്ഥാനത്തുള്ള elements തമ്മിൽ കണക്ക് ചെയ്യുന്നു.  $x * y$  element-wise ഗുണനമാണ്, matrix multiplication അല്ല, ഇത് പരീക്ഷയിൽ ശ്രദ്ധിക്കണം. ഒരു number കൊണ്ട് ഗുണിച്ചാൽ എല്ലാ element-ഉം ആ number കൊണ്ട് ഗുണിക്കപ്പെടും.

### EXAMPLE: STATISTICS AND AXIS

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(np.sum(arr), np.mean(arr), np.std(arr), np.max(arr), np.min(arr))
mat = np.array([[1, 2, 3], [4, 5, 6]])
print("Column-wise sum:", np.sum(mat, axis=0))
print("Row-wise sum:", np.sum(mat, axis=1))
```

```
15 3.0 1.4142135623730951 5 1
Column-wise sum: [ 5  7  9]
Row-wise sum: [ 6 15]
```

Sum = 15, mean =  $15/5 = 3$ . The standard deviation is  $\sqrt{[(1-3)^2 + (2-3)^2 + 0 + 1^2 + 2^2]/5} = \sqrt{(10/5)} = \sqrt{2} \approx 1.414$ . With axis=0 the sums go down each column (1+4, 2+5, 3+6); with axis=1 they go across each row (1+2+3, 4+5+6).

Mean 3, standard deviation  $\sqrt{2} \approx 1.414$  എന്ന് കൈകൊണ്ട് കണക്കാക്കിയാലും കിട്ടും. axis=0 ഓരോ column-ഉം കൂട്ടുന്നു, axis=1 ഓരോ row-ഉം. NumPy-യുടെ std n കൊണ്ടാണ് ഹരിക്കുന്നത് (population standard deviation).

## NumPy: matrices, random numbers, broadcasting, normalisation

**Broadcasting:** NumPy's rule for combining arrays of different shapes by stretching the smaller array to match the larger one, without copying data.

**Broadcasting:** വ്യത്യസ്ത shapes ഉള്ള arrays-നെ, data copy ചെയ്യാതെ, ചെറിയ array-യെ വലുതിന്റെ shape-ലേക്ക് നീട്ടി സംയോജിപ്പിക്കുന്ന NumPy-യുടെ rule.

**Normalisation (z-score):** Rescaling data to mean 0 and standard deviation 1 using  $z = (x - \text{mean}) / \text{std}$ .

**Normalisation (z-score):**  $z = (x - \text{mean}) / \text{std}$  ഉപയോഗിച്ച് data-യെ mean 0, standard deviation 1 ആകുന്ന രീതിയിൽ rescale ചെയ്യൽ.

NumPy does linear algebra: `np.dot(A, B)` (or `A @ B`) multiplies matrices, `A.T` is the transpose, `np.linalg.det` and `np.linalg.inv` give determinant and inverse. The `np.random` module creates random data for weight initialisation and shuffling. **Broadcasting** lets arrays of different shapes combine: a smaller array is stretched to match the larger one without copying. **Normalisation** (z-score) rescales data to mean 0 and standard deviation 1, which helps models train.

### മലയാളത്തിൽ

Broadcasting ഒരു ക്ലാസ്സിലെ എല്ലാ കുട്ടികൾക്കും ഒരേ grace mark ചേർക്കുന്ന പോലെയാണ്: ഒരു row മാത്രം എഴുതിയാൽ മതി, NumPy അത് ഓരോ row-യിലും ചേർത്തോളും. Normalisation എല്ലാ features-നെയും ഒരേ scale-ലേക്ക് കൊണ്ടുവരുന്നു, അല്ലെങ്കിൽ വലിയ numbers ഉള്ള feature (ശമ്പളം പോലെ) ചെറിയ numbers ഉള്ളതിനെ (പ്രായം പോലെ) മുകളിക്കളയും.

$$(AB)_{ij} = \sum_k A_{ik} B_{kj}$$

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc; A^{-1} = (1/\det A) \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

$$z = (x - \text{mean}) / \text{std}$$

### EXAMPLE: MATRIX OPERATIONS

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print("A @ B:\n", np.dot(A, B))
print("Transpose of A:\n", A.T)
print("det(A):", np.linalg.det(A))
print("inverse of A:\n", np.linalg.inv(A))
```

```
A @ B:
[[19 22]
 [43 50]]
Transpose of A:
[[1 3]
 [2 4]]
det(A): -2.0000000000000004
inverse of A:
[[-2.  1.]
 [ 1.5 -0.5]]
```

Check by hand: row 1 of A times column 1 of B =  $1 \times 5 + 2 \times 7 = 19$ ;  $1 \times 6 + 2 \times 8 = 22$ ;  $3 \times 5 + 4 \times 7 = 43$ ;  $3 \times 6 + 4 \times 8 = 50$ .  $\det A = 1 \times 4 - 2 \times 3 = -2$ . The printed  $-2.0000000000000004$  is a tiny floating-point rounding error, because  $\det$  is computed numerically. Inverse =  $(1/-2)[[4, -2], [-3, 1]] = [[-2, 1], [1.5, -0.5]]$ .

`np.dot` ശരിയായ matrix multiplication ചെയ്യുന്നു: row  $\times$  column. `.T` rows-നെ columns ആക്കുന്നു. Determinant  $-2$  ആണ്, അവസാനത്തെ  $\dots 0004$  കമ്പ്യൂട്ടറിന്റെ decimal rounding കാരണമാണ്, തെറ്റല്ല. Inverse കൈകൊണ്ട് കണക്കാക്കിയാലും ഇതേ ഉത്തരം കിട്ടും.

### EXAMPLE: RANDOM MODULE

```
import numpy as np

np.random.seed(42)
print(np.random.rand(5))           # uniform in [0, 1)
print(np.random.randint(1, 10, size=5)) # integers 1..9
print(np.random.randn(5))          # normal, mean 0, std 1
```

```
[0.37454012 0.95071431 0.73199394 0.59865848 0.15601864]
[3 7 8 5 4]
[ 0.39257975 -0.92918467  0.07983181 -0.1595165   0.02222183]
```

rand gives uniform values between 0 and 1, randint(1, 10) gives integers from 1 to 9 (10 is excluded), and randn samples the standard normal distribution. Without seed the numbers change every run, which is why the unit's printed numbers will not match yours.

rand 0-ാം 1-ാം ഇടയിലുള്ള numbers, randint(1, 10) 1 മുതൽ 9 വരെ integers (10 വരില്ല), randn bell curve-ൽ നിന്നുള്ള numbers. Seed ഇല്ലെങ്കിൽ ഓരോ run-ലും വേറെ numbers വരും.

### EXAMPLE: BROADCASTING AND NORMALISATION

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
b = np.array([1, 2, 3])
print(a + b)
data = np.array([10, 20, 30, 40, 50])
normalized = (data - np.mean(data)) / np.std(data)
print("Normalized:", normalized)
```

```
[[2 4 6]
 [5 7 9]]
Normalized: [-1.41421356 -0.70710678  0.          0.70710678  1.41421356]
```

a has shape (2, 3) and b has shape (3,), so b is added to each row. For normalisation: mean = 30, std =  $\sqrt{[(400 + 100 + 0 + 100 + 400)/5]} = \sqrt{200} \approx 14.142$ . So  $z(10) = -20/14.142 = -1.414$ ,  $z(20) = -0.707$ , and so on. data - np.mean(data) is itself broadcasting (array minus scalar).

[1, 2, 3] എന്ന row രണ്ട് rows-ലും ചേർന്നു, അതാണ് broadcasting. Normalisation-ൽ ഓരോ value-ൽ നിന്നും mean 30 കുറച്ച്, std 14.142 കൊണ്ട് ഹരിക്കുന്നു. Result-ന്റെ mean 0, std 1 ആകും.

**Exam tip:** Advantages of NumPy: speed (vectorised, faster than lists), memory efficiency, many built-in maths functions, works with Pandas/SciPy/TensorFlow/PyTorch. Limitations: most operations use a single CPU core (only some linear algebra is multi-threaded) and there is no GPU support, and it has no models, layers or training utilities like TensorFlow and PyTorch.

## Pandas: Series, DataFrame and reading files

**Pandas:** A Python library for loading, organising, cleaning and analysing tabular data (data preprocessing).

**Pandas:** Tabular data load ചെയ്യാനും ക്രമീകരിക്കാനും clean ചെയ്യാനും analyse ചെയ്യാനുമുള്ള (data preprocessing) Python library.

**Series:** A one-dimensional labelled array, like a single column.

**Series:** ഒരു column പോലെയുള്ള, labels ഉള്ള one-dimensional array.

**DataFrame:** A two-dimensional labelled table of rows and columns.

**DataFrame:** Rows-ഉം columns-ഉം ഉള്ള, labels ഉള്ള two-dimensional table.

Before training a model the data must be organised, cleaned and analysed: this is **data preprocessing**, and it often takes most of the time in an AI project. **Pandas** is the main preprocessing library. Its two data structures are the **Series** (a one-dimensional labelled array, like one column) and the **DataFrame** (a two-dimensional labelled table of rows and columns). Pandas reads and writes CSV, Excel and JSON files.

### മലയാളത്തിൽ

Pandas ഒരു Excel sheet Python-നുള്ളിൽ കിട്ടുന്ന പോലെയാണ്. Series ഒരു column മാത്രം, ഓരോ value-നും ഒരു label (index) ഉണ്ട്. DataFrame മുഴുവൻ table ആണ്, ഒരു school register പോലെ: ഓരോ row ഒരു കുട്ടി, ഓരോ column ഒരു വിവരം. Raw data-യും ML model-ഉം തമ്മിലുള്ള പാലാണ് Pandas.

```
pd.Series(values, index=labels)
pd.DataFrame(dict_of_columns)
pd.read_csv("f.csv"), pd.read_excel("f.xlsx"), pd.read_json("f.json")
df.to_csv("out.csv", index=False)
```

### EXAMPLE: SERIES WITH DEFAULT AND CUSTOM INDEX

```
import pandas as pd

s1 = pd.Series([10, 20, 30, 40])
print(s1)
s2 = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s2)
print("s2['b'] =", s2['b'])
```

```
0    10
1    20
2    30
3    40
dtype: int64
a     10
b     20
c     30
dtype: int64
s2['b'] = 20
```

With no index given, Pandas labels the values 0, 1, 2, 3 (left column). With `index=['a', 'b', 'c']` the labels are letters, and a value can be fetched by its label, `s2['b']`. The last line of each print shows the dtype.

Index കൊടുത്തില്ലെങ്കിൽ 0 മുതൽ നമ്പർ ഇടും. നമ്മൾ label കൊടുത്താൽ ആ label വെച്ച് value എടുക്കാം, വീട്ടുപേര് വെച്ച് ആളെ കണ്ടുപിടിക്കുന്ന പോലെ.



## EXAMPLE: DATAFRAME AND QUICK SUMMARY

```
import pandas as pd

data = {'Name': ['P', 'Q', 'R'], 'Age': [18, 27, 20], 'Score': [85, 90, 88]}
df = pd.DataFrame(data)
print(df)
print(df.shape)
print(list(df.columns))
print(df.describe())
```

```
   Name  Age  Score
0     P   18    85
1     Q   27    90
2     R   20    88
(3, 3)
['Name', 'Age', 'Score']
      Age      Score
count  3.000000  3.000000
mean   21.666667  87.666667
std     4.725816   2.516611
min    18.000000  85.000000
25%    19.000000  86.500000
50%    20.000000  88.000000
75%    23.500000  89.000000
max    27.000000  90.000000
```

A dictionary of lists becomes a table: keys are column names, lists are column values. shape is (rows, columns). describe() gives count, mean, std, min, quartiles and max for the numeric columns only (Name is skipped). Check: mean age =  $(18 + 27 + 20)/3 = 21.67$ .

Dictionary-യിലെ key column-ന്റെ പേരും list അതിലെ values-ഉം ആകുന്നു.

describe() ഒറ്റ line കൊണ്ട് count, mean, std, min, max എല്ലാം തരും, numbers ഉള്ള columns-ന് മാത്രം.

### EXAMPLE: WRITING AND READING A CSV FILE

```
import pandas as pd

df = pd.DataFrame({'Name': ['P', 'Q'], 'Age': [18, 27]})
df.to_csv("students.csv", index=False)
back = pd.read_csv("students.csv")
print(back)
```

|   | Name | Age |
|---|------|-----|
| 0 | P    | 18  |
| 1 | Q    | 27  |

`to_csv` saves the table; `index=False` stops Pandas from writing the 0, 1 row labels as an extra column. `read_csv` loads it back into a new DataFrame. A relative path such as "students.csv" is looked up in the folder the program runs from; an absolute path works from anywhere.

`to_csv` file-ലേക്ക് എഴുതുന്നു, `read_csv` തിരിച്ച് വായിക്കുന്നു. `index=False` ഇല്ലെങ്കിൽ 0, 1 എന്ന row number-ഉം ഒരു column ആയി file-ൽ കയറും.

## Pandas: exploring, selecting and cleaning data

**Data cleaning:** Detecting and fixing missing values, duplicate rows and wrong data types so the data is fit for analysis.

**Data cleaning:** Data analysis-ന് യോജ്യമാക്കാൻ missing values, duplicate rows, തെറ്റായ data types എന്നിവ കണ്ടെത്തി ശരിയാക്കൽ.

**iloc / loc:** `iloc` selects rows by integer position; `loc` selects rows by index label.

**iloc / loc:** `iloc` integer position വെച്ച് rows തിരഞ്ഞെടുക്കുന്നു; `loc` index label വെച്ച് rows തിരഞ്ഞെടുക്കുന്നു.

**Exploration** functions show the structure of a dataset: `head()` (first 5 rows), `tail()` (last 5), `info()`, `shape`, `columns`, `describe()`. **Selection:** `df['col']` or `df[['c1', 'c2']]` for columns, `df.iloc[i]` for a row by position, `df.loc[label]` for a row by index label, and `df[condition]` to filter rows. **Cleaning** handles missing values (`isnull`, `dropna`, `fillna`), duplicates (`drop_duplicates`) and wrong types (`astype`).

### മലയാളത്തിൽ

യഥാർത്ഥ data എപ്പോഴും വൃത്തികേടാണ്: ചില കള്ളികൾ ഒഴിഞ്ഞിരിക്കും, ചില rows രണ്ടുതവണ വരും. ചന്തയിൽ നിന്ന് കൊണ്ടുവന്ന മീൻ കഴുകി വൃത്തിയാക്കുന്ന പോലെ, model-ന് കൊടുക്കും മുമ്പ് data വൃത്തിയാക്കണം. `iloc` സ്ഥാനം (position) വെച്ചും `loc` label വെച്ചും row എടുക്കുന്നു.

`df.iloc[0]` – first row by position; `df.loc[1]` – row whose index label is 1

`df[df['Age'] > 23]` – rows satisfying a condition

`df.isnull().sum(); df.dropna(); df.fillna(0); df.drop_duplicates(); df['Age'].astype(float)`

### EXAMPLE: SELECTING COLUMNS AND ROWS

```
import pandas as pd

df = pd.DataFrame({'Name': ['P', 'Q', 'R'], 'Age': [18, 27, 20], 'Score': [85, 90, 88]})
print(df[['Name', 'Age']])
print(df.iloc[0])
print(df[df['Age'] > 19])
```

```
   Name  Age
0    P   18
1    Q   27
2    R   20
Name    P
Age     18
Score   85
Name: 0, dtype: object
   Name  Age  Score
1    Q   27     90
2    R   20     88
```

Double brackets `[['Name', 'Age']]` return a smaller DataFrame. `df.iloc[0]` returns the first row as a Series whose labels are the column names. `df['Age'] > 19` makes a True/False column (False, True, True); putting it inside `df[...]` keeps only the True rows, Q and R.

രണ്ട് columns വേണമെങ്കിൽ ഇരുട്ട bracket വേണം. `iloc[0]` ആദ്യ row. Condition എഴുതുമ്പോൾ ഓരോ row-നും True/False കിട്ടും, True ഉള്ള rows മാത്രം ബാക്കിയാകും, അരിഖ കൊണ്ട് അരിക്കുന്ന പോലെ.

### EXAMPLE: CLEANING MISSING VALUES AND DUPLICATES

```
import pandas as pd
import numpy as np

df = pd.DataFrame({'Name': ['P', 'Q', 'Q', 'S'], 'Age': [18, 27, 27, np.nan]})
print(df.isnull().sum())
print(df.dropna())
print(df.fillna(0))
print(df.drop_duplicates())
df['Age'] = df['Age'].fillna(0).astype(int)
print(df['Age'].dtype)
```

```
Name    0
Age      1
dtype: int64
   Name  Age
0    P  18.0
1    Q  27.0
2    Q  27.0
   Name  Age
0    P  18.0
1    Q  27.0
2    Q  27.0
3    S   0.0
   Name  Age
0    P  18.0
1    Q  27.0
3    S   NaN
int64
```

`np.nan` marks a missing age. `isnull().sum()` counts missing values per column (Age has 1). `dropna()` removes row 3; `fillna(0)` keeps it but puts 0. `drop_duplicates()` removes row 2, an exact copy of row 1. A column with NaN is stored as float, so we fill first and then `astype(int)`. None of these change `df` unless you assign the result back, as in `df = df.dropna()`.

NaN എന്നാൽ value ഇല്ല. `dropna` ആ row മൊത്തം കളയും, `fillna(0)` ഒഴിഞ്ഞ കള്ളിയിൽ 0 ഇടും. ഒരേപോലുള്ള രണ്ട് rows ഉണ്ടെങ്കിൽ `drop_duplicates` ഒന്ന് മാത്രം നിർത്തും. Result തിരിച്ച് `df`-ലേക്ക് assign ചെയ്തില്ലെങ്കിൽ original മാറില്ല.

**Exam tip:** `i loc` = integer position, `loc` = label. After sorting, row labels are no longer in order, so `df.iloc[0]` and `df.loc[0]` can give different rows.

## Pandas: transforming, combining and analysing data

**Concatenation (pd.concat):** Stacking DataFrames one below the other into a single DataFrame.

**Concatenation (pd.concat):** DataFrames ഒന്നിനു താഴെ ഒന്നായി അടുക്കി ഒരൊറ്റ DataFrame ആക്കൽ.

**Merge (pd.merge):** Joining two DataFrames on a common column, like a database join.

**Merge (pd.merge):** ഒരു പൊതു column അടിസ്ഥാനമാക്കി രണ്ട് DataFrames-നെ, database join പോലെ, യോജിപ്പിക്കൽ.

**Transformation:** rename columns, sort\_values by a column, and apply a function to every value. inplace=True changes the original DataFrame (default False returns a new one); ascending=False sorts in descending order. **Combining:** pd.concat stacks DataFrames one below the other; pd.merge joins them on a common column, like a database join. **Analysis:** mean, median, std, corr.

### മലയാളത്തിൽ

concat രണ്ട് attendance sheets ഒന്നിന് താഴെ ഒന്നായി ഒട്ടിക്കുന്ന പോലെ. merge ഒരു common column (roll number പോലെ) നോക്കി രണ്ട് tables-ലെ വിവരം ചേർക്കുന്നു, രണ്ടിലും ഉള്ള values-ന് മാത്രം. apply ഒരു function ഓരോ value-ലും പ്രയോഗിക്കുന്നു.

```
df.rename(columns={'old': 'new'}, inplace=True)
df.sort_values('col', ascending=False, inplace=True)
df['c'] = df['c'].apply(lambda x: x / 10)
pd.concat([df1, df2]) – stack rows; pd.merge(df1, df2, on='A') – join on column A
```

## EXAMPLE: TRANSFORMATION AND STATISTICS

```
import pandas as pd

df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [24, 27, 22], 'Score': [85, 90, 88]})
df.rename(columns={'Score': 'Marks'}, inplace=True)
df.sort_values('Age', ascending=False, inplace=True)
df['Marks'] = df['Marks'].apply(lambda x: x / 10)
print(df)
print("Average Marks:", df['Marks'].mean())
print("Median:", df['Marks'].median())
print("Standard deviation:", df['Marks'].std())
print("Correlation:", df['Marks'].corr(df['Age']))
```

```
      Name  Age  Marks
1     Bob   27    9.0
0  Alice   24    8.5
2  Charlie  22    8.8
Average Marks: 8.766666666666667
Median: 8.8
Standard deviation: 0.2516611478423584
Correlation: 0.4999999999999984
```

Score is renamed to Marks, rows are sorted by Age from oldest to youngest (note the index labels 1, 0, 2 travel with their rows), and the lambda divides each mark by 10. Mean =  $(9.0 + 8.5 + 8.8)/3 = 8.767$ ; median = middle value 8.8. The correlation is exactly 0.5 in theory; 0.4999999999999984 is floating-point rounding.

ആദ്യം column-ന്റെ പേര് മാറ്റി, പിന്നെ പ്രായം കൂടിയ ആൾ ആദ്യം വരുന്ന രീതിയിൽ sort ചെയ്തു, പിന്നെ എല്ലാ marks-ഉം 10 കൊണ്ട് ഹരിച്ചു. Index 1, 0, 2 ആയത് row-കൾ അവയുടെ label-ഉം കൊണ്ടാണ് നീങ്ങിയത് എന്നതുകൊണ്ടാണ്. Correlation 0.5 എന്നാൽ പ്രായവും marks-ഉം തമ്മിൽ ഇടത്തരം positive ബന്ധം.

### EXAMPLE: CONCAT AND MERGE

```
import pandas as pd

df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [1, 6], 'B': [7, 8]})
print(pd.concat([df1, df2]))
print(pd.merge(df1, df2, on='A'))
```

```
   A  B
0  1  3
1  2  4
0  1  7
1  6  8
   A  B_x  B_y
0  1    3    7
```

concat puts df2's rows under df1's (the old index labels 0, 1 repeat; pass ignore\_index=True to renumber). merge keeps only values of A found in both tables: A = 1. Both tables have a column B, so Pandas names them B\_x (from df1) and B\_y (from df2).

Concat-ൽ നാല് rows കിട്ടി, രണ്ട് tables-ഉം അടുക്കി വെച്ചു. Merge-ൽ A = 1 മാത്രമാണ് രണ്ടിലും ഉള്ളത്, അതുകൊണ്ട് ഒരു row മാത്രം. രണ്ടിലും B ഉള്ളതുകൊണ്ട് B\_x, B\_y എന്ന് പേരിട്ടു.

### EXAMPLE: NUMPY STD VS PANDAS STD

```
import numpy as np, pandas as pd

v = [1, 2, 3, 4, 5]
print("NumPy std :", np.std(v))
print("Pandas std:", pd.Series(v).std())
print("np.std(ddof=1):", np.std(v, ddof=1))
```

```
NumPy std : 1.4142135623730951
Pandas std: 1.5811388300841898
np.std(ddof=1): 1.5811388300841898
```

NumPy divides by n (population std:  $\sqrt{(10/5)} = 1.414$ ); Pandas divides by n - 1 (sample std:  $\sqrt{(10/4)} = 1.581$ ). ddof=1 makes NumPy match Pandas.

ഒരേ data-ക്ക് NumPy-യും Pandas-ഉം വേറെ std തരുന്നു: NumPy n കൊണ്ടും Pandas n - 1 കൊണ്ടും ഹരിക്കുന്നു. ഉത്തരം ചെറുതായി വ്യത്യാസപ്പെട്ടാൽ പേടിക്കേണ്ട, കാരണം ഇതാണ്.

**Exam tip:** The unit says `pd.concat()` "concatenates two or more dataframes based on columns". By default it stacks rows (`axis=0`), as the output shows; use `axis=1` to place them side by side. Advantages of Pandas: easy syntax, Series/DataFrame, CSV/Excel/JSON support, integrates with NumPy, scikit-learn, TensorFlow. Limitation: works in one machine's memory, so very large or distributed data is hard.

## Matplotlib: visualising data

**Definition:** Matplotlib is a Python plotting library for creating static, animated and interactive visualisations; its pyplot module is imported as `plt`.

Static, animated, interactive visualisations ഉണ്ടാക്കാനുള്ള Python plotting library ആണ് Matplotlib; അതിന്റെ pyplot module `plt` എന്ന പേരിൽ import ചെയ്യുന്നു.

Visualisation shows patterns, trends and outliers that are hard to see in raw numbers. **Matplotlib** (imported as `import matplotlib.pyplot as plt`) creates static, animated and interactive plots. In AI it is used for exploratory data analysis, plotting loss/accuracy curves during training, showing feature importance and presenting results.

### മലയാളത്തിൽ

ആയിരം numbers നോക്കിയാൽ ഒന്നും മനസ്സിലാകില്ല, ഒരു graph നോക്കിയാൽ പെട്ടെന്ന് trend കാണാം, cricket-ലെ run-rate graph പോലെ. Line plot trend-ന്, bar chart താരതമ്യത്തിന്, histogram distribution-ന്, scatter രണ്ട് variables തമ്മിലുള്ള ബന്ധത്തിന്, pie chart ഭാഗങ്ങളുടെ ശതമാനത്തിന്.



| Plot      | Function                                               | Use in AI                         |
|-----------|--------------------------------------------------------|-----------------------------------|
| Line      | <code>plt.plot(x, y, color, linestyle, marker)</code>  | Trends, loss curves               |
| Bar       | <code>plt.bar(labels, values)</code>                   | Compare model accuracies          |
| Histogram | <code>plt.hist(data, bins=30)</code>                   | Distribution of a feature         |
| Scatter   | <code>plt.scatter(x, y)</code>                         | Correlation between two variables |
| Pie       | <code>plt.pie(sizes, labels, autopct='%1.1f%%')</code> | Class proportions                 |
| Subplots  | <code>fig, axs = plt.subplots(2, 2)</code>             | Several views side by side        |

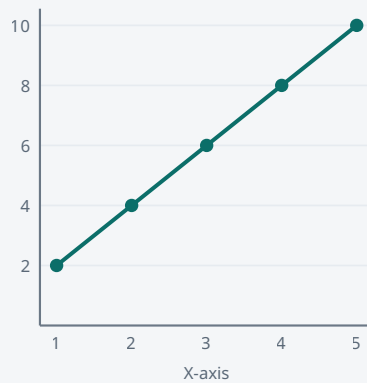
`plt.title()`, `plt.xlabel()`, `plt.ylabel()` – labels

`plt.show()` – display the figure; `plt.savefig("plot.png")` – save as PNG/JPG/PDF/SVG

### EXAMPLE: LINE PLOT

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.plot(x, y, color='black', linestyle='--', marker='o')
plt.title("Simple Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.savefig("line.png")
plt.show()
```



What the program draws: the points (1, 2) ... (5, 10) marked with circles and joined by a dashed line ( $y = 2x$ ).

`plt.plot` joins the points (`x[i]`, `y[i]`); `color` sets the line colour, `linestyle='--'` makes it dashed, `marker='o'` puts a circle on each data point. `title`, `xlabel`, `ylabel` add text. `savefig` writes the picture to a file and must come before `show()`, because `show()` clears the figure when its window closes.

`plot` ഓരോ (`x`, `y`) point-ഉം വരകൊണ്ട് കൂട്ടിച്ചേർക്കുന്നു. '--' ഇടവിട്ട വര, 'o' ഓരോ point-ലും ഒരു വട്ടം. `savefig` file ആയി സൂക്ഷിക്കുന്നു, അത് `show()`-ന് മുമ്പ് എഴുതണം, അല്ലെങ്കിൽ ശൂന്യമായ picture save ആകും.

### EXAMPLE: 2×2 SUBPLOTS

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
fig, axs = plt.subplots(2, 2)
axs[0, 0].plot(x, y);      axs[0, 0].set_title("Line Plot")
axs[0, 1].bar(x, y);       axs[0, 1].set_title("Bar Chart")
axs[1, 0].hist(x, bins=3);  axs[1, 0].set_title("Histogram")
axs[1, 1].scatter(x, y);    axs[1, 1].set_title("Scatter Plot")
plt.tight_layout()
plt.show()
```

`plt.subplots(2, 2)` returns a figure and a 2×2 grid of axes; `axs[row, col]` picks one panel. On an axes object the title method is `set_title` (not `title`). `tight_layout()` spaces the panels so titles do not overlap. The output is one window with four small charts.

ഒരു പേജിനെ നാല് കളികളാക്കി ഓരോന്നിലും ഓരോ graph വരക്കുന്നു. `axs[0, 1]` എന്നാൽ മുകളിലെ row, വലത്തെ കളി. `tight_layout` തലക്കെട്ടുകൾ തമ്മിൽ കൂട്ടിമുട്ടാതെ അകലം ഇടുന്നു.

**Exam tip:** The unit's scatter-plot program ends with `plt.show` without brackets. That only names the function and does not call it, so in a plain script no window appears (Jupyter still draws the plot automatically, which hides the bug). Write `plt.show()`. Advantages of Matplotlib: flexible, many chart types, works with NumPy/Pandas, fully customisable. Limitation: verbose syntax, detailed plots need many lines.

## Scikit-learn: the ML workflow and Linear Regression

**Scikit-learn:** The standard Python library for classical machine learning, in which every model is trained with `fit()` and used with `predict()`.

**Scikit-learn:** Classical machine learning-നുള്ള standard Python library; ഇതിലെ ഓരോ model-ഉം `fit()` വഴി train ചെയ്ത് `predict()` വഴി ഉപയോഗിക്കുന്നു.

**Linear regression:** A supervised learning method that predicts a continuous value by fitting a straight line  $y = mx + c$  to the data.

**Linear regression:** Data-യിലേക്ക്  $y = mx + c$  എന്ന നേർരേഖ `fit` ചെയ്ത് ഒരു continuous value predict ചെയ്യുന്ന supervised learning രീതി.

**Scikit-learn** (sklearn) is the standard library for classical machine learning, built on NumPy, Pandas and Matplotlib. Every model has the same interface: create it, `fit(X, y)` to train, `predict(X)` to predict. A typical task follows seven steps: load data → split into train/test → preprocess → choose model → train → evaluate → fine-tune. **Linear Regression** is supervised learning that predicts a continuous value using a straight line  $y = mx + c$ .

#### മലയാളത്തിൽ

Sklearn-ൽ ഏത് model എടുത്താലും രീതി ഒന്നുതന്നെ: `fit` കൊണ്ട് പഠിപ്പിക്കുക, `predict` കൊണ്ട് ഉത്തരം ചോദിക്കുക. Data-യുടെ ഒരു ഭാഗം (test set) മാറ്റിവെക്കുന്നത് പരീക്ഷയ്ക്ക് കാണാത്ത ചോദ്യം ചോദിക്കുന്ന പോലെയാണ്, model ശരിക്കും പഠിച്ചോ എന്ന് അറിയാൻ. Linear regression ഒരു നേർരേഖ വരച്ച് പഠിച്ച മണിക്കൂറിൽ നിന്ന് marks പ്രവചിക്കുന്നു.

`X = features (2-D, e.g. df[['Hours']]); y = target (1-D)`

`train_test_split(X, y, test_size=0.2, random_state=42)`

Linear regression:  $\hat{y} = m \cdot x + c$ ,  $m$  and  $c$  chosen to minimise  $\sum (y - \hat{y})^2$

$m = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sum (x - \bar{x})^2}$ ,  $c = \bar{y} - m \cdot \bar{x}$

### EXAMPLE: PREDICT MARKS FROM HOURS STUDIED

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df = pd.DataFrame({'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8],
                   'Marks': [30, 35, 50, 55, 65, 70, 75, 85]})
X = df[['Hours_Studied']]
y = df['Marks']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Test hours:", X_test['Hours_Studied'].tolist())
print("Actual marks:", y_test.tolist())
print("Predicted Marks:", y_pred.round(2))
print("Slope:", round(model.coef_[0], 2), "Intercept:", round(model.intercept_, 2))
```

```
Test hours: [2, 6]
Actual marks: [35, 70]
Predicted Marks: [40. 70.]
Slope: 7.5 Intercept: 25.0
```

Line by line: X uses double brackets because sklearn needs a 2-D feature table; y is one column.

test\_size=0.2 keeps 20% of 8 rows (2 rows: hours 2 and 6) for testing; random\_state=42 makes the split repeatable. fit learns the line from the other 6 rows; predict applies it to the test hours. Hand check on the training rows (hours 1, 3, 4, 5, 7, 8; marks 30, 50, 55, 65, 75, 85):  $\bar{x} = 28/6 = 4.667$ ,  $\bar{y} = 60$ ,  $\sum(x - \bar{x})(y - \bar{y}) = 250$ ,  $\sum(x - \bar{x})^2 = 33.33$ , so  $m = 7.5$  and  $c = 60 - 7.5 \times 4.667 = 25$ . Predictions:  $7.5 \times 2 + 25 = 40$  and  $7.5 \times 6 + 25 = 70$ .

8 rows-ൽ 2 എണ്ണം test-ന് മാറ്റിവെച്ചു, ബാക്കി 6 കൊണ്ട് model രേഖ പഠിച്ചു: Marks =  $7.5 \times \text{Hours} + 25$ . ഒരു മണിക്കൂർ കൂടുതൽ പഠിച്ചാൽ 7.5 mark കൂടും എന്നാണ് slope പറയുന്നത്. 6 മണിക്കൂറിന് 70 എന്ന് കൃത്യമായി പ്രവചിച്ചു, 2 മണിക്കൂറിന് 40 പ്രവചിച്ചു, യഥാർത്ഥം 35.

**Exam tip:** The unit's regression and decision-tree programs call train\_test\_split without random\_state, so the split (and the printed "Predicted Marks: [54.17 85.60]") changes on every run. Always add random\_state when you want reproducible output, and remember X must be 2-D (df[['col']], not df['col']).

## Scikit-learn: Decision Tree and K-Means

**Decision tree:** A supervised model that predicts a class by learning a tree of if-else questions on the features.

**Decision tree:** Features-നെക്കുറിച്ചുള്ള if-else ചോദ്യങ്ങളുടെ ഒരു tree പരിച്ച് class predict ചെയ്യുന്ന supervised model.

**K-Means:** An unsupervised clustering algorithm that divides n points into k clusters, assigning each point to the cluster with the nearest centroid (mean).

**K-Means:** n points-നെ k clusters ആയി വിഭജിക്കുന്ന unsupervised clustering algorithm; ഓരോ point-നെയും ഏറ്റവും അടുത്ത centroid (mean) ഉള്ള cluster-ലേക്ക് ചേർക്കുന്നു.

A **DecisionTreeClassifier** predicts a category by learning a tree of if-else questions on the features; `accuracy_score` measures the fraction of correct predictions. **K-Means** is unsupervised clustering: it puts n points into k groups so each point is near its group's centre (centroid). No labels are needed. Scikit-learn's strengths: beginner friendly, almost all classical algorithms, reliable, integrates with NumPy/Pandas. Limitations: not meant for deep learning, suits small-to-medium data (very large data needs Spark ML or GPUs), and has limited online/real-time learning.

### മലയാളത്തിൽ

Decision tree ഒരു ഡോക്ടർ ചോദ്യങ്ങൾ ചോദിച്ച് രോഗം കണ്ടുപിടിക്കുന്ന പോലെയാണ്: "Score 62.5-ൽ കൂടുതലാണോ?" എന്ന് ചോദിച്ച് pass/fail തീരുമാനിക്കും. K-Means-ന് ആരും ഉത്തരം പറഞ്ഞുകൊടുക്കുന്നില്ല, കല്യാണത്തിന് വന്നവരെ അടുത്തടുത്ത് ഇരിക്കുന്നവരുടെ കൂട്ടങ്ങളാക്കി തിരിക്കുന്ന പോലെ അത് സ്വയം groups ഉണ്ടാക്കുന്നു.

Accuracy = correct predictions / total predictions

K-Means: repeat { assign each point to nearest centroid; centroid = mean of its points } until no change

## EXAMPLE: PASS/FAIL WITH A DECISION TREE

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

df = pd.DataFrame({'Age': [18, 19, 20, 21, 22, 23],
                   'Score': [55, 60, 65, 70, 75, 80],
                   'Passed': [0, 0, 0, 1, 1, 1]})
X = df[['Age', 'Score']]
y = df['Passed']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
clf = DecisionTreeClassifier(random_state=0)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Test rows:", X_test.index.tolist())
print("Predicted:", y_pred.tolist(), "Actual:", y_test.tolist())
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Test rows: [2, 1]
Predicted: [1, 0] Actual: [0, 0]
Accuracy: 0.5
```

30% of 6 rows rounds up to 2 test rows (rows 2 and 1). The tree trains on rows 0, 3, 4, 5, i.e. one fail (age 18, score 55) and three passes (ages 21–23). The only split it can learn is halfway between them (Age 19.5 or Score 62.5). Row 1 (19, 60) falls on the fail side, correct; row 2 (20, 65) falls on the pass side, wrong. So accuracy =  $1/2 = 0.5$ . The lesson: with 6 rows the model cannot learn where the real boundary (between 20 and 21) is. Accuracy depends heavily on data size and on the split.

Training-ൽ കണ്ടത് 18 വയസ്സുള്ള ഒരു fail-ഉം 21–23 വയസ്സുള്ള pass-കളും മാത്രം. അതുകൊണ്ട് tree 19.5-ൽ അതിർ വെച്ചു, 20 വയസ്സുകാരനെ pass എന്ന് തെറ്റായി പറഞ്ഞു. ഇത്ര ചെറിയ data-യിൽ accuracy 0.5 ആയത് model-ന്റെ കുറ്റമല്ല, data കുറവാണ്.

### EXAMPLE: K-MEANS WITH 2 CLUSTERS

```
import numpy as np
from sklearn.cluster import KMeans

X = np.array([[1, 2], [1, 4], [1, 0], [10, 2], [10, 4], [10, 0]])
kmeans = KMeans(n_clusters=2, random_state=0, n_init=10).fit(X)
print("Cluster Centers:\n", kmeans.cluster_centers_)
print("Labels:", kmeans.labels_)
print("New point [0, 3] goes to cluster", kmeans.predict([[0, 3]])[0])
```

```
Cluster Centers:
[[10.  2.]
 [ 1.  2.]]
Labels: [1 1 1 0 0 0]
New point [0, 3] goes to cluster 1
```

The six points form two obvious groups,  $x = 1$  and  $x = 10$ . Each centre is the mean of its group:  $((1+1+1)/3, (2+4+0)/3) = (1, 2)$  and  $(10, 2)$ . `labels_` gives each point's cluster number; the numbers 0/1 are arbitrary names. `predict` assigns a new point to its nearest centre:  $(0, 3)$  is nearest to  $(1, 2)$ . The unit then plots the points coloured by `labels_` with `plt.scatter(X[:,0], X[:,1], c=kmeans.labels_)` and marks the centres with a red "X".

$x = 1$ -ൽ മൂന്ന് points,  $x = 10$ -ൽ മൂന്ന്, അതുകൊണ്ട് രണ്ട് കൂട്ടം. ഓരോ കൂട്ടത്തിന്റെയും നടുവ് (mean) ആണ് centre. Cluster-ന് 0 എന്നോ 1 എന്നോ കിട്ടുന്ന നമ്പർ വെറും പേരാണ്, അതിന് ക്രമമില്ല.

**Exam tip:** In scikit-learn 1.2–1.3, KMeans without `n_init` prints a FutureWarning (the default changed to 'auto' in 1.4). Passing `n_init=10` avoids it on every version.



## TensorFlow and Keras

**TensorFlow:** Google's open-source machine learning and deep learning framework in which data, stored as tensors, flows through a computation graph.

**TensorFlow:** Tensors ആയി സൂക്ഷിക്കുന്ന data ഒരു computation graph-യുടെ ഒഴുക്കുന്ന, Google-ന്റെ open-source machine learning, deep learning framework.

**Tensor:** A multidimensional array: a scalar is 0-D, a vector 1-D, a matrix 2-D, and images or videos use higher dimensions.

**Tensor:** ഒരു multidimensional array: scalar 0-D, vector 1-D, matrix 2-D; images, videos എന്നിവയ്ക്ക് കൂടുതൽ dimensions.

**Keras:** TensorFlow's high-level API for building neural networks layer by layer.

**Keras:** Neural networks layer by layer ആയി നിർമ്മിക്കാനുള്ള TensorFlow-ന്റെ high-level API.

**TensorFlow** is Google Brain's open-source ML/DL framework (2015). Data is stored as **tensors**, multidimensional arrays: a scalar is a 0-D tensor, a vector 1-D, a matrix 2-D, and images or videos use higher dimensions. Operations form a **computation graph** in which nodes are operations and edges are tensors flowing between them. TensorFlow 1.x used static graphs (define the graph, then run it in a session: powerful but hard to debug); TensorFlow 2.0 introduced **eager execution**, where operations run immediately like normal Python. **Keras** is TensorFlow's high-level API for building neural networks layer by layer.

### മലയാളത്തിൽ

Tensor എന്നത് scalar, vector, matrix എന്നിവയുടെ പൊതുരൂപമാണ്: ഒരു number 0-D, ഒരു list 1-D, ഒരു table 2-D, ഒരു colour photo (height × width × 3) 3-D.

TensorFlow എന്ന പേര് തന്നെ "tensors operations-യുടെ ഒഴുക്കുന്ന" എന്നതിൽ നിന്നാണ്. Keras ഉപയോഗിച്ചാൽ Lego കട്ടകൾ അടുക്കുന്ന പോലെ layers ഒന്നിന് മുകളിൽ ഒന്നായി വെച്ച് neural network ഉണ്ടാക്കാം.

Dense layer parameters = (inputs × units) + units (one bias per unit)

Dense(16) on 10 inputs:  $10 \times 16 + 16 = 176$ ; Dense(1) on 16 inputs:  $16 + 1 = 17$ ; total 193

## EXAMPLE: A TENSOR AND A SMALL NETWORK

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

x = tf.constant([[1, 2], [3, 4]])
print(x)

model = keras.Sequential([
    keras.Input(shape=(10,)),
    layers.Dense(16, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
model.summary()
```

```
tf.Tensor(
[[1 2]
 [3 4]], shape=(2, 2), dtype=int32)
Model: "sequential"
```

| Layer (type)    | Output Shape | Param # |
|-----------------|--------------|---------|
| dense (Dense)   | (None, 16)   | 176     |
| dense_1 (Dense) | (None, 1)    | 17      |

```
Total params: 193 (772.00 B)
Trainable params: 193 (772.00 B)
Non-trainable params: 0 (0.00 B)
```

`tf.constant` makes a fixed  $2 \times 2$  tensor (a rank-2 tensor) of 32-bit integers. `Sequential` stacks layers in order. `keras.Input(shape=(10,))` says each sample has 10 features. `Dense(16, activation='relu')` is a fully connected hidden layer of 16 neurons; `Dense(1, activation='sigmoid')` outputs one number between 0 and 1 (a probability for a yes/no class). None in the output shape is the batch size, decided later. The parameter counts match the formula above;  $193 \text{ parameters} \times 4 \text{ bytes} = 772 \text{ bytes}$ .

ആദ്യം ഒരു  $2 \times 2$  tensor ഉണ്ടാക്കി print ചെയ്തു. പിന്നെ network: 10 inputs, 16 neurons ഉള്ള hidden layer, ഒരു output neuron. ReLU negative values-നെ 0 ആക്കുന്നു, sigmoid output-നെ 0-നും 1-നും ഇടയിലാക്കുന്നു, അത് probability ആയി വായിക്കാം. Summary-യിൽ ഓരോ layer-ലും എത്ര weights ഉണ്ടെന്ന് കാണാം:  $176 + 17 = 193$ .

## EXAMPLE: COMPILE AND TRAIN

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers

keras.utils.set_random_seed(0)
X_train = np.random.rand(100, 10)
y_train = (X_train.sum(axis=1) > 5).astype(int) # label 1 if features sum to more than 5

model = keras.Sequential([
    keras.Input(shape=(10,)),
    layers.Dense(16, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=100, verbose=0)
loss, acc = model.evaluate(X_train, y_train, verbose=0)
print("Training accuracy:", round(acc, 2))
```

Training accuracy: 0.73

We make 100 random samples with a simple rule for the label. `compile` chooses the optimiser (Adam, a form of gradient descent), the loss (binary cross-entropy for yes/no output) and the metric to report. `fit` runs 100 epochs (passes over the data); TensorFlow does backpropagation and weight updates automatically. `verbose=0` hides the per-epoch log. `evaluate` returns loss and accuracy.

`compile` എങ്ങനെ പഠിക്കണം (optimizer), എന്ത് തെറ്റ് കുറയ്ക്കണം (loss), എന്ത് അളക്കണം (accuracy) എന്ന് പറയുന്നു. `fit` data 100 തവണ കാണിച്ച് weights ശരിയാക്കുന്നു, backpropagation TensorFlow തന്നെ ചെയ്യും. കൂടുതൽ epochs കൊടുത്താൽ accuracy കൂടും.

**Exam tip:** The unit's model uses `layers.Dense(16, activation='relu', input_shape=(10,))`. In current Keras 3 this still runs but prints a `UserWarning` ("Do not pass an `input_shape/input_dim` argument to a layer ... prefer using an `Input(shape)` object"). Use `keras.Input(shape=(10,))` as the first item, as above. The layer names ("sequential\_3", "dense\_6") in the unit's output only mean it was run several times in the same notebook.

# PyTorch

**PyTorch:** An open-source deep learning framework from Facebook AI Research that uses tensors and a dynamic computation graph built as the code runs.

**PyTorch:** Tensors-ഉം, code run ചെയ്യുമ്പോൾ തന്നെ നിർമ്മിക്കപ്പെടുന്ന dynamic computation graph-ഉം ഉപയോഗിക്കുന്ന, Facebook AI Research-ന്റെ open-source deep learning framework.

**Autograd:** PyTorch's engine that automatically computes gradients for backpropagation.

**Autograd:** Backpropagation-നുള്ള gradients സ്വയം കണക്കാക്കുന്ന PyTorch-ന്റെ engine.

**PyTorch** was released by Facebook AI Research (FAIR) in 2016. It uses a **dynamic computation graph**: the graph is built as the code runs, so models feel like ordinary Python and are easy to debug (TensorFlow 1.x needed the whole static graph first). Its basic object is the **tensor**, like a NumPy array but able to run on GPUs. **Autograd** automatically computes gradients for backpropagation, and **torch.nn** provides layers, activation functions and loss functions.

## മലയാളത്തിൽ

PyTorch-ൽ code എഴുതുന്നത് സാധാരണ Python എഴുതുന്ന പോലെയാണ്, graph വഴിയിൽ വെച്ച് തന്നെ ഉണ്ടാകുന്നു, അതുകൊണ്ട് researchers-നും students-നും ഇഷ്ടമാണ്. Autograd ഒരു കണക്ക് മിഷൻ പോലെയാണ്: നമ്മൾ function എഴുതിയാൽ മതി, derivative അത് തന്നെ കണ്ടുപിടിക്കും.

$$y = x^3 + 4x \Rightarrow dy/dx = 3x^2 + 4; \text{ at } x = 2: 3(4) + 4 = 16$$

`nn.Linear(in, out) parameters = in × out + out`

## EXAMPLE: TENSORS, AUTOGRAD AND NN.SEQUENTIAL

```
import torch
import torch.nn as nn

x = torch.tensor([[1, 2], [3, 4]])
print(x)
print(x * 2)

x = torch.tensor(2.0, requires_grad=True)
y = x ** 3 + 4 * x
y.backward()
print(x.grad)

model = nn.Sequential(
    nn.Linear(2, 3), # 2 inputs -> 3 hidden neurons
    nn.ReLU(),
    nn.Linear(3, 1) # 3 hidden -> 1 output
)
print(model)
print(sum(p.numel() for p in model.parameters()))
```

```
tensor([[1, 2],
        [3, 4]])
tensor([[2, 4],
        [6, 8]])
tensor(16.)
Sequential(
  (0): Linear(in_features=2, out_features=3, bias=True)
  (1): ReLU()
  (2): Linear(in_features=3, out_features=1, bias=True)
)
13
```

`torch.tensor` makes a  $2 \times 2$  tensor and `x * 2` doubles every element. `requires_grad=True` tells PyTorch to record operations on `x`; `y.backward()` computes  $dy/dx$  and stores it in `x.grad = 16`, matching  $3x^2 + 4$  at `x = 2`. `nn.Sequential` stacks a  $2 \rightarrow 3$  linear layer, a ReLU and a  $3 \rightarrow 1$  linear layer. Parameters:  $(2 \times 3 + 3) + (3 \times 1 + 1) = 9 + 4 = 13$ .

`requires_grad=True` എന്നാൽ "ഈ variable-ന്റെ derivative പിന്നീട് വേണം" എന്ന്. `backward()` വിളിച്ചപ്പോൾ  $3x^2 + 4 = 16$  എന്ന് കണക്കാക്കി `x.grad`-ൽ ഇട്ടു. Model-ൽ 2 inputs, 3 hidden neurons, 1 output, ആകെ 13 weights-ഉം biases-ഉം.

**Exam tip:** The unit does not print the autograd output; it is `tensor(16.)`. Compare TensorFlow and PyTorch: both are DL frameworks using tensors and automatic differentiation; TensorFlow (Google, 2015) had static graphs until 2.0 and uses Keras as its high-level API; PyTorch (FAIR, 2016) uses dynamic graphs and `torch.nn`.

## N-Queen problem (backtracking)

**N-Queen problem:** Placing N queens on an  $N \times N$  chessboard so that no two queens attack each other (share a row, column or diagonal).

**N-Queen problem:**  $N \times N$  chessboard-ൽ, രണ്ട് queens-ഉം പരസ്പരം ആക്രമിക്കാത്ത വിധം (ഒരേ row, column, diagonal പങ്കിടാത്ത വിധം) N queens-നെ വെക്കൽ.

**Backtracking:** Building a solution step by step and undoing the last choice whenever it leads to a dead end, then trying the next option.

**Backtracking:** ഘട്ടം ഘട്ടമായി solution നിർമ്മിക്കുകയും, ഒരു choice dead end-ൽ എത്തിയാൽ അത് പിൻവലിച്ച് അടുത്ത option പരീക്ഷിക്കുകയും ചെയ്യുന്ന രീതി.

Place N queens on an  $N \times N$  board so that no two attack each other (same row, column or diagonal). The program places one queen per row. For each column of the current row, `is_safe` checks the column above and the two upper diagonals; if safe, the queen is placed and the function recurses to the next row. After the recursive call the queen is removed (**backtracking**) so the next column can be tried. When `row == n`, all queens are placed and a solution is recorded.

### മലയാളത്തിൽ

ഒരു row-ൽ ഒരു queen മാത്രം വെക്കുന്നതുകൊണ്ട് row-ൽ ആക്രമണം ഉണ്ടാകില്ല, അതുകൊണ്ട് column-ഉം മുകളിലെ രണ്ട് diagonals-ഉം മാത്രം നോക്കിയാൽ മതി. വഴി അടഞ്ഞാൽ തിരിച്ചുപോയി മുമ്പത്തെ queen-നെ മാറ്റി വെക്കുന്നതാണ് backtracking, ഒരു maze-ൽ വഴി തെറ്റിയാൽ അവസാനത്തെ junction-ലേക്ക് തിരിച്ചു പോകുന്ന പോലെ.

Queens at  $(r1, c1)$  and  $(r2, c2)$  attack if  $c1 = c2$  or  $|r1 - r2| = |c1 - c2|$

Number of solutions:  $N = 4 \rightarrow 2$ ,  $N = 5 \rightarrow 10$ ,  $N = 6 \rightarrow 4$ ,  $N = 8 \rightarrow 92$  ( $N = 2, 3 \rightarrow \text{none}$ )

### EXAMPLE: ALL SOLUTIONS FOR N = 4

```
def is_safe(board, row, col):
    n = len(board)
    for i in range(row):           # same column above
        if board[i][col] == 1:
            return False
    i, j = row, col               # upper-left diagonal
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1; j -= 1
    i, j = row, col               # upper-right diagonal
    while i >= 0 and j < n:
        if board[i][j] == 1:
            return False
        i -= 1; j += 1
    return True

def solve(board, row, solutions):
    n = len(board)
    if row == n:                  # all queens placed
        solutions.append([" ".join("Q" if c else "." for c in r) for r in board])
        return
    for col in range(n):
        if is_safe(board, row, col):
            board[row][col] = 1    # place queen
            solve(board, row + 1, solutions)
            board[row][col] = 0    # backtrack

def n_queens(n):
    board = [[0] * n for _ in range(n)]
    solutions = []
    solve(board, 0, solutions)
    return solutions

sols = n_queens(4)
print("Number of solutions:", len(sols))
for s in sols:
    print("\n".join(s))
    print()
```

```
Number of solutions: 2
```

```
. Q . .  
. . . Q  
Q . . .  
. . Q .  
  
. . Q .  
Q . . .  
. . . Q  
. Q . .
```

board is an  $n \times n$  grid of 0/1. `is_safe` only looks upward, because rows below are still empty. `solve` is the recursive backtracking function: base case `row == n` saves a copy of the board as text; otherwise it tries every column, places a queen where safe, recurses, then resets the square to 0. Trace for  $N = 4$ : row 0 col 0 → row 1 col 2 → row 2 has no safe column → backtrack ... the first full solution found is columns (1, 3, 0, 2), the second (2, 0, 3, 1), a mirror image of the first.

`is_safe` മുകളിലേക്ക് മാത്രം നോക്കുന്നു, താഴെ ഇതുവരെ queen ഇല്ലല്ലോ. `solve` ഓരോ column-ഉം പരീക്ഷിക്കുന്നു, safe ആണെങ്കിൽ queen വെച്ച് അടുത്ത row-ലേക്ക് പോകുന്നു, തിരിച്ച് വന്നാൽ queen എടുത്തുമാറ്റുന്നു.  $4 \times 4$  board-ന് രണ്ട് ഉത്തരങ്ങൾ മാത്രം, അവ പരസ്പരം കണ്ണാടി പ്രതിബിംബങ്ങളാണ്.

**Exam tip:** The unit's text says the program "prints the total number of them", but its code only prints the boards; the version above collects solutions and prints the count. The unit's answer key (Check Your Progress 2, Q1) says `is_safe()` checks the column, row and diagonals. It actually checks only the column and the two upper diagonals; rows need no check because the algorithm puts exactly one queen in each row.

## Min-Max algorithm for Tic-Tac-Toe

**Definition:** Min-Max is a recursive algorithm for two-player games that explores the game tree to terminal states and backs up values, Max taking the maximum and Min the minimum, to choose the optimal move.

രണ്ട് പേർ കളിക്കുന്ന games-നുള്ള recursive algorithm ആണ് Min-Max; game tree terminal states വരെ പരിശോധിച്ച്, Max പരമാവധിയും Min ഏറ്റവും കുറഞ്ഞതും എടുത്ത് values മുകളിലേക്ക് കൊണ്ടുവന്ന്, optimal move തിരഞ്ഞെടുക്കുന്നു.

**Min-Max** is for two-player games where the Maximizer (the AI, "X") wants a high score and the Minimizer (the opponent, "O") wants a low score, both playing optimally. The algorithm recursively explores the game tree to terminal states (win, loss, draw), scores them and backs the values up: Max nodes take the maximum of their children, Min nodes the minimum. The `evaluate` function gives +10 if X has three in a line, -10 if O does, else



0. Subtracting or adding depth makes the AI prefer quicker wins and slower losses. `find_best_move` tries every empty square and picks the one with the highest minimax value.

#### മലയാളത്തിൽ

Minimax ഒരു ചെസ്സ് കളിക്കാരൻ "ഞാൻ ഇങ്ങനെ നീക്കിയാൽ എതിരാളി ഏറ്റവും മോശം മറുപടി തരും, എന്നാലും എനിക്ക് ഏറ്റവും നല്ലത് ഏത്" എന്ന് ചിന്തിക്കുന്ന പോലെയാണ്. X-ന്റെ ഊഴത്തിൽ max, O-യുടെ ഊഴത്തിൽ min. Depth കുറച്ചാൽ വേഗം ജയിക്കുന്ന വഴിക്ക് കൂടുതൽ score കിട്ടും.

Base case: X wins  $\rightarrow 10 - \text{depth}$ ; O wins  $\rightarrow -10 + \text{depth}$ ; board full  $\rightarrow 0$

Max node value = max over children; Min node value = min over children

**EXAMPLE: X MUST BLOCK O**

```

import math

LINES = [(r, 0), (r, 1), (r, 2)] for r in range(3)] + \
        [(0, c), (1, c), (2, c)] for c in range(3)] + \
        [(0, 0), (1, 1), (2, 2)], [(0, 2), (1, 1), (2, 0)]

def evaluate(board):
    for line in LINES:
        a, b, c = (board[i][j] for i, j in line)
        if a == b == c == "X":
            return 10
        if a == b == c == "O":
            return -10
    return 0

def empty_cells(board):
    return [(i, j) for i in range(3) for j in range(3) if board[i][j] == "_"]

calls = 0
def minimax(board, depth, is_max):
    global calls
    calls += 1
    score = evaluate(board)
    if score == 10:
        return score - depth          # prefer faster wins
    if score == -10:
        return score + depth          # prefer slower losses
    if not empty_cells(board):
        return 0                      # draw
    best = -math.inf if is_max else math.inf
    for i, j in empty_cells(board):
        board[i][j] = "X" if is_max else "O"
        val = minimax(board, depth + 1, not is_max)
        board[i][j] = "_"              # undo the move
        best = max(best, val) if is_max else min(best, val)
    return best

def find_best_move(board):
    best_val, best_move = -math.inf, None
    for i, j in empty_cells(board):
        board[i][j] = "X"
        val = minimax(board, 0, False)
        board[i][j] = "_"
        print((i, j), "->", val)
        if val > best_val:
            best_val, best_move = val, (i, j)
    return best_move

board = ["X", "O", "X"],
        ["_", "O", "_"],
        ["_", "_", "_"]

```

```
print("The AI should place at:", find_best_move(board))
print("minimax calls:", calls)
```

```
(1, 0) -> -9
(1, 2) -> -9
(2, 0) -> -9
(2, 1) -> 0
(2, 2) -> -9
The AI should place at: (2, 1)
minimax calls: 205
```

This is the same algorithm as the unit's program, written with a list of the 8 winning lines to keep it short. O threatens the middle column (0,1)–(1,1)–(2,1). Every X move except (2, 1) lets O complete that column on the next move: O wins at depth 1, value  $-10 + 1 = -9$ . Blocking at (2, 1) leads, with best play from both sides, to a draw, value 0. So Max picks (2, 1). The search visited 205 board positions.

O നടുവിലെ column-ൽ രണ്ടെണ്ണം വെച്ചിട്ടുണ്ട്. X (2, 1)-ൽ തടഞ്ഞില്ലെങ്കിൽ അടുത്ത നീക്കത്തിൽ O ജയിക്കും, അതാണ്  $-9$ . തടഞ്ഞാൽ കളി draw ആകും, 0. അതുകൊണ്ട് AI (2, 1) തിരഞ്ഞെടുക്കുന്നു. ഓരോ നീക്കവും പരീക്ഷിച്ച ശേഷം "\_" ഇട്ട് board പഴയപടി ആക്കുന്നത് ശ്രദ്ധിക്കുക.

#### EXAMPLE: THE UNIT'S OWN BOARD

The unit runs the program on X O X / O O \_ / \_ \_ \_ but does not show the output. What does it print?

O now threatens two lines: row 1 at (1, 2) and column 1 at (2, 1). X has no winning move, and one move cannot block both threats, so every move loses at depth 1. Running the same code gives (1, 2)  $\rightarrow -9$ , (2, 0)  $\rightarrow -9$ , (2, 1)  $\rightarrow -9$ , (2, 2)  $\rightarrow -9$ , and "The AI should place at: (1, 2)", simply the first move tried, since all are equal (the strict  $>$  keeps the first maximum).

**Exam tip:** The base case of the recursion (Check Your Progress 2, Q3) is a terminal state: someone has won (score  $\pm 10$ ) or no moves are left (draw, 0). Minimax is optimal but explores every move,  $O(b^d)$  positions; that is why alpha-beta pruning is added.

## Alpha-Beta pruning

**Alpha-Beta pruning:** An optimisation of Min-Max that skips branches which cannot affect the final decision, giving exactly the same result.

**Alpha-Beta pruning:** അന്തിമ തീരുമാനത്തെ ബാധിക്കാത്ത branches ഒഴിവാക്കി, അതേ ഫലം തന്നെ നൽകുന്ന Min-Max-ന്റെ optimisation.

**$\alpha$  and  $\beta$ :**  $\alpha$  is the best value the Maximizer is guaranteed so far,  $\beta$  the best (lowest) value the Minimizer is guaranteed; prune when  $\beta \leq \alpha$ .

**$\alpha$  and  $\beta$ :** Maximizer-ന് ഇതുവരെ ഉറപ്പായ ഏറ്റവും നല്ല value ആണ്  $\alpha$ ; Minimizer-ന് ഉറപ്പായ ഏറ്റവും നല്ല (കുറഞ്ഞ) value ആണ്  $\beta$ ;  $\beta \leq \alpha$  ആകുമ്പോൾ prune ചെയ്യുന്നു.

**Alpha-Beta pruning** gives exactly the same answer as Min-Max but skips branches that cannot change the decision.  $\alpha$  is the best value the Maximizer is already guaranteed on the current path;  $\beta$  is the best (lowest) value the Minimizer is guaranteed. A Max node raises  $\alpha$ , a Min node lowers  $\beta$ , and as soon as  $\beta \leq \alpha$  the remaining children of that node are pruned, because the opponent would never let play reach them.

### മലയാളത്തിൽ

$\alpha$  എന്നാൽ "എനിക്ക് ഇതിനോടകം ഉറപ്പായ ഏറ്റവും കുറഞ്ഞ നേട്ടം",  $\beta$  "എതിരാളി എനിക്ക് പരമാവധി അനുവദിക്കുന്നത്".  $\beta \leq \alpha$  ആയാൽ ആ വഴി ഇനി നോക്കേണ്ട, കല്യാണാലോചനയിൽ ആദ്യ കൂടിക്കാഴ്ചയിൽ തന്നെ ഒത്തുപോകില്ല എന്ന് ഉറപ്പായാൽ ബാക്കി വിശദാംശങ്ങൾ അന്വേഷിക്കാത്ത പോലെ. ഉത്തരം minimax-ന്റേത് തന്നെ, പക്ഷേ വേഗം കൂടുതൽ.

Start:  $\alpha = -\infty$ ,  $\beta = +\infty$

Max node: best = max(best, child);  $\alpha = \max(\alpha, \text{best})$ ; if  $\beta \leq \alpha$ : prune

Min node: best = min(best, child);  $\beta = \min(\beta, \text{best})$ ; if  $\beta \leq \alpha$ : prune

### EXAMPLE: SAME BOARD, FEWER POSITIONS

```

# uses LINES, evaluate() and empty_cells() from the Min-Max program above
import math

calls = 0
def alphabeta(board, depth, alpha, beta, is_max):
    global calls
    calls += 1
    score = evaluate(board)
    if score == 10:
        return score - depth
    if score == -10:
        return score + depth
    if not empty_cells(board):
        return 0
    if is_max:
        best = -math.inf
        for i, j in empty_cells(board):
            board[i][j] = "X"
            best = max(best, alphabeta(board, depth + 1, alpha, beta, False))
            board[i][j] = "_"
            alpha = max(alpha, best)
            if beta <= alpha:
                break                # prune remaining moves
        return best
    else:
        best = math.inf
        for i, j in empty_cells(board):
            board[i][j] = "O"
            best = min(best, alphabeta(board, depth + 1, alpha, beta, True))
            board[i][j] = "_"
            beta = min(beta, best)
            if beta <= alpha:
                break
        return best

def find_best_move(board):
    best_val, best_move = -math.inf, None
    for i, j in empty_cells(board):
        board[i][j] = "X"
        val = alphabeta(board, 0, -math.inf, math.inf, False)
        board[i][j] = "_"
        if val > best_val:
            best_val, best_move = val, (i, j)
    return best_move, best_val

board = [["X", "O", "X"],
         ["_", "O", "_"],
         ["_", "_", "_"]]
print("Best move and value:", find_best_move(board))
print("alphabeta calls:", calls)

```

Best move and value: ((2, 1), 0)  
alphabeta calls: 123

The only change from Min-Max is the pair  $\alpha, \beta$  passed down the recursion and the break when  $\beta \leq \alpha$ . The answer is the same move (2, 1) with value 0, but only 123 positions are visited instead of 205. Looping over the list from `empty_cells` means one break leaves the whole loop.

ഉത്തരം ഒന്നുതന്നെ (2, 1), പക്ഷേ 205-ന് പകരം 123 positions മാത്രം നോക്കി.  
ബാക്കി branches "ഇത് ഫലം മാറ്റില്ല" എന്ന് ഉറപ്പായതുകൊണ്ട് വെട്ടിക്കളഞ്ഞു.

**Exam tip:** In the unit's alpha-beta code the break sits inside two nested loops (for i / for j), so it only leaves the inner j loop and the outer loop keeps trying the next rows. The answer is still correct, but less is pruned: on the board above the unit's version makes 162 calls against 123 for the single-loop version. Pruning never changes the minimax value; it only saves time.

## The sample graph and Breadth First Search

**Adjacency list:** A graph representation that maps each node to the list of its neighbours.

**Adjacency list:** ഓരോ node-നെയും അതിന്റെ neighbours-ന്റെ list-ലേക്ക് map ചെയ്യുന്ന graph representation.

**Breadth First Search (BFS):** A search that explores a graph level by level using a FIFO queue; it finds the path with the fewest edges.

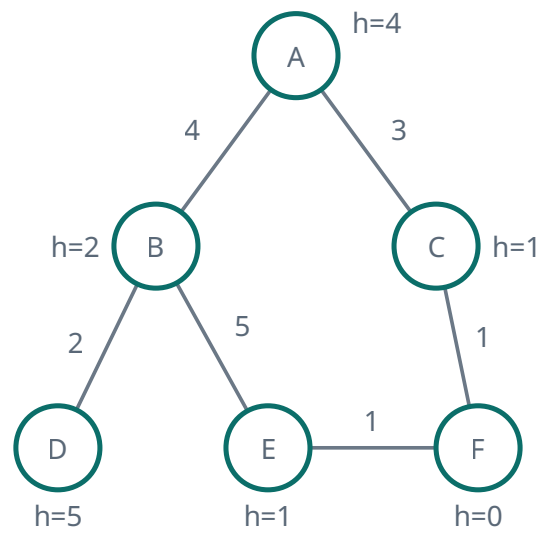
**Breadth First Search (BFS):** FIFO queue ഉപയോഗിച്ച് graph-നെ level by level ആയി പരിശോധിക്കുന്ന search; ഏറ്റവും കുറച്ച് edges ഉള്ള path ഇത് കണ്ടെത്തുന്നു.

The unit's Figures 1–3 are one small directed graph:  $A \rightarrow B, C$ ;  $B \rightarrow D, E$ ;  $C \rightarrow F$ ;  $E \rightarrow F$ . For  $A^*$  and  $IDA^*$  the edges carry costs and each node has a heuristic  $h$  (estimated distance to goal  $F$ ). **Breadth-First Search (BFS)** visits nodes level by level using a FIFO **queue** (`collections.deque`): take the front node, mark it visited, add its unvisited neighbours at the back. BFS finds the shortest path (fewest edges) in an unweighted graph and is used for social networks, web crawlers and game moves.

### മലയാളത്തിൽ

BFS ഒരു കളത്തിൽ കല്ലെറിഞ്ഞാൽ ഓളങ്ങൾ വട്ടത്തിൽ പരക്കുന്ന പോലെയാണ്: ആദ്യം അടുത്തുള്ളവർ, പിന്നെ അവരുടെ അടുത്തുള്ളവർ. Queue ബസ്സ് സ്റ്റോപ്പിലെ ക്യൂ പോലെ, ആദ്യം വന്നയാൾ ആദ്യം പോകും (FIFO).





Sample graph (reconstructed from the unit's code). Edges point downward and  $E \rightarrow F$  points right.  
Numbers on edges are the A\*/IDA\* costs; h is the A\*/IDA\* heuristic to goal F.

### EXAMPLE: BFS FROM A

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()          # take from the front
        if node not in visited:
            visited.add(node)
            order.append(node)
            for nb in graph[node]:
                if nb not in visited:
                    queue.append(nb)    # add at the back
    return order

graph = {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'],
        'D': [], 'E': ['F'], 'F': []}
print("BFS Traversal:", bfs(graph, 'A'))
```

```
BFS Traversal: ['A', 'B', 'C', 'D', 'E', 'F']
```

The graph is an adjacency list: a dictionary from each node to its neighbours. Trace of the queue: [A] → visit A, queue [B, C] → visit B, queue [C, D, E] → visit C, queue [D, E, F] → visit D → visit E, queue [F, F] → visit F → the second F is skipped because it is already visited. The visited set stops a node being processed twice.

ഓരോ ഘട്ടത്തിലും queue-ന്റെ മുന്നിൽ നിന്ന് ഒരു node എടുക്കുന്നു, അതിന്റെ അയൽക്കാരെ പിന്നിൽ ചേർക്കുന്നു. F രണ്ടുതവണ queue-ൽ കയറി (C വഴിയും E വഴിയും), പക്ഷേ visited set ഉള്ളതുകൊണ്ട് ഒരു തവണ മാത്രം എണ്ണി.

**Exam tip:** The unit's printed BFS output ends with a stray "0" (['A', 'B', 'C', 'D', 'E', 'F']0); that is a typesetting slip, the real output is just the list. The unit's figures are missing from this text; the graph above is built from the adjacency lists and costs in the code.

## Depth First Search

**Definition:** Depth First Search (DFS) explores as deep as possible along one path before backtracking, using a stack (LIFO) or recursion.

Depth First Search (DFS) ഒരു stack (LIFO) അല്ലെങ്കിൽ recursion ഉപയോഗിച്ച്, backtrack ചെയ്യുന്നതിന് മുമ്പ് ഒരു path-യുടെ പരമാവധി ആഴത്തിൽ പോകുന്നു.

**Depth-First Search (DFS)** goes as deep as possible along one path before backtracking. The unit writes it recursively: visit the node, then call DFS on each neighbour in turn. Recursion uses the call stack, so DFS is LIFO where BFS is FIFO. When DFS reaches a node with no unvisited neighbours, the call returns and the algorithm backtracks to the previous node to try its remaining neighbours. DFS is used for maze solving, path finding and topological sorting.

### മലയാളത്തിൽ

DFS ഒരു ഇടവഴിയിലൂടെ അവസാനം വരെ പോയി നോക്കി, വഴി തീർന്നാൽ തിരിച്ചു വന്ന് അടുത്ത ഇടവഴി പിടിക്കുന്ന പോലെയാണ്. BFS വീതിയിൽ, DFS ആഴത്തിൽ.

### EXAMPLE: RECURSIVE DFS FROM A

```
def dfs(graph, node, visited=None, order=None):
    if visited is None:
        visited, order = set(), []
    if node not in visited:
        visited.add(node)
        order.append(node)
        for nb in graph[node]:
            dfs(graph, nb, visited, order)    # go deeper first
    return order

graph = {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'],
        'D': [], 'E': ['F'], 'F': []}
print("DFS Traversal:", dfs(graph, 'A'))
```

DFS Traversal: ['A', 'B', 'D', 'E', 'F', 'C']

visited and order default to None and are created on the first call; writing `visited=set()` as a default would share one set between separate calls, a classic Python bug. Trace:  $A \rightarrow B \rightarrow D$  (dead end, back to B)  $\rightarrow E \rightarrow F$  (dead end, back to E, B, A)  $\rightarrow C \rightarrow$  its neighbour F is already visited, so nothing more.

A-യിൽ നിന്ന് B, B-യിൽ നിന്ന് D. D-ക്ക് അയൽക്കാർ ഇല്ല, തിരിച്ച് B-യിലേക്ക്, പിന്നെ E, E-യിൽ നിന്ന് F. അവസാനം A-യിലേക്ക് തിരിച്ച് വന്ന് C. C-യുടെ F നേരത്തെ കണ്ടതാണ്. Default ആയി None വെക്കുന്നത് ഓരോ പുതിയ call-നും പുതിയ set കിട്ടാനാണ്.

|                            | BFS                            | DFS                             |
|----------------------------|--------------------------------|---------------------------------|
| Data structure             | Queue (FIFO)                   | Stack / recursion (LIFO)        |
| Order on sample graph      | A B C D E F                    | A B D E F C                     |
| Shortest path (unweighted) | Yes                            | No                              |
| Memory                     | Stores a whole level: $O(b^d)$ | Stores one path: $O(b \cdot d)$ |

## Best First Search

**Heuristic  $h(n)$ :** An estimate of the cost from node  $n$  to the goal.

**Heuristic  $h(n)$ :** Node  $n$ -ൽ നിന്ന് goal-ലേക്കുള്ള cost-ന്റെ ഒരു ഏകദേശ കണക്ക് (estimate).

**Greedy Best First Search:** An informed search that always expands the node with the smallest  $h(n)$ , using a priority queue; it does not guarantee the shortest path.

**Greedy Best First Search:** Priority queue ഉപയോഗിച്ച് എപ്പോഴും ഏറ്റവും ചെറിയ  $h(n)$  ഉള്ള node expand ചെയ്യുന്ന informed search; ഇത് shortest path ഉറപ്പ് നൽകുന്നില്ല.

**Greedy Best First Search** uses a heuristic  $h(n)$ , an estimate of how close  $n$  is to the goal, and always expands the node with the smallest  $h$ . A **priority queue** (min-heap, Python's `heapq`) holds  $(h, \text{node})$  pairs, so `heappop` returns the most promising node. It is fast when  $h$  is good but ignores the cost already paid, so it does not guarantee the shortest path ( $A^*$  fixes this).

### മലയാളത്തിൽ

Best First Search ഒരു നാട്ടുവഴിയിൽ "ലക്ഷ്യസ്ഥാനം ഏത് ദിശയിലാണോ അങ്ങോട്ട് തോന്നുന്ന വഴി" പിടിക്കുന്ന പോലെയാണ്, ഇതുവരെ എത്ര നടന്നു എന്ന് നോക്കുന്നില്ല. Heap എപ്പോഴും ഏറ്റവും ചെറിയ  $h$  ഉള്ള node മുന്നിൽ തരും.

Expand the node with minimum  $h(n)$

```
heapq.heappush(pq, (h[n], n)); h_n, n = heapq.heappop(pq)
```

## EXAMPLE: A TO F

```
import heapq

def best_first_search(graph, h, start, goal):
    visited, pq, path = set(), [(h[start], start)], []
    while pq:
        _, node = heapq.heappop(pq)          # smallest h first
        if node not in visited:
            visited.add(node)
            path.append(node)
            if node == goal:
                return path
            for nb in graph[node]:
                if nb not in visited:
                    heapq.heappush(pq, (h[nb], nb))
    return path

graph = {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'],
        'D': [], 'E': ['F'], 'F': []}
h = {'A': 6, 'B': 4, 'C': 3, 'D': 5, 'E': 2, 'F': 0}
print("Best First path:", best_first_search(graph, h, 'A', 'F'))
h3 = {'A': 6, 'B': 2, 'C': 3, 'D': 1, 'E': 3, 'F': 0}
print("Visit order with h3:", best_first_search(graph, h3, 'A', 'F'))
```

```
Best First path: ['A', 'C', 'F']
Visit order with h3: ['A', 'B', 'D', 'C', 'F']
```

With the unit's h: pop A, push B (4) and C (3); pop C (smaller h), push F (0); pop F = goal. Result A, C, F.  
With h3 (B and D look very close): pop A; pop B (2), push D (1), E (3); pop D (1), dead end; C and E both have h = 3, and the tie is broken by name, so C is popped; push F; pop F. The returned list A, B, D, C, F is the order of expansion, not a path: there is no edge D → C.

Unit-ന്റെ h വെച്ച് C-ക്ക് ചെറിയ h ഉള്ളതുകൊണ്ട് നേരെ C വഴി F. പക്ഷേ h3 കൊടുത്താൽ B, D ആദ്യം നോക്കി, പിന്നെ C-യിലേക്ക് ചാടി. ഈ program തിരിച്ചു തന്നത് സന്ദർശിച്ച ക്രമമാണ്, യഥാർത്ഥ വഴി അല്ല.

**Exam tip:** The unit's `best_first_search` returns the list of expanded nodes and calls it "path". It equals the real path only when the search never backtracks (true for the unit's example). To return a true path, keep a parent dictionary as in the A\* program.

## A\* algorithm

**A\*:** An informed search that expands the node with the smallest  $f(n) = g(n) + h(n)$ , where  $g(n)$  is the cost so far and  $h(n)$  the estimated cost to the goal.

**A\*:** ഏറ്റവും ചെറിയ  $f(n) = g(n) + h(n)$  ഉള്ള node expand ചെയ്യുന്ന informed search;  $g(n)$  ഇതുവരെയുള്ള cost,  $h(n)$  goal-ലേക്കുള്ള estimated cost.

**Admissible heuristic:** A heuristic that never overestimates the true cost to the goal; with it A\* is guaranteed to find an optimal path.

**Admissible heuristic:** Goal-ലേക്കുള്ള യഥാർത്ഥ cost-നെക്കാൾ ഒരിക്കലും കൂടുതൽ കണക്കാക്കാത്ത heuristic; ഇതുണ്ടെങ്കിൽ A\* optimal path കണ്ടെത്തുമെന്ന് ഉറപ്പാണ്.

A\* combines Dijkstra's algorithm (cost so far) with Best First Search (estimate to go). For each node it uses  $g(n)$ , the actual cost from start to  $n$ , and  $h(n)$ , the heuristic estimate from  $n$  to the goal, and expands the node with the smallest  $f(n) = g(n) + h(n)$ . If  $h$  is **admissible** (never overestimates the true cost), A\* is guaranteed to return an optimal (cheapest) path. It is used in maps and GPS routing, robotics and puzzles.

### മലയാളത്തിൽ

A\* രണ്ടും നോക്കുന്നു: ഇതുവരെ എത്ര ദൂരം വന്നു ( $g$ ), ഇനി ഏകദേശം എത്ര ബാക്കി ( $h$ ). ഒരു ഓട്ടോക്കാരൻ "ഇതുവരെ ഓടിയ km + ഇനി ഏകദേശം km" എന്ന് കൂട്ടി ഏറ്റവും കുറഞ്ഞ വഴി തിരഞ്ഞെടുക്കുന്ന പോലെ.  $h$  ഒരിക്കലും യഥാർത്ഥ ദൂരത്തെക്കാൾ കൂടുതൽ പറയാതിരുന്നാൽ (admissible) ഏറ്റവും ചെറിയ വഴി ഉറപ്പ്.

$$f(n) = g(n) + h(n)$$

Admissible:  $h(n) \leq h^*(n)$  (true cost to goal) for every  $n$

On relaxation: if  $g(\text{parent}) + \text{cost} < g(\text{child})$ , update  $g(\text{child})$  and  $\text{parent}(\text{child})$

### EXAMPLE: A\* FROM A TO F

```
import heapq

def a_star_search(graph, h, start, goal):
    pq = [(h[start], start)]          # (f, node)
    parent = {start: None}
    g_cost = {start: 0}
    while pq:
        f, node = heapq.heappop(pq)
        if node == goal:              # rebuild path from parents
            path = []
            while node:
                path.append(node)
                node = parent[node]
            return path[::-1], g_cost[goal]
        for nb, cost in graph[node]:
            new_g = g_cost[node] + cost
            if nb not in g_cost or new_g < g_cost[nb]:
                g_cost[nb] = new_g
                parent[nb] = node
                heapq.heappush(pq, (new_g + h[nb], nb))
    return None

graph = {'A': [('B', 4), ('C', 3)], 'B': [('D', 2), ('E', 5)],
        'C': [('F', 1)], 'D': [], 'E': [('F', 1)], 'F': []}
h = {'A': 4, 'B': 2, 'C': 1, 'D': 5, 'E': 1, 'F': 0}
path, cost = a_star_search(graph, h, 'A', 'F')
print("A* Path from A to F:", path, "cost =", cost)
```

A\* Path from A to F: ['A', 'C', 'F'] cost = 4

Each graph entry is a list of (neighbour, edge cost). Trace: pop A ( $f = 4$ ). Push B with  $g = 4$ ,  $f = 4 + 2 = 6$ , and C with  $g = 3$ ,  $f = 3 + 1 = 4$ . Pop C ( $f = 4$ ), push F with  $g = 4$ ,  $f = 4 + 0 = 4$ . Pop F = goal. Follow parents  $F \rightarrow C \rightarrow A$  and reverse: A, C, F with cost  $3 + 1 = 4$ . The other route A, B, E, F costs  $4 + 5 + 1 = 10$ . The goal test is done when a node is popped, not when it is pushed; that is what makes A\* optimal.

A-യിൽ നിന്ന് B-ക്ക്  $f = 6$ , C-ക്ക്  $f = 4$ . ചെറിയ  $f$  ഉള്ള C ആദ്യം, അവിടെ നിന്ന് F-ന്  $f = 4$ . F heap-ൽ നിന്ന് പുറത്തെടുക്കുമ്പോഴാണ് goal എന്ന് തീരുമാനിക്കുന്നത്. parent dictionary പിന്നോട്ട് നോക്കി വഴി ഉണ്ടാക്കുന്നു. ആകെ ചെലവ് 4, B-E വഴി 10 ആകുമായിരുന്നു.



**Exam tip:** The unit's A\* graph contains a typo, 'F' : []4, which is a SyntaxError; it should be 'F' : []. The unit also pushes the start with f = 0 instead of h(start); harmless here because the start is popped first anyway. Why A\* over BFS for road maps (Check Your Progress 3, Q3): BFS minimises the number of edges and ignores road lengths; A\* minimises total distance and, with an admissible heuristic such as straight-line distance, finds the optimal route while expanding far fewer nodes.

## AO\* algorithm

**AND-OR graph:** A graph in which a node is solved either by any one of its OR children or by solving all of its AND children together.

**AND-OR graph:** ഒരു node-നെ അതിന്റെ OR children-ൽ ഏതെങ്കിലും ഒന്ന് വഴിയോ, എല്ലാ AND children-ഉം ഒരുമിച്ച് solve ചെയ്തോ പരിഹരിക്കുന്ന graph.

**AO\*:** A search algorithm for AND-OR graphs that finds the least-cost solution graph by revising node costs bottom-up.

**AO\*:** Node costs താഴെ നിന്ന് മുകളിലേക്ക് പുതുക്കി ഏറ്റവും കുറഞ്ഞ cost ഉള്ള solution graph കണ്ടെത്തുന്ന, AND-OR graphs-നുള്ള search algorithm.

AO\* searches an **AND-OR graph**, where a problem can be solved either by one of several alternatives (**OR** node: pick the cheapest child) or only by solving several subproblems together (**AND** node: add the costs of all children). The result is a solution graph, not a single path. The unit's program revises each node's cost bottom-up, with every edge costing  $w_t = 1$ , and then follows the cheapest option from the root to print the solution graph.

### മലയാളത്തിൽ

OR എന്നാൽ "ഇതോ അതോ ഒന്ന് മതി", ബസ്സിലോ ട്രെയിനിലോ പോകാം, വില കുറഞ്ഞത് തിരഞ്ഞെടുക്കും. AND എന്നാൽ "എല്ലാം വേണം", കല്യാണത്തിന് ഹാളും സദ്യയും രണ്ടും ബുക്ക് ചെയ്യണം, ചെലവ് കൂട്ടണം. AO\* ഇങ്ങനെയുള്ള graph-ൽ ഏറ്റവും കുറഞ്ഞ ചെലവുള്ള പരിഹാരം കണ്ടെത്തുന്നു.

OR option via child  $n$ :  $\text{cost} = h(n) + w_t$

AND arc over children  $n_1, \dots, n_k$ :  $\text{cost} = \sum (h(n_i) + w_t)$

Revised  $h(\text{node}) = \min$  over all its options

AND-OR graph used in the unit (wt = 1 on every edge)

A1 --OR--> A2                      A1 --AND--> {A3, A4}

A2 --OR--> A5 | A6

A3 --OR--> A7                      A3 --AND--> {A8, A9}

A4 --OR--> A10

Initial h: A1=-1   A2=6   A3=3   A4=5   A5=8   A6=10   A7=4   A8=0   A9=0   A10=0

## EXAMPLE: COST REVISION AND SOLUTION GRAPH

```
def cal_cost(H, cd, wt=1):
    cost = {}
    if 'AND' in cd:
        nodes = cd['AND']
        cost[' AND '.join(nodes)] = sum(H[n] + wt for n in nodes)
    if 'OR' in cd:
        for n in cd['OR']:
            # one entry per OR child
            cost[n] = H[n] + wt
    return cost

def update_cost(H, graph, wt=1):
    mincost = {}
    for key in reversed(list(graph.keys())): # leaves first
        c = cal_cost(H, graph[key], wt)
        print(key, ': ', graph[key], '->', c)
        H[key] = min(c.values()) # revise h
        mincost[key] = c
    return mincost

def shortest_path(start, cost, H):
    path = start
    if start in cost:
        best = min(cost[start], key=cost[start].get)
        nn = best.split()
        if len(nn) == 1:
            # OR: one child
            path += '<<- ' + shortest_path(nn[0], cost, H)
        else:
            # AND: both children
            path += '<<-(' + best + ') '
            path += '[' + shortest_path(nn[0], cost, H) + ' + '
            path += shortest_path(nn[-1], cost, H) + ']'
    return path

H = {'A1': -1, 'A2': 6, 'A3': 3, 'A4': 5, 'A5': 8,
     'A6': 10, 'A7': 4, 'A8': 0, 'A9': 0, 'A10': 0}
graph = {
    'A1': {'OR': ['A2'], 'AND': ['A3', 'A4']},
    'A2': {'OR': ['A5', 'A6']},
    'A3': {'OR': ['A7'], 'AND': ['A8', 'A9']},
    'A4': {'OR': ['A10']}
}
cost = update_cost(H, graph, 1)
print('Revised H:', H)
print('Solution graph:\n', shortest_path('A1', cost, H))
```

```

A4 : {'OR': ['A10']} -> {'A10': 1}
A3 : {'OR': ['A7'], 'AND': ['A8', 'A9']} -> {'A8 AND A9': 2, 'A7': 5}
A2 : {'OR': ['A5', 'A6']} -> {'A5': 9, 'A6': 11}
A1 : {'OR': ['A2'], 'AND': ['A3', 'A4']} -> {'A3 AND A4': 5, 'A2': 10}
Revised H: {'A1': 5, 'A2': 9, 'A3': 2, 'A4': 1, 'A5': 8, 'A6': 10, 'A7': 4, 'A8':
0, 'A9': 0, 'A10': 0}
Solution graph:
A1<<-(A3 AND A4) [A3<<-(A8 AND A9) [A8 + A9] + A4<<-A10]

```

Nodes are revised from the bottom (A4) to the root (A1). A4: only option A10,  $0 + 1 = 1$ . A3: OR A7 =  $4 + 1 = 5$ , AND (A8, A9) =  $(0 + 1) + (0 + 1) = 2$ , so  $h(A3) = 2$ . A2: A5 =  $8 + 1 = 9$ , A6 =  $10 + 1 = 11$ , so  $h(A2) = 9$ . A1: OR A2 =  $9 + 1 = 10$ , AND (A3, A4) =  $(2 + 1) + (1 + 1) = 5$ , so  $h(A1) = 5$ . `shortest_path` then starts at A1, takes the cheapest option (A3 AND A4), and recurses into both children: A3 via (A8 AND A9), A4 via A10. `<<-` means "is solved by".

താഴെ നിന്ന് മുകളിലേക്ക് ഓരോ node-ന്റെയും ചെലവ് പുതുക്കുന്നു. തുടക്കത്തിലെ  $h$  വെച്ച് A1-ന് A2 വഴി 7-ഉം A3 AND A4 വഴി 10-ഉം ആയിരുന്നു, A2 നല്ലതായി തോന്നി. പക്ഷേ താഴത്തെ nodes പുതുക്കിയപ്പോൾ A2 വഴി 10, A3 AND A4 വഴി 5 ആയി, അതുകൊണ്ട് അവസാന ഉത്തരം A3 AND A4. AND ആയതുകൊണ്ട് A3-ഉം A4-ഉം രണ്ടും പരിഹരിക്കണം.

**Exam tip:** Three corrections to the unit. (1) Its explanation says  $A1 \rightarrow A2$  "is followed" and that A2 becomes  $\min(9, 10)$ . With the initial  $h$  the options are 7 (A2) and 10 (A3 AND A4), but after revision  $A2 = \min(9, 11) = 9$  and the final choice is A3 AND A4 with cost 5, as its own printed output shows. (2) The unit stores all OR children under one key such as 'A5 OR A6'; `shortest_path` splits that into three words and prints it like an AND arc. Change  $h(A5)$  to 1 and the unit's code prints  $A1<<-A2<<-(A5 \text{ OR } A6) [A5 + A6]$ ; the version above gives the correct  $A1<<-A2<<-A5$ . (3) This program is a simplified one-pass bottom-up cost revision, not full AO\* (which expands the best partial solution graph step by step and propagates revised costs upward), and it prints only the first and last child of an AND arc.

## Iterative Deepening A\* (IDA\*)

**Definition:** IDA\* runs repeated depth-first searches with a cost bound on  $f(n) = g(n) + h(n)$ , raising the bound each round to the smallest  $f$  that exceeded it; it is optimal like A\* but needs little memory.

$f(n) = g(n) + h(n)$ -ന് ഒരു cost bound വെച്ച് IDA\* ആവർത്തിച്ച് depth-first searches നടത്തുന്നു; ഓരോ round-ലും bound കവിഞ്ഞ ഏറ്റവും ചെറിയ  $f$  പുതിയ bound ആകുന്നു; A\* പോലെ optimal ആണ്, പക്ഷേ വളരെ കുറച്ച് memory മതി.

**IDA\*** combines A\*'s optimality with DFS's small memory. It sets a cost **bound** =  $h(\text{start})$  and runs a depth-first search that cuts off any node with  $f(n) = g(n) + h(n) > \text{bound}$ . If the goal is not found, the new bound is the smallest  $f$  that exceeded the old bound, and DFS restarts. Only the current path is stored, not the whole frontier, so memory is  $O(\text{depth})$ . It is used for the 8-puzzle and 15-puzzle.

#### മലയാളത്തിൽ

IDA\* ഓരോ റൗണ്ടിലും "ഇത്ര ചെലവ് വരെ മാത്രം നോക്കാം" എന്ന ഒരു പരിധി വെക്കുന്നു. കിട്ടിയില്ലെങ്കിൽ, പരിധി കടന്ന  $f$  values-ൽ ഏറ്റവും ചെറുത് പുതിയ പരിധിയാക്കി വീണ്ടും തുടങ്ങുന്നു. ഒരു വഴി മാത്രം memory-യിൽ വെക്കുന്നതുകൊണ്ട് A\*-നെക്കാൾ വളരെ കുറച്ച് memory മതി.

$\text{bound}_0 = h(\text{start})$

DFS: if  $f(n) = g(n) + h(n) > \text{bound}$ , return  $f(n)$  (cut off)

$\text{bound}_{\text{next}} = \min \{ f(n) : f(n) > \text{bound in this round} \}$

### EXAMPLE: IDA\* ON THE SAMPLE GRAPH

```
graph = {'A': [('B', 4), ('C', 3)], 'B': [('D', 2), ('E', 5)],
        'C': [('F', 1)], 'D': [], 'E': [('F', 1)], 'F': []}
h = {'A': 4, 'B': 2, 'C': 1, 'D': 5, 'E': 1, 'F': 0}

def ida_star(start, goal):
    def dfs(node, g, bound, path):
        f = g + h[node]
        if f > bound:
            return f, None          # cut off, report f
        if node == goal:
            return "FOUND", path
        min_over = float('inf')
        for nb, cost in graph[node]:
            if nb not in path:      # avoid cycles
                t, p = dfs(nb, g + cost, bound, path + [nb])
                if t == "FOUND":
                    return t, p
            min_over = min(min_over, t)
        return min_over, None

    bound = h[start]
    while True:
        print("Trying bound =", bound)
        t, path = dfs(start, 0, bound, [start])
        if t == "FOUND":
            return path
        if t == float('inf'):
            return None             # no solution
        bound = t                  # smallest f above old bound

print("IDA* Path from A to F:", ida_star('A', 'F'))
```

```
Trying bound = 4
IDA* Path from A to F: ['A', 'C', 'F']
```

Round 1, bound 4: A has  $f = 0 + 4 = 4$  (OK). B:  $f = 4 + 2 = 6 > 4$ , cut off. C:  $f = 3 + 1 = 4$  (OK). F:  $f = 4 + 0 = 4$  (OK) and it is the goal. Found in the first round. The unit's version returns True as the "found" signal; a string is used here so it can never be confused with a number.

ആദ്യ റൗണ്ടിൽ പരിധി 4. B-യുടെ  $f = 6$ , പരിധി കടന്നു, വെട്ടി. C വഴി F-ന്റെ  $f = 4$ , പരിധിക്കുള്ളിൽ, goal കിട്ടി. ഒറ്റ റൗണ്ട് മതിയായി.

### EXAMPLE: WHEN THE BOUND MUST GROW

Change the edge  $C \rightarrow F$  to cost 5 (everything else the same). Trace IDA\*.

```
Trying bound = 4
Trying bound = 6
Trying bound = 8
IDA* Path from A to F: ['A', 'C', 'F']
```

Bound 4: B cut at  $f = 6$ ; C OK ( $f = 4$ ); F via C has  $f = 8 + 0 = 8$ , cut. Smallest cut value = 6. Bound 6: B OK (6); D:  $f = 6 + 5 = 11$ , cut; E:  $f = 9 + 1 = 10$ , cut; C OK; F via C: 8, cut. Smallest = 8. Bound 8: F via C has  $f = 8 \leq 8$ , goal. Path A, C, F costs 8, cheaper than A, B, E, F = 10, so IDA\* is still optimal.

**Exam tip:** IDA\* re-expands nodes in every round (extra time) but needs only linear memory; A\* is faster but keeps every generated node in memory. Both are optimal with an admissible heuristic.

## Model evaluation metrics

**Confusion matrix:** A table that counts a classifier's True Positives, False Negatives, False Positives and True Negatives.

**Confusion matrix:** ഒരു classifier-ന്റെ True Positives, False Negatives, False Positives, True Negatives എന്നിവ എണ്ണിക്കാണിക്കുന്ന table.

**Precision:**  $TP / (TP + FP)$ : of the cases predicted positive, the fraction that are actually positive.

**Precision:**  $TP / (TP + FP)$ : positive എന്ന് predict ചെയ്തവയിൽ യഥാർത്ഥത്തിൽ positive ആയവയുടെ അനുപാതം.

**Recall:**  $TP / (TP + FN)$ : of the actual positive cases, the fraction the model correctly finds.

**Recall:**  $TP / (TP + FN)$ : യഥാർത്ഥ positive cases-ൽ model ശരിയായി കണ്ടെത്തിയവയുടെ അനുപാതം.

After building a classifier we must check how accurate and reliable it is. For binary classification the **confusion matrix** counts True Positives (TP), False Negatives (FN), False Positives (FP) and True Negatives (TN). From it we get **accuracy** (overall correctness), **precision** ("when the model says yes, how often is it right?"), **recall** ("of all real positives, how many did we find?") and **F1-score** (harmonic mean of precision and recall). The **ROC curve** plots True Positive Rate (recall) against False Positive Rate at different thresholds; **AUC** is the area under it: 1.0 = perfect, 0.5 = random guessing.

#### മലയാളത്തിൽ

ഒരു COVID test ഉദാഹരണമായി എടുക്കാം. Precision: test "positive" എന്ന് പറഞ്ഞവരിൽ എത്ര പേർക്ക് ശരിക്കും രോഗമുണ്ട്. Recall: ശരിക്കും രോഗമുള്ളവരിൽ എത്ര പേരെ test കണ്ടുപിടിച്ചു. Recall കുറഞ്ഞാൽ രോഗികളെ വിട്ടുപോകും, precision കുറഞ്ഞാൽ ആരോഗ്യമുള്ളവരെ വെറുതെ പേടിപ്പിക്കും. F1 രണ്ടിനെയും സന്തുലിതമാക്കുന്നു.

|                 | Predicted Positive | Predicted Negative |
|-----------------|--------------------|--------------------|
| Actual Positive | TP                 | FN                 |
| Actual Negative | FP                 | TN                 |

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

$$\text{Precision} = TP / (TP + FP)$$

$$\text{Recall} = TPR = TP / (TP + FN)$$

$$F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

$$FPR = FP / (FP + TN)$$



### EXAMPLE: ALL METRICS WITH SKLEARN

```
from sklearn.metrics import (confusion_matrix, accuracy_score, precision_score,
                             recall_score, f1_score, roc_auc_score)

y_true = [1, 1, 1, 1, 0, 0, 0, 0, 0, 0]
y_pred = [1, 1, 1, 0, 1, 0, 0, 0, 0, 0]
y_prob = [0.9, 0.8, 0.7, 0.4, 0.6, 0.3, 0.2, 0.2, 0.1, 0.1]
print(confusion_matrix(y_true, y_pred))
tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
print("TP, FN, FP, TN =", tp, fn, fp, tn)
print("Accuracy :", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall   :", recall_score(y_true, y_pred))
print("F1-score :", f1_score(y_true, y_pred))
print("ROC-AUC  :", roc_auc_score(y_true, y_prob))
```

```
[[5 1]
 [1 3]]
TP, FN, FP, TN = 3 1 1 5
Accuracy : 0.8
Precision: 0.75
Recall   : 0.75
F1-score : 0.75
ROC-AUC  : 0.9583333333333334
```

Hand check: 4 actual positives, 3 caught (TP = 3, FN = 1); 6 actual negatives, 1 wrongly flagged (FP = 1, TN = 5). Accuracy =  $(3 + 5)/10 = 0.8$ . Precision =  $3/(3 + 1) = 0.75$ . Recall =  $3/(3 + 1) = 0.75$ .  $F1 = 2 \times 0.75 \times 0.75 / 1.5 = 0.75$ . AUC uses the probabilities: of the  $4 \times 6 = 24$  (positive, negative) pairs, the positive gets the higher score in 23 (only  $0.4 < 0.6$  fails), so  $AUC = 23/24 = 0.958$ .

4 രോഗികളിൽ 3 പേരെ കണ്ടുപിടിച്ചു (recall 0.75), positive എന്ന് പറഞ്ഞ 4 പേരിൽ 3 പേർ ശരി (precision 0.75). AUC-ന് probabilities വേണം, 0/1 prediction പോരാ. ഒരു രോഗിക്കും ഒരു ആരോഗ്യവാനും ഇടയിൽ 24 ജോഡികളിൽ 23-ലും രോഗിക്ക് ഉയർന്ന score കിട്ടി.

**Exam tip:** sklearn's `confusion_matrix` sorts labels 0, 1, so it prints `[[TN, FP], [FN, TP]]`: the negative class comes first, the reverse of the unit's table. Read it carefully or use `.ravel()` as above. Accuracy is misleading on imbalanced data (99% "no fraud" gives 99% accuracy with zero recall); then quote precision, recall and F1.

# Hyperparameter tuning

**Hyperparameter:** A setting of a model chosen before training and not learned from the data, such as max\_depth or the learning rate.

**Hyperparameter:** Training-ന് മുമ്പ് തിരഞ്ഞെടുക്കുന്ന, data-യിൽ നിന്ന് പഠിക്കാത്ത model setting; ഉദാ: max\_depth, learning rate.

**Hyperparameter tuning:** Searching for the hyperparameter values that give the best performance, e.g. by grid search or random search scored with cross-validation.

**Hyperparameter tuning:** ഏറ്റവും നല്ല performance നൽകുന്ന hyperparameter values തിരയൽ; ഉദാ: cross-validation വഴി score ചെയ്യുന്ന grid search അല്ലെങ്കിൽ random search.

**Hyperparameters** are settings chosen before training, not learned from data: the number of trees in a Random Forest, the learning rate in Gradient Boosting/XGBoost, the kernel in an SVM, max\_depth of a decision tree. Good choices can improve performance a lot. Methods: **Grid Search** tries every combination (GridSearchCV); **Random Search** tries a fixed number of random combinations, which is faster (RandomizedSearchCV); **Bayesian optimisation** (e.g. Optuna) uses earlier results to choose the next trial. All of these score each setting by **k-fold cross-validation**.

## മലയാളത്തിൽ

Parameters (weights) model സ്വയം പഠിക്കുന്നു, hyperparameters നമ്മൾ മുൻകൂട്ടി തീരുമാനിക്കുന്നു, ഒരു കറിക്ക് എത്ര നേരം വേവിക്കണം, എത്ര തീ വേണം എന്ന പോലെ. Grid search എല്ലാ combinations-ഉം പരീക്ഷിക്കും, random search കുറച്ചെണ്ണം മാത്രം ക്രമമില്ലാതെ, അതുകൊണ്ട് വേഗം കൂടും.

Grid size = product of the number of values per hyperparameter

k-fold CV: split data into k parts; train on k - 1, test on 1; repeat k times; average the scores

### EXAMPLE: GRIDSEARCHCV ON THE PASS/FAIL DATA

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

df = pd.DataFrame({'Age': [18, 19, 20, 21, 22, 23],
                   'Score': [55, 60, 65, 70, 75, 80],
                   'Passed': [0, 0, 0, 1, 1, 1]})
X, y = df[['Age', 'Score']], df['Passed']
params = {'max_depth': [2, 3, 4]}
grid = GridSearchCV(DecisionTreeClassifier(random_state=0), params, cv=3)
grid.fit(X, y)
print("Best Parameters:", grid.best_params_)
print("Best CV accuracy:", grid.best_score_)
```

```
Best Parameters: {'max_depth': 2}
Best CV accuracy: 0.8333333333333334
```

GridSearchCV trains 3 depths  $\times$  3 folds = 9 trees. With 3-fold stratified CV each fold tests one fail and one pass row. Fold 1 tests rows 0 and 3: the tree trained on the other rows puts its boundary at age 21 (between 20 and 22), so row 3 (age 21) is misclassified, accuracy 0.5. Folds 2 and 3 score 1.0. Mean =  $2.5/3 = 0.833$ . One split already separates the data, so all depths score the same and the first one, 2, is reported.

മൂന്ന് depths  $\times$  മൂന്ന് folds = 9 തവണ train ചെയ്തു. ഒരു fold-ൽ 21 വയസ്സുകാരനെ fail എന്ന് തെറ്റി, ബാക്കി രണ്ട് fold-ഉം ശരി, ശരാശരി 0.833. എല്ലാ depths-നും ഒരേ score ആയതുകൊണ്ട് ആദ്യത്തേത് (2) best ആയി കാണിച്ചു.

### EXAMPLE: RANDOMIZEDSEARCHCV ON A RANDOM FOREST

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV

X, y = load_iris(return_X_y=True)
params = {'n_estimators': [10, 50, 100], 'max_depth': [2, 3, 4, None]}
search = RandomizedSearchCV(RandomForestClassifier(random_state=0), params,
                             n_iter=5, cv=3, random_state=0)

search.fit(X, y)
print("Best Parameters:", search.best_params_)
print("Best CV accuracy:", round(search.best_score_, 3))
```

```
Best Parameters: {'n_estimators': 10, 'max_depth': 4}
Best CV accuracy: 0.967
```

The full grid has  $3 \times 4 = 12$  combinations; `n_iter=5` tries only 5 of them at random, each with 3-fold CV (15 fits instead of 36). The Iris flower dataset (150 samples, 3 classes) is built into sklearn. The best of the 5 sampled settings reached 96.7% cross-validated accuracy.

12 combinations-ൽ 5 എണ്ണം മാത്രം random ആയി എടുത്ത് പരീക്ഷിച്ചു, സമയം ലാഭം. Grid search എല്ലാം നോക്കും, അതുകൊണ്ട് ഏറ്റവും നല്ലത് ഉറപ്പ്, പക്ഷേ hyperparameters കൂടുമ്പോൾ വളരെ പതുക്കെയാകും.

**Exam tip:** In the unit, the GridSearchCV snippet comes right after the K-Means snippet, which overwrote `X` with six 2-D points, so `grid.fit(X, y)` would tune the tree on the wrong features. Always define `X` and `y` right before fitting, as above.

## Quick recap

- Python suits AI: easy syntax, rich libraries, large community, platform independence.
- NumPy: fast ndarray; shape, ndim, dtype, size; slicing excludes the stop index; vectorised maths; axis=0 columns, axis=1 rows; broadcasting;  $z = (x - \text{mean})/\text{std}$ .
- $x * y$  is element-wise; `np.dot(A, B)` / `A @ B` is matrix multiplication.
- Pandas: Series (1-D labelled) and DataFrame (table); `iloc` by position, `loc` by label; `dropna`, `fillna`, `drop_duplicates`, `astype`.
- concat stacks rows; merge joins on a common column; NumPy std divides by `n`, Pandas std by `n - 1`.
- Matplotlib: plot, bar, hist, scatter, pie, subplots; call `plt.show()` with brackets; `savefig` before show.
- Scikit-learn: split → fit → predict → evaluate; `X` must be 2-D; use `random_state` for repeatable results.
- TensorFlow (Google, 2015): tensors, computation graph, eager execution since 2.0, Keras Sequential + Dense; Dense params = `in × out + out`.

- PyTorch (FAIR, 2016): dynamic graph, GPU tensors, autograd (`backward()`, `.grad`), `torch.nn`.
- N-Queen: one queen per row, `is_safe` checks column and two upper diagonals, backtrack by resetting the square;  $N = 4$  has 2 solutions.
- Minimax: X maximises, O minimises, terminal scores  $\pm 10$  adjusted by depth; alpha-beta prunes when  $\beta \leq \alpha$  and gives the same answer faster.
- BFS uses a queue (A B C D E F), DFS uses recursion/stack (A B D E F C); Best First expands the smallest  $h$ ;  $A^*$  the smallest  $f = g + h$  and is optimal with an admissible  $h$ .
- AO\*: OR takes the minimum, AND takes the sum; the result is a solution graph (A1 via A3 AND A4, cost 5). IDA\*: DFS with an  $f$ -bound that grows to the smallest exceeded  $f$ .
- Accuracy, precision, recall, F1 from the confusion matrix; AUC 1 = perfect, 0.5 = random; tune hyperparameters with Grid Search, Random Search or Bayesian optimisation using cross-validation.

## Check yourself

**Q. 1. Write a Python program using Matplotlib to draw a pie chart of your marks in 5 subjects.**

```
import matplotlib.pyplot as plt

subjects = ['Maths', 'AI', 'DBMS', 'Python', 'English']
marks = [78, 85, 62, 90, 71]
plt.figure()
plt.pie(marks, labels=subjects, autopct='%1.1f%%', startangle=90)
plt.title("My Marks")
plt.show()
```

`plt.pie` draws each slice in proportion to its value; `autopct='%1.1f%%'` writes the percentage with one decimal (the doubled `%%` prints a literal `%` sign); `startangle=90` starts the first slice at the top. The total is 386, so the slices show Maths 20.2%, AI 22.0%, DBMS 16.1%, Python 23.3%, English 18.4%.

**Q. 2. Write Python code for the average, minimum, maximum and standard deviation of the marks in Q1.**

```
import numpy as np

arr = np.array([78, 85, 62, 90, 71])
print("Total:", np.sum(arr))
print("Average:", np.mean(arr))
print("Min:", np.min(arr))
print("Max:", np.max(arr))
print("Std:", round(np.std(arr), 2))
```

```
Total: 386
Average: 77.2
Min: 62
Max: 90
Std: 9.95
```

Mean =  $386/5 = 77.2$ . The unit's model answer uses `np` without `import numpy as np`; include the import. `np.std` is the population standard deviation; `pd.Series(arr).std()` would give the sample value.

**Q. 3. What are tensors and how do they help in deep learning programs?**

Tensors are multidimensional arrays: a scalar is 0-D, a vector 1-D, a matrix 2-D, an image 3-D. They help because (i) all inputs (images, text, audio) and all weights are stored in one uniform numeric form; (ii) frameworks record operations on tensors and compute gradients automatically for backpropagation (autograd in PyTorch, GradientTape in TensorFlow); (iii) tensor operations run in parallel on GPUs, making training much faster.

**Q. 4. In the N-Queen program, what does `is_safe()` do, and what happens when a position is not safe?**

`is_safe(board, row, col)` returns False if any queen is already in the same column above, or on the upper-left or upper-right diagonal; otherwise True. Rows need no check because one queen is placed per row. If a position is not safe, the loop simply tries the next column. If no column in the row is safe, the function returns without placing a queen, control goes back to the previous row, that queen is removed (`board[row][col] = 0`) and moved to its next column: this is backtracking.

**Q. 5. In the Minimax implementation, what is the base case of the recursion? What value is returned when X wins at depth 3?**

The base case is a terminal state: X has won (evaluate = +10), O has won (-10), or no empty squares remain (draw, 0). The function returns the score instead of recursing. For an X win at depth 3 it returns  $10 - 3 = 7$ , so a win at depth 1 (value 9) is preferred over a later one.

**Q. 6. For the tree Max  $\rightarrow$  {Min1: leaves 3, 5; Min2: leaves 2, 9}, find the minimax value and the leaf that alpha-beta prunes.**

Min1 =  $\min(3, 5) = 3$ , so at the root  $\alpha = 3$ . At Min2 the first leaf is 2, so  $\beta = 2$ . Now  $\beta = 2 \leq \alpha = 3$ , so leaf 9 is pruned: Min2 will be at most 2, and Max already has 3. Root value =  $\max(3, 2) = 3$ , the same as plain minimax ( $\max(3, \min(2, 9)) = 3$ ).

**Q. 7. In DFS, what happens when the algorithm reaches a node with no unvisited neighbours? Give the DFS and BFS orders for the sample graph.**

The recursive call for that node ends and control returns (backtracks) to the previous node, which tries its next unvisited neighbour. On the sample graph DFS gives A, B, D, E, F, C (D is a dead end, so it backtracks to B and goes to E) and BFS gives A, B, C, D, E, F.

**Q. 8. For the 8-puzzle,  $h1$  = number of misplaced tiles and  $h2$  = sum of Manhattan distances. Which expands fewer nodes with A\*, and why?**

$h2$ . Both are admissible (every misplaced tile needs at least one move, and at least its Manhattan distance in moves). Every misplaced tile contributes 1 to  $h1$  but at least 1 to  $h2$ , so  $h2(n) \geq h1(n)$  for every state:  $h2$  dominates  $h1$ . A larger admissible heuristic is closer to the true cost, so fewer nodes have  $f$  below the optimal cost and A\* expands fewer nodes. Example: a tile two squares from its place adds 1 to  $h1$  but 2 to  $h2$ .

**Q. 9. An agent must find the shortest route in a city map (intersections = nodes, roads with distances = edges). Formulate it as state-space search and explain why A\* is preferred over BFS.**

State: the intersection where the agent is. Initial state: start intersection S. Goal state: destination G. Actions: move along a road to an adjacent intersection. Transition model: the action moves the agent to that neighbour. Path cost: sum of road lengths. Objective: a path from S to G with minimum total distance. BFS finds the path with the fewest roads, not the shortest distance, and searches blindly in all directions. A\* orders nodes by  $f = g + h$ , where  $g$  is the distance travelled and  $h$  the straight-line distance to G. Straight-line distance never overestimates road distance, so  $h$  is admissible and A\* returns the shortest route while expanding far fewer intersections.

**Q. 10.** A spam filter's confusion matrix has TP = 40, FN = 10, FP = 5, TN = 45. Compute accuracy, precision, recall and F1.

Total = 100. Accuracy =  $(40 + 45)/100 = 0.85$ . Precision =  $40/(40 + 5) = 40/45 = 0.889$ . Recall =  $40/(40 + 10) = 0.80$ . F1 =  $2 \times 0.889 \times 0.80 / (0.889 + 0.80) = 1.4222/1.6889 = 0.842$ . So 89% of mails marked spam really are spam, and the filter catches 80% of all spam.



## UNIT 12

# Applications of AI

ഇതുവരെ പഠിച്ച AI techniques യഥാർത്ഥ ലോകത്ത് എവിടെയൊക്കെ ഉപയോഗിക്കുന്നു എന്നതാണ് ഈ unit: healthcare, finance, transportation, customer service, education. ഓരോ മേഖലയിലും AI എന്ത് ചെയ്യുന്നു, ഏത് techniques ഉപയോഗിക്കുന്നു, ഒരു case study, പിന്നെ ഗുണങ്ങളും വെല്ലുവിളികളും നമ്മൾ കാണുന്നു.

Why AI is needed for real-world problems

AI in healthcare: overview

AI in diagnosis

AI in treatment

AI in medical imaging

Case study: IBM Watson for Oncology

Challenges and ethical concerns in healthcare AI

AI in finance and fraud detection

AI in algorithmic trading

AI in risk assessment

Case study: JPMorgan Chase COiN

AI in transportation: autonomous vehicles

AI in traffic optimization

Case study: Tesla Autopilot and FSD

AI in customer service and chatbots

Case study: Amazon Alexa

AI in education: personalized learning

Intelligent Tutoring Systems (ITS)

Case study: Duolingo

Comparing the five domains; common challenges

## Why AI is needed for real-world problems

**Definition:** AI provides intelligent systems that learn from large amounts of data and make timely decisions in dynamic environments, where fixed rules and manual processing fail.

Fixed rules-ഉം manual processing-ഉം പരാജയപ്പെടുന്ന dynamic environments-ൽ, വലിയ അളവിലുള്ള data-യിൽ നിന്ന് പഠിച്ച് സമയബന്ധിതമായി തീരുമാനങ്ങൾ എടുക്കുന്ന intelligent systems ആണ് AI നൽകുന്നത്.

Organizations in every domain now generate huge amounts of data. Traditional computational approaches (fixed rules, manual processing) struggle to **extract meaningful insights, make timely decisions and find complex patterns** in such large-scale data. So we need intelligent systems that **learn from data and make decisions in dynamic environments**. The unit lists the pressure points in each domain.

#### മലയാളത്തിൽ

ഇന്ന് ആശുപത്രി, ബാങ്ക്, റോഡ്, സ്കൂൾ എല്ലായിടത്തും data കണ്ടുകൂടുകയാണ്. പഴയ rule-based രീതികൾ കൊണ്ട് ഈ data-യിൽ നിന്ന് വേഗത്തിൽ തീരുമാനം എടുക്കാൻ പറ്റില്ല. അതുകൊണ്ടാണ് data-യിൽ നിന്ന് പഠിച്ച്, മാറ്റുന്ന സാഹചര്യത്തിൽ സ്വയം തീരുമാനം എടുക്കുന്ന AI systems വേണ്ടിവരുന്നത്.

| Domain                    | Problem the unit identifies                                                                                                      | What AI brings                                             |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| Healthcare                | Complex data (images, EHRs, histories); slow, variable manual processing; demand for personalized treatment and early prediction | Faster, more accurate diagnosis; personalized treatment    |
| Education                 | Diverse learning needs; one-size-fits-all teaching; teachers burdened by repetitive assessment                                   | Adaptive learning, automated grading, analytics            |
| Finance                   | Fraud, market volatility, credit default; need for real-time support                                                             | Fraud detection, risk models, algo-trading, chatbots       |
| Transportation & industry | Congestion, delays, equipment failure, downtime, defects                                                                         | Self-driving, traffic optimization, predictive maintenance |
| Agriculture               | Climate variability, scarce resources, pests, need for sustainable yield                                                         | Precision farming, pest/disease detection, forecasts       |
| Others                    | Cybersecurity, environmental monitoring, entertainment, space exploration                                                        | Reasoning under uncertainty, adapting over time            |

The unit then studies five domains in detail: **healthcare, finance, transportation, customer service, education**. Agriculture, manufacturing and the others are only mentioned in the introduction.

### EXAMPLE

Why can a fixed rule not keep up in these domains? Take a bank rule: "alert if a transaction is above ₹50,000."

Fraudsters learn the rule and split the theft into many ₹45,000 transactions, so the rule never fires. It also raises false alarms on every genuine large payment, such as school fees. An ML model looks at many features together (amount, place, device, time, usual behaviour) and is retrained on new fraud cases, so it adapts as the pattern changes. That is the "learn from data, act in a dynamic environment" idea.

**Exam tip:** A common short question is "Why is AI important for solving real-world problems?" Answer with three points (huge data, need for timely decisions, complex and changing patterns), then give one line per domain.

## AI in healthcare: overview

**Definition:** AI in healthcare is the use of machine learning, deep learning, NLP and computer vision as a decision-support tool for diagnosis, treatment planning, drug discovery and medical imaging.

Diagnosis, treatment planning, drug discovery, medical imaging എന്നിവയ്ക്കുള്ള decision-support tool ആയി machine learning, deep learning, NLP, computer vision എന്നിവ ഉപയോഗിക്കുന്നതാണ് healthcare-ലെ AI.

AI has become an important **decision-support tool** in healthcare, making it faster, more accurate and more accessible. AI systems can analyse massive amounts of data in seconds and spot patterns that even trained experts may miss. The main application areas are **diagnosis, treatment planning, drug discovery and medical imaging**. Healthcare worldwide faces a shortage of doctors, rising patient demand and rising costs. AI acts as a **digital assistant** that speeds up doctors' work and reduces human error.

### മലയാളത്തിൽ

ഒരു സർക്കാർ ആശുപത്രി OP-യിൽ ഒരു ഡോക്ടർ നൂറുകണക്കിന് രോഗികളെ കാണേണ്ടിവരുന്നു. അവിടെ AI ഡോക്ടർക്ക് പകരമല്ല, ഒരു digital assistant ആണ്. Scan നോക്കി സംശയമുള്ള ഭാഗം കാണിച്ചുകൊടുക്കും, പഴയ records വായിച്ച് സംഗ്രഹിക്കും. അവസാന തീരുമാനം ഡോക്ടറുടേതുതന്നെ.

Healthcare AI = decision-support tool, not a replacement for doctors

Four areas: diagnosis · treatment planning · drug discovery · medical imaging

Main techniques: machine learning, deep learning / computer vision, NLP

| Area            | Technique                             | Typical input → output                                |
|-----------------|---------------------------------------|-------------------------------------------------------|
| Diagnosis       | ML classifiers, NLP                   | EHRs, lab results, symptoms → probable disease        |
| Treatment       | ML on genetic data, surgical robotics | Genetic profile → most effective drug                 |
| Drug discovery  | Deep learning                         | Amino-acid sequence → predicted 3D protein structure  |
| Medical imaging | Computer vision (CNNs)                | X-ray / CT / MRI / retinal image → marked abnormality |

### EXAMPLE

The unit quotes an Accenture estimate that AI could save the U.S. healthcare system \$150 billion a year by 2026. What does it mean?

It is a projection of potential yearly savings from applications such as robot-assisted surgery, virtual nursing assistants and administrative automation. It is an estimate, not a measured result. For the exam, remember the direction of the claim: AI can cut cost as well as improve care.

**Exam tip:** The unit dates this Accenture report to 2022. The widely cited "\$150 billion annual savings by 2026" figure comes from Accenture's 2017 report "Artificial Intelligence: Healthcare's New Nervous System". If you quote it, write 2017.

## AI in diagnosis

**Definition:** AI-based diagnosis uses ML models trained on past cases to analyse EHRs, lab results, genetic data and symptoms and detect diseases early and accurately.

മുൻകാല cases-ൽ train ചെയ്ത ML models ഉപയോഗിച്ച് EHRs, lab results, genetic data, symptoms എന്നിവ analyse ചെയ്ത് രോഗങ്ങൾ നേരത്തെയും കൃത്യമായും കണ്ടെത്തുന്നതാണ് AI-based diagnosis.

An early and accurate diagnosis often decides whether treatment succeeds. AI helps by analysing **electronic health records (EHRs), lab test results, genetic data and patient-reported symptoms**. ML models trained on thousands of past cases learn to detect signs of cancer, diabetes, Alzheimer's, heart disease and rare disorders. The unit's examples are IBM Watson Health, which compares a patient's symptoms with medical literature, and symptom-checker chatbots such as Babylon Health.

### മലയാളത്തിൽ

പരിചയസമ്പന്നനായ ഒരു ഡോക്ടർ ആയിരക്കണക്കിന് രോഗികളെ കണ്ടാണ് രോഗം തിരിച്ചറിയാൻ പഠിക്കുന്നത്. ML model-ഉം അങ്ങനെതന്നെ: ആയിരക്കണക്കിന് പഴയ cases (labelled data) കണ്ട് pattern പഠിക്കുന്നു, പിന്നെ പുതിയ രോഗിയുടെ data കണ്ട് "ഈ രോഗം ആകാൻ സാധ്യത" എന്ന് പറയുന്നു.

| Specialty     | What AI analyses                                   | What it detects                                                                      |
|---------------|----------------------------------------------------|--------------------------------------------------------------------------------------|
| Oncology      | Mammograms, lung CT, skin images, pathology slides | Cancer at an early stage; LYNA detects metastatic breast cancer in lymph-node slides |
| Cardiology    | ECG and echocardiogram signals                     | Heart-rhythm and other heart problems                                                |
| Neurology     | Brain scans, behaviour data                        | Early signs of Alzheimer's or Parkinson's                                            |
| Mental health | Questionnaires, text, facial expressions, voice    | Assessment of mental-health condition                                                |

### EXAMPLE

An AI screening tool checks 1000 chest CT scans. 50 patients really have lung nodules. The AI flags 76 scans; 46 of them are real nodules and 30 are false alarms. Find sensitivity, specificity, precision and accuracy.

$TP = 46, FN = 50 - 46 = 4, FP = 30, TN = 950 - 30 = 920.$

$Sensitivity (recall) = TP / (TP + FN) = 46 / 50 = 92\%.$

$Specificity = TN / (TN + FP) = 920 / 950 \approx 96.8\%.$

$Precision = TP / (TP + FP) = 46 / 76 \approx 60.5\%.$

$Accuracy = (TP + TN) / 1000 = 966 / 1000 = 96.6\%.$

Lesson: "96.6% accurate" sounds excellent, yet 4 cancer patients are missed and 4 in 10 alarms are false. In medicine, always ask which metric a percentage refers to.

### EXAMPLE

How does a symptom-checker chatbot work?

The patient types "fever for 3 days, joint pain, rash". NLP extracts the symptoms, and a model trained on past cases ranks possible conditions (in Kerala's monsoon season these might include dengue and viral fever). It advises whether to see a doctor and how urgently. It gives a pre-visit suggestion, not a diagnosis.

**Exam tip:** The unit says LYNA has "accuracy of over 92%". Google's 2018 LYNA study reported an area under the ROC curve (AUC) of about 99% for classifying slides as cancer or not. The earlier 2017 Google study reported about 92% tumour-detection sensitivity. Either way, write "very high accuracy, better than or comparable to pathologists in studies". Also note that Babylon Health went into administration in 2023, so treat it as a past example.

## AI in treatment

**Precision (personalized) treatment:** Tailoring treatment to the individual patient, e.g. choosing a drug from the patient's genetic profile.

**Precision (personalized) treatment:** ചികിത്സ ഓരോ രോഗിക്കും അനുസരിച്ച് ക്രമീകരിക്കൽ; ഉദാ: രോഗിയുടെ genetic profile നോക്കി മരുന്ന് തിരഞ്ഞെടുക്കൽ.

**Robot-assisted surgery:** Surgery in which robots help surgeons perform minimally invasive procedures with high precision.

**Robot-assisted surgery:** Minimally invasive procedures ഉയർന്ന കൃത്യതയോടെ ചെയ്യാൻ robots surgeons-നെ സഹായിക്കുന്ന surgery.

**AI in drug discovery:** Using AI to predict how molecules interact with diseases, e.g. AlphaFold predicting protein structures.

**AI in drug discovery:** Molecules രോഗങ്ങളുമായി എങ്ങനെ പ്രതിപ്രവർത്തിക്കുന്നു എന്ന് AI ഉപയോഗിച്ച് പ്രവചിക്കൽ; ഉദാ: AlphaFold protein structures പ്രവചിക്കുന്നു.

Once the diagnosis is made, AI helps to plan and carry out treatment in four ways:

1. Precision (personalized) treatment: tailor treatment to the individual, not "one-size-fits-all"; e.g. choose a cancer drug from the patient's genetic profile
2. Robot-assisted surgery: robots help surgeons in minimally invasive surgery with high precision, analyse real-time data and alert on risk
3. Drug discovery: predict how molecules interact with diseases; e.g. DeepMind's AlphaFold predicts protein structures
4. Chronic-disease monitoring: wearables track diabetes, hypertension, heart rhythm and raise alerts (Apple Watch, Fitbit)

#### മലയാളത്തിൽ

തയ്യാൽക്കാരൻ ഓരോരുത്തരുടെയും അളവെടുത്ത് ചർട്ട് തയ്ക്കുന്നതുപോലെയാണ് precision treatment. എല്ലാവർക്കും ഒരേ മരുന്നല്ല, ഓരോ രോഗിയുടെയും genes നോക്കി ഏറ്റവും ഫലപ്രദമായ മരുന്ന്. Smartwatch കയ്യിൽ കെട്ടിയാൽ ഹൃദയമിടിപ്പ് സ്ഥിരമായി നോക്കി, അസാധാരണമായ rhythm കണ്ടാൽ alert തരും.

#### EXAMPLE

Why was AlphaFold important for drug discovery?

A drug works by fitting into a protein, like a key in a lock, so scientists need the protein's 3D shape. Finding that shape in a lab (for example by X-ray crystallography) can take months or years. AlphaFold, a deep-learning system from DeepMind (Alphabet), predicts the shape from the protein's amino-acid sequence, often with accuracy close to experiments. It stood out at the CASP14 competition in 2020. Demis Hassabis and John Jumper received half of the 2024 Nobel Prize in Chemistry for this work (the other half went to David Baker).

#### EXAMPLE

A wearable records a resting heart rate of about 70 beats/min for weeks. One night the beat-to-beat intervals become highly irregular for over an hour. What does the AI do?

The model has learned what a normal rhythm looks like for this user. Irregular intervals are an anomaly and may suggest atrial fibrillation. The watch sends an "irregular rhythm" notification and advises the user to consult a doctor, who confirms with a proper ECG. The watch screens; it does not diagnose.

**Exam tip:** For "role of AI in treatment", list all four points (personalized treatment, robotic surgery, drug discovery, chronic-disease monitoring), each with its example. In surgery the robot assists the surgeon; it does not operate on its own.

## AI in medical imaging

**Definition:** AI in medical imaging is the use of computer vision models, usually CNNs, to analyse X-rays, MRIs, CT scans and ultrasound images and detect abnormalities.

X-rays, MRIs, CT scans, ultrasound images എന്നിവ analyse ചെയ്ത് abnormalities കണ്ടെത്താൻ computer vision models (സാധാരണയായി CNNs) ഉപയോഗിക്കുന്നതാണ് medical imaging-ലെ AI.

Medical imaging is one of the most advanced and successful uses of AI in healthcare. **Computer vision** models, usually convolutional neural networks (CNNs), analyse X-rays, MRIs, CT scans and ultrasound images to find abnormalities.

മലയാളത്തിൽ

ഒരു radiologist ദിവസം നൂറുകണക്കിന് scans നോക്കണം; ക്ഷീണിക്കുമ്പോൾ ചെറിയ nodule കണ്ണിൽപ്പെടാതെ പോകാം. AI-ക്ക് ക്ഷീണമില്ല. സംശയമുള്ള ഭാഗം അത് highlight ചെയ്യും, radiologist അത് പരിശോധിച്ച് ഉറപ്പിക്കും.

| Application       | What AI does                                          | Example                                                |
|-------------------|-------------------------------------------------------|--------------------------------------------------------|
| Radiology support | Marks suspected areas in the scan                     | Lung nodules in CT (possible early lung cancer)        |
| Digital pathology | Scans and classifies tissue patterns in biopsy slides | Faster biopsy results                                  |
| Ophthalmology     | Reads retinal images / eye scans                      | Diabetic retinopathy, age-related macular degeneration |
| Dermatology       | Classifies photos of skin lesions                     | Melanoma risk (SkinVision app)                         |

EXAMPLE

An Indian example of eye-screening AI.

Diabetic retinopathy can cause blindness if missed, and India has many diabetics but few eye specialists. Google's retinal-image model was tested and used in India with partner hospitals such as Aravind Eye Hospital. A technician photographs the retina, the model grades the retinopathy, and only referable cases go to the ophthalmologist. This is how AI makes healthcare more accessible.

EXAMPLE

Outline the steps an imaging AI follows on a new chest X-ray.

1. Pre-process: resize and normalize the image. 2. A CNN extracts features such as edges, textures and shapes. 3. A classifier outputs probabilities (normal, pneumonia, nodule, ...). 4. A heat-map shows which region caused the prediction. 5. The radiologist reviews the flagged region and writes the report.



**Exam tip:** The unit credits diabetic retinopathy detection to "Google's DeepMind AI". To be precise, DeepMind with Moorfields Eye Hospital (2018) detected over 50 eye diseases, including diabetic eye disease and macular degeneration, from OCT eye scans. The retinopathy screening from fundus photographs used in India came from Google's health research team. "Google/DeepMind" is acceptable in an answer.

## Case study: IBM Watson for Oncology

**Definition:** A clinical decision support system is an AI system that analyses patient data against medical knowledge and gives doctors evidence-based recommendations, such as ranked treatment options.

രോഗിയുടെ data medical knowledge-മായി താരതമ്യം ചെയ്ത്, ranked treatment options പോലെയുള്ള evidence-based recommendations ഡോക്ടർമാർക്ക് നൽകുന്ന AI system ആണ് clinical decision support system.

IBM built Watson for Oncology together with **Memorial Sloan Kettering Cancer Center (MSKCC)** to help doctors diagnose cancer and recommend personalized treatment plans. It uses **NLP and machine learning** to read unstructured medical data (clinical notes, pathology reports, research papers) and compares that knowledge with the patient's history, test results and genetic data. It suggests treatment options ranked in three classes, "**recommended**", "**for consideration**" and "**not recommended**", each with supporting evidence. It was used in hospitals in the USA, India and South Korea; Manipal Hospitals in India used it to support cancer specialists.

### മലയാളത്തിൽ

ലോകത്ത് ഓരോ ദിവസവും പുതിയ cancer research papers ഇറങ്ങുന്നു; ഒരു ഡോക്ടർക്ക് എല്ലാം വായിക്കാൻ പറ്റില്ല. Watson NLP ഉപയോഗിച്ച് ലക്ഷക്കണക്കിന് പേജുകൾ "വായിച്ച്", രോഗിയുടെ report-മായി താരതമ്യം ചെയ്ത്, ചികിത്സാ options മൂന്ന് തട്ടുകളായി നിരത്തുന്നു. പക്ഷേ ഓരോ രാജ്യത്തെയും ചികിത്സാരീതിയുമായി പൊരുത്തപ്പെടാൻ അതിന് ബുദ്ധിമുട്ടുണ്ടായി.

Input: medical literature + clinical guidelines + patient records (history, tests, genetics)

Techniques: NLP (read unstructured text) + ML (match and rank)

Output: ranked options: recommended / for consideration / not recommended, with evidence

Benefit: less time reviewing cases, more precise treatment

Challenge: adapting to local treatment protocols and fast-changing medical knowledge

### EXAMPLE

Trace Watson's workflow for one patient with stage II breast cancer.

1. The oncologist enters the patient's attributes: age, tumour size, stage, receptor status, other illnesses.
2. NLP extracts facts from pathology reports and clinical notes.
3. The system matches these against guidelines and the literature it was trained on (largely MSKCC's practice).
4. It outputs, for example, surgery followed by a named chemotherapy regimen as "recommended", an alternative as "for consideration", and a regimen unsuitable because of a heart condition as "not recommended", each linked to studies.
5. The tumour board discusses and decides.

**Exam tip:** Useful background for a "critically evaluate" question: the system was trained largely on one U.S. hospital's practice, which is exactly the unit's "local protocols" problem. It drew public criticism around 2017–2018 for some unsafe or unhelpful suggestions, and IBM sold its Watson Health business in 2022 (it became Merative). So it is a lesson in both the promise and the limits of healthcare AI.

## Challenges and ethical concerns in healthcare AI

**Algorithmic bias:** Systematically wrong or unfair results for some groups because a model was trained on non-representative data.

**Algorithmic bias:** പ്രതിനിധ്യമില്ലാത്ത (non-representative) data-യിൽ model train ചെയ്തതിനാൽ ചില groups-ന് സ്ഥിരമായി തെറ്റായതോ അന്യായമായതോ ആയ ഫലങ്ങൾ ലഭിക്കും.

**Accountability:** Deciding who is responsible when an AI system makes a wrong decision, such as a wrong diagnosis.

**Accountability:** AI system ഒരു തെറ്റായ തീരുമാനം (ഉദാ: തെറ്റായ diagnosis) എടുക്കുമ്പോൾ ആരാണ് ഉത്തരവാദി എന്ന് തീരുമാനിക്കൽ.

The unit lists four concerns: (1) **privacy**: medical data is highly sensitive and misuse can harm patients; (2) **bias**: a model trained on non-representative data gives wrong results for under-represented groups; (3) **assist, not replace**: final decisions must stay with human experts; (4) **accountability**: who is responsible if the AI makes a wrong diagnosis? The answer key adds lack of quality data, high implementation cost and the need for expert supervision.

#### മലയാളത്തിൽ

വെളുത്ത നിറമുള്ളവരുടെ skin photos മാത്രം കണ്ട് പഠിച്ച model ഇരുണ്ട നിറമുള്ള ഒരു മലയാളിയുടെ skin lesion ശരിയായി തിരിച്ചറിയണമെന്നില്ല. ഇതാണ് dataset bias. AI തെറ്റായ diagnosis പറഞ്ഞാൽ ആർ ഉത്തരവാദി: ഡോക്ടറോ, company-യോ, ആശുപത്രിയോ? ഈ ചോദ്യത്തിന് ഇപ്പോഴും വ്യക്തമായ ഉത്തരമില്ല.

| Concern               | Why it matters                                                          | Remedy                                                                           |
|-----------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Privacy               | Leaked records can lead to discrimination and fraud                     | Consent, anonymization, encryption, data-protection law (India's DPDP Act, 2023) |
| Bias                  | Unequal accuracy across groups                                          | Diverse datasets, bias audits, validation on local populations                   |
| Over-reliance         | Doctors may stop questioning the AI                                     | Human-in-the-loop; AI only recommends                                            |
| Accountability        | Unclear liability for errors                                            | Regulation, explainable outputs, audit trails                                    |
| Data quality and cost | Poor labels mean poor models; small hospitals cannot afford the systems | Standardized EHRs, shared public infrastructure                                  |

#### EXAMPLE

A heart-risk model trained mostly on data from middle-aged men has 90% sensitivity for men but 70% for women. What goes wrong and how do you fix it?

Women's heart attacks often show different symptoms, and the model saw too few of them. So 30 of every 100 at-risk women are missed. That is unequal treatment, and it erodes trust in the system. Fix: collect more representative data (or re-weight it), report accuracy separately for each group, validate clinically before deployment, and keep a doctor reviewing the output.

**Exam tip:** For the bias question (Check Your Progress 1, Q3) give both halves: the effect (inaccurate diagnosis, unequal treatment, wider disparities, loss of trust) and the solutions (diverse data, continuous bias auditing, transparency, inclusive clinical validation, multidisciplinary teams).

## AI in finance and fraud detection

**AI fraud detection:** Using ML models that learn continuously from transaction data to detect suspicious, anomalous transactions in real time.

**AI fraud detection:** Transaction data-യിൽ നിന്ന് തുടർച്ചയായി പഠിക്കുന്ന ML models ഉപയോഗിച്ച് സംശയാസ്പദമായ, അസാധാരണമായ transactions real time-ൽ കണ്ടെത്തൽ.

**Anomaly detection:** Identifying data points that deviate significantly from normal patterns.

**Anomaly detection:** സാധാരണ patterns-ൽ നിന്ന് കാര്യമായി വ്യതിചലിക്കുന്ന data points തിരിച്ചറിയൽ.

Finance is one of the most data-driven sectors: banks, stock markets and insurers produce enormous data every second (big data). AI techniques (**ML, deep learning, NLP**) improve accuracy, efficiency and security: they prevent card fraud, execute trades in milliseconds and predict loan and investment risk. Traditional fraud detection uses **fixed rules** (e.g. "alert if amount > limit"), which fraudsters learn to evade. AI models **learn continuously from new transactions** and detect **anomalies in real time**.

### മലയാളത്തിൽ

നിങ്ങൾ സാധാരണ കൊച്ചിയിൽ ₹500-ന്റെ UPI payments മാത്രമാണ് ചെയ്യുന്നത്. പെട്ടെന്ന് പുലർച്ചെ 3 മണിക്ക് പുതിയൊരു phone-ൽ നിന്ന് ₹90,000 transfer വന്നാൽ അത് നിങ്ങളുടെ "സാധാരണ സ്വഭാവം" അല്ല. AI model ഈ anomaly ഉടനെ പിടിച്ച് transaction തടയുകയോ OTP ചോദിക്കുകയോ ചെയ്യും. Rules ഒരിക്കൽ എഴുതിയാൽ അതുപോലെ നിൽക്കും; ML model പുതിയ തട്ടിപ്പുകളിൽ നിന്നും പഠിക്കും.

|                 | Rule-based                             | AI / ML-based                                                        |
|-----------------|----------------------------------------|----------------------------------------------------------------------|
| Logic           | Hand-written if-then rules             | Patterns learned from labelled and unlabelled data                   |
| New fraud types | Missed until someone writes a new rule | Adapts through retraining; anomaly detection catches unseen patterns |
| False alarms    | Many (every large genuine payment)     | Fewer, because it uses context (device, location, history)           |
| Explainability  | Easy                                   | Harder; needs explainable-AI tools                                   |

Unit's examples:

1. Transaction monitoring: flag a card used in two countries within minutes
2. Login/behaviour analysis: access from an unusual device
3. Insurance: scan claims and medical records for false claims

#### EXAMPLE (IMPOSSIBLE TRAVEL)

A card is swiped in Kochi at 10:00 and in London at 10:10. Kochi to London is roughly 8,000 km. Should it be flagged?

Required speed =  $8000 \text{ km} \div (10/60) \text{ h} = 48,000 \text{ km/h}$ . A passenger jet flies at about 900 km/h, so one person cannot have made both swipes. Flag it: at least one transaction uses a cloned card.

#### EXAMPLE (ANOMALY SCORE)

A customer's card spends average ₹2,000 per transaction with standard deviation ₹500. A new transaction is ₹9,000. Compute the z-score.

$z = (9000 - 2000)/500 = 14$ . Values beyond about  $|z| = 3$  are rare for this customer, so the transaction is a strong anomaly. A real model combines many such features (amount, merchant type, time, device, location) instead of amount alone.

**Exam tip:** "How can AI ensure security in finance?" (Check Your Progress 1, Q2): fraud detection, real-time transaction monitoring for unusual patterns, detection of abnormal logins, and protection of customer data. An Indian example: in December 2024 the RBI announced MuleHunter.AI, an AI model to help banks detect "mule" accounts used to move fraud money.

## AI in algorithmic trading

**Definition:** Algorithmic trading is the use of computer programs to execute trades automatically based on pre-defined rules; AI adds price prediction and real-time strategy optimisation.

മുൻകൂട്ടി നിശ്ചയിച്ച rules അനുസരിച്ച് trades സ്വയമേവ നടത്താൻ computer programs ഉപയോഗിക്കുന്നതാണ് algorithmic trading; AI ഇതിലേക്ക് price prediction-ഉം real-time strategy optimisation-ഉം ചേർക്കുന്നു.

**Algorithmic trading** (algo-trading) is the use of computer programs to execute trades based on pre-defined rules. Human traders cannot analyse market data at very high speed, and a delay of milliseconds can turn a profit into a loss. AI adds learning: it predicts price movements, reads market sentiment and optimizes strategies in real time.

#### മലയാളത്തിൽ

മീൻ ചന്തയിൽ വില സെക്കൻഡുകൾക്കുള്ളിൽ മാറുന്നതുപോലെയാണ് share market. മനുഷ്യൻ വാർത്ത വായിച്ച് തീരുമാനിക്കുമ്പോഴേക്കും വില മാറിയിട്ടുണ്ടാകും. AI ഒരേസമയം വിലകൾ, news, tweets എല്ലാം വിശകലനം ചെയ്ത് ഒരു സെക്കൻഡിൽ ആയിരക്കണക്കിന് trades നടത്തും.

| Use                    | Technique                             | Data                                         |
|------------------------|---------------------------------------|----------------------------------------------|
| Price prediction       | ML / deep learning time-series models | Historical prices, news, economic indicators |
| Sentiment analysis     | NLP                                   | Financial news, tweets, company reports      |
| High-speed execution   | Automated decision programs           | Real-time market feed                        |
| Portfolio optimization | Optimization / ML                     | Risk and return of assets                    |

Real-world: JPMorgan's LOXM (executes trades with minimal market impact) · Goldman Sachs (AI platforms to analyse investment opportunities) · Renaissance Technologies (hedge fund built on quantitative, ML-driven models)

#### EXAMPLE (SENTIMENT SIGNAL)

An NLP model scores headlines from -1 (very negative) to +1 (very positive). Today's three headlines about Company X score +0.8 ("record quarterly profit"), +0.4 ("new plant approved") and -0.3 ("CFO resigns"). What is the signal?

Average =  $(0.8 + 0.4 - 0.3) / 3 = 0.9 / 3 = 0.3$ . That is mildly positive, so the strategy might tilt towards buying, but only if the price model agrees. Sentiment is one input among many.

#### EXAMPLE (BALANCING RISK AND RETURN)

An investor puts 60% in a stock fund expected to return 12% and 40% in a bond fund expected to return 6%. What is the expected portfolio return?

$0.6 \times 12\% + 0.4 \times 6\% = 7.2\% + 2.4\% = 9.6\%$ . An AI portfolio tool searches over many such weightings and picks the mix that gives the best return for the risk the investor accepts.

**Exam tip:** Define algo-trading exactly as the unit does ("computer programs to execute trades based on pre-defined rules"), then say what AI adds: learning patterns, prediction and real-time optimization. Risks worth mentioning: flash crashes from many algorithms reacting together, and models failing when the market behaves unlike the training data.

## AI in risk assessment

**Risk assessment:** Estimating the likelihood and size of possible loss before lending, insuring or investing.

**Risk assessment:** Loan, insurance, investment എന്നിവ നൽകുന്നതിന് മുമ്പ് നഷ്ടം ഉണ്ടാകാനുള്ള സാധ്യതയും അതിന്റെ വലുപ്പവും കണക്കാക്കൽ.

**Credit risk:** The risk that a borrower will not repay a loan.

**Credit risk:** കടം വാങ്ങിയയാൾ loan തിരിച്ചടയ്ക്കാതിരിക്കാനുള്ള risk.

Every loan, insurance policy or investment carries risk. Traditional assessment uses **credit scores and static rules**. AI finds hidden patterns and assesses risk more accurately in four areas: **credit risk** (will the person repay?), **market risk** (volatility and potential losses), **operational risk** (system failures, cyberattacks, staff misconduct) and **insurance risk** (e.g. driving behaviour from car telematics to set fair premiums).

### മലയാളത്തിൽ

ഗ്രാമത്തിലെ ഒരു ചെറിയ കച്ചവടക്കാരന് credit card-ഉം loan history-യും ഇല്ല, അതിനാൽ credit score-ഉം ഇല്ല. പക്ഷേ അയാൾ എല്ലാ മാസവും phone bill, വൈദ്യുതി bill കൃത്യമായി അടയ്ക്കുന്നു. ഇത്തരം alternative data നോക്കി AI അയാൾക്ക് loan കൊടുക്കാമോ എന്ന് വിലയിരുത്തും. നന്നായി ഓടിക്കുന്ന driver-ക്ക് കുറഞ്ഞ insurance premium, അതാണ് telematics.

Real-world: ZestFinance (now Zest AI): alternative data such as phone-bill payments and online behaviour for credit scoring

FICO: AI in its credit-scoring system to reduce defaults

Swiss Re: AI for underwriting and predicting claim risk

### EXAMPLE

Two applicants have the same credit score of 700. A: salary credited every month, bills paid on time, stable address for 5 years. B: irregular income, three new loan apps installed last month, frequent late bill payments. How does an AI model treat them?

A static rule ("score  $\geq 700 \rightarrow$  approve") treats them the same. An ML model trained on past repayments has learned that irregular income, many new loan apps and late payments go with default, so it gives B a higher default probability and may charge more or ask for collateral. It gives A a lower rate.

### EXAMPLE (TELEMATICS INSURANCE)

A telematics device records harsh braking, speeding and night driving. Driver P brakes harshly twice per 100 km; driver Q does so 15 times. Premium?

The model links harsh braking and speeding to claims, so P gets a discount and Q a higher premium. This is "pay-how-you-drive" insurance.

**Exam tip:** Mention the fairness problem too. Using "digital footprint and social behaviour" (the unit's words) can discriminate against people and invade privacy, and lenders must be able to explain a rejection. RBI's digital lending guidelines (2022) regulate app-based lending in India.

## Case study: JPMorgan Chase COiN

**Definition:** Contract intelligence is the use of ML and NLP to read legal and financial documents automatically and extract important clauses, risks and compliance requirements.

Legal, financial documents സ്വയമേവ വായിച്ച് പ്രധാന clauses, risks, compliance requirements എന്നിവ കണ്ടെത്താൻ ML-ഉം NLP-യും ഉപയോഗിക്കുന്നതാണ് contract intelligence.

**COiN (Contract Intelligence)** is JPMorgan Chase's platform for automatically reviewing legal and financial documents, especially **commercial loan agreements**. Using **ML and NLP** it identifies important clauses, risks and compliance requirements. It cut review work that took **360,000 hours** a year to a few seconds, with greater consistency and accuracy. JPMorgan also uses AI to monitor transactions in real time for fraud; these models keep learning new fraud patterns that rule-based systems miss.



#### മലയാളത്തിൽ

ഒരു loan agreement നൂറുകണക്കിന് പേജ്; അതിലെ പ്രധാന clauses കണ്ടെത്താൻ വക്കീലന്മാർ ആഴ്ചകൾ എടുക്കും, ക്ഷീണിക്കുമ്പോൾ തെറ്റും വരും. COiN NLP ഉപയോഗിച്ച് സെക്കൻഡുകൾക്കുള്ളിൽ ഇത് വായിച്ച് പ്രധാന ഭാഗങ്ങൾ വേർതിരിക്കും. ഇതോടെ ജീവനക്കാർക്ക് കൂടുതൽ ബുദ്ധി വേണ്ട ജോലികളിൽ ശ്രദ്ധിക്കാം.

Problem: manual contract review is slow, costly, error-prone

Techniques: NLP (read clauses) + ML (classify and extract)

Result: 360,000 hours/year → seconds; consistency and accuracy

Also: real-time AI fraud detection and transaction monitoring

#### EXAMPLE

Put 360,000 hours in perspective. Assume one employee works about 2,000 hours a year.

$360,000 \div 2,000 = 180$  full-time employees working for a whole year. COiN freed that much skilled time for higher-value work every year.

#### EXAMPLE

What does COiN "extract" from a clause such as "The Borrower shall maintain a debt-to-equity ratio not exceeding 2:1; breach constitutes an Event of Default"?

Clause type: financial covenant. Parameter: debt-to-equity  $\leq 2:1$ . Consequence: event of default. Risk flag: yes. Stored as structured data, this lets the bank search thousands of contracts at once ("which loans have covenants tighter than 2:1?").

**Exam tip:** Remember three facts: the name COiN = Contract Intelligence, the task (commercial loan agreements), and the number 360,000 hours → seconds. JPMorgan made COiN public in 2017.

## AI in transportation: autonomous vehicles

**Autonomous vehicle:** A vehicle that uses sensors, AI and control systems to perceive its environment, decide in real time and navigate safely with minimal or no driver input.

**Autonomous vehicle:** Sensors, AI, control systems എന്നിവ ഉപയോഗിച്ച് ചുറ്റുപാട് perceive ചെയ്ത്, real time-ൽ തീരുമാനമെടുത്ത്, driver-ന്റെ ഇടപെടൽ ഇല്ലാതെയോ വളരെ കുറച്ചോ സുരക്ഷിതമായി സഞ്ചരിക്കുന്ന vehicle.

**Sensor fusion:** Combining data from several sensors so that each covers the others' weaknesses and provides a backup if one fails.

**Sensor fusion:** ഓരോ sensor-ഉം മറ്റുള്ളവയുടെ പോരായ്മകൾ നികത്തുന്ന വിധത്തിലും ഒന്ന് പരാജയപ്പെട്ടാൽ backup ലഭിക്കുന്ന വിധത്തിലും പല sensors-ന്റെ data സംയോജിപ്പിക്കൽ.

Transportation moves people and goods but suffers from congestion, accidents, high fuel use and pollution. Using ML, computer vision, robotics and big data, AI improves vehicles, traffic flow and networks. The two major applications are **autonomous vehicles** and **traffic optimization**. An **autonomous vehicle (AV)** is an intelligent transport system that uses sensors, AI, ML and control systems to perceive its environment, decide in real time and navigate safely with minimal or no driver input under defined operating conditions.

### മലയാളത്തിൽ

ഒരു driver കണ്ണുകൊണ്ട് കാണുന്നു, തലച്ചോറുകൊണ്ട് തീരുമാനിക്കുന്നു, കൈകാലുകൾ കൊണ്ട് steering-ഉം brake-ഉം പ്രവർത്തിപ്പിക്കുന്നു. Self-driving car-ൽ camera, LiDAR, radar എന്നിവ കണ്ണുകൾ; AI model തലച്ചോർ; steering, throttle, brake എന്നിവയാണ് കൈകാലുകൾ. കേരളത്തിലെ ഇടുങ്ങിയ റോഡുകളിൽ ഓട്ടോയും പശുവും കാൽനടക്കാരും ഒരുമിച്ചുള്ളപ്പോൾ ഇതെല്ലാം ശരിയായി ചെയ്യുക വളരെ പ്രയാസമാണ്.

AV pipeline (the unit's five technologies):

Sensing (cameras, LiDAR, radar, ultrasonic) → Perception (AI identifies cars, pedestrians, signs, obstacles) → Localization (GPS + AI position on a map) → Decision/planning (ML decides brake, accelerate, change lane) → Control (steering, throttle, brakes)

| Sensor     | Strength                                  | Weakness                                         |
|------------|-------------------------------------------|--------------------------------------------------|
| Camera     | Colour, text, signs, lane markings; cheap | Poor in dark, glare, heavy rain; no direct depth |
| LiDAR      | Precise 3D distance map                   | Expensive; degraded by fog and heavy rain        |
| Radar      | Distance and speed; works in rain and fog | Low resolution; cannot read signs                |
| Ultrasonic | Very short range (parking)                | Only a few metres                                |

**Sensor fusion** combines these sensors so that each covers the others' weaknesses (complementary strengths) and a failed sensor has a backup (redundancy).

| SAE level | Name                   | Who drives / monitors                                                | Example                                    |
|-----------|------------------------|----------------------------------------------------------------------|--------------------------------------------|
| 0         | No automation          | Human does everything (warnings may exist)                           | Basic car with collision warning           |
| 1         | Driver assistance      | System steers or controls speed                                      | Adaptive cruise control                    |
| 2         | Partial automation     | System steers and controls speed; human must watch constantly        | Tesla Autopilot / FSD (Supervised)         |
| 3         | Conditional automation | System drives in limited conditions; human must take over when asked | Mercedes-Benz Drive Pilot (approved roads) |
| 4         | High automation        | No human needed within a defined area and conditions                 | Waymo robotaxis                            |
| 5         | Full automation        | Drives anywhere, any condition, no human                             | None commercially available                |

Examples: Waymo (Google's self-driving project, now an Alphabet company): driverless taxis in Phoenix and other U.S. cities · Tesla Autopilot: computer vision + neural networks, semi-autonomous · Nuro: autonomous delivery vehicles for groceries and small goods · Uber ATG: self-driving unit, sold to Aurora in 2020

Challenges: safety and liability in accidents · expensive LiDAR and computing · roads, signals and networks need upgrading · public trust

### EXAMPLE

Classify: (a) a car that keeps its lane and distance on the highway, but the driver must keep watching and hold the wheel; (b) a taxi with no one in the driver's seat operating within one city's mapped area; (c) a car with only adaptive cruise control.

(a) Level 2: steering and speed are both automated, but a human supervises continuously. (b) Level 4: no human needed, but only within a defined operating area. (c) Level 1: only one function (speed) is automated.

### EXAMPLE (SENSOR FUSION)

At night in heavy monsoon rain the camera cannot see a stalled lorry ahead, but the radar reports an object 40 m ahead with closing speed 15 m/s. What does fusion do?

The radar reading is trusted more in rain, so the object is kept even without camera confirmation. Time to collision =  $40 \div 15 \approx 2.7$  s, so the planner brakes. A camera-only system might have missed the lorry; this is the redundancy benefit.

**Exam tip:** The unit says Tesla Autopilot and GM Cruise are "around Level 2 or 3".  
Correction: Tesla's systems are Level 2 (the driver must supervise at all times). Cruise ran driverless Level 4 robotaxis in San Francisco until its permit was suspended after a 2023 accident, and GM ended the robotaxi programme in December 2024. A good Level 3 example is Mercedes-Benz Drive Pilot.

## AI in traffic optimization

**Traffic optimization:** Using AI to predict traffic flow, adapt signal timings and guide vehicles along the best routes to reduce congestion.

**Traffic optimization:** Congestion കുറയ്ക്കാൻ AI ഉപയോഗിച്ച് traffic flow പ്രവചിക്കുകയും signal timings ക്രമീകരിക്കുകയും vehicles-നെ മികച്ച routes-ലൂടെ നയിക്കുകയും ചെയ്യൽ.

**Smart traffic light:** A traffic signal whose timings adapt to real-time traffic flow instead of following fixed timers.

**Smart traffic light:** Fixed timers-ന് പകരം real-time traffic flow അനുസരിച്ച് timings മാറ്റുന്ന traffic signal.

Cities suffer congestion, long commutes, extra fuel use, pollution, greenhouse-gas emissions and economic losses. AI makes existing transport smarter by **predicting traffic flow, optimizing signal timings and guiding vehicles along the best routes**.

#### മലയാളത്തിൽ

ബൈപ്പാസിലെ ഒരു signal-ൽ ഒരു വശത്ത് റൂട്ട് വണ്ടികൾ, മറുവശത്ത് ആരുമില്ലാതിരിക്കുമ്പോഴും രണ്ടിനും ഒരേ 60 സെക്കൻഡ് green കിട്ടുന്നത് എത്ര അർത്ഥശൂന്യമാണ്. AI signal camera വഴി queue കണ്ട് ആവശ്യമുള്ള വശത്തിന് കൂടുതൽ സമയം നൽകും. Google Maps മുന്നിലെ block കാണിച്ച് വേറെ വഴി പറഞ്ഞുതരുന്നതും ഇതേ ആശയമാണ്.

| Application          | How AI helps                                              | Real example                                                            |
|----------------------|-----------------------------------------------------------|-------------------------------------------------------------------------|
| Smart traffic lights | Timings adapt to real-time flow instead of fixed timers   | Surtrac, Pittsburgh: about 25% less travel time, about 40% less waiting |
| Predictive analysis  | ML predicts jams from historical data, weather, accidents | Google Maps delay prediction                                            |
| Smart navigation     | GPS + AI suggest the fastest route                        | Google Maps, Waze                                                       |
| Connected vehicles   | Cars communicate for cooperative driving                  | Vehicle-to-vehicle (V2V) systems                                        |
| Public transport     | Bus scheduling, demand forecasting, route planning        | Shorter waiting times                                                   |
| City-wide monitoring | Sensors + analytics, emergency response                   | Singapore Smart Nation; smart cities in China                           |

#### EXAMPLE (ADAPTIVE SIGNAL)

A junction gives 80 s of green per cycle to two directions. Fixed timing gives 40 s each. Cameras count 30 waiting vehicles north-south and 10 east-west. Split green time in proportion to the queues.

Total = 40. North-south =  $30/40 \times 80 = 60$  s; east-west =  $10/40 \times 80 = 20$  s. The long queue clears faster, and the green is not wasted on an empty road. Surtrac goes further: each signal plans its own timing and shares its planned outflow with neighbouring signals.

#### EXAMPLE

A commute takes 20 minutes, with 50 seconds of waiting at each signal. Apply Surtrac's reported reductions.

Travel time:  $20 \times (1 - 0.25) = 15$  minutes. Wait per signal:  $50 \times (1 - 0.40) = 30$  seconds.

### EXAMPLE (KERALA)

How is AI used for traffic enforcement in Kerala?

Under the "Safe Kerala" project, the Motor Vehicles Department installed AI cameras across the state's roads in 2023. They automatically detect violations such as riding without a helmet, not wearing a seat belt, and using a mobile phone while driving, then generate challans. This is AI for road safety and enforcement rather than flow optimization, but it uses the same computer-vision techniques.

**Exam tip:** Clearly separate the two transport applications: autonomous vehicles (make the vehicle intelligent) and traffic optimization (make the network intelligent). Give two real examples of each.

## Case study: Tesla Autopilot and FSD

**Definition:** A driver-assistance (self-driving) system lets a car steer, accelerate and brake automatically in many conditions, using deep learning, computer vision and sensor fusion.

Deep learning, computer vision, sensor fusion എന്നിവ ഉപയോഗിച്ച് പല സാഹചര്യങ്ങളിലും car-നെ സ്വയം steer ചെയ്യാനും accelerate ചെയ്യാനും brake ചെയ്യാനും അനുവദിക്കുന്ന system ആണ് driver-assistance (self-driving) system.

Tesla's **Autopilot** and **Full Self-Driving (FSD)** let the car steer, accelerate and brake automatically in many conditions. They are built on **deep learning, computer vision and sensor fusion**. Neural networks identify traffic lights, pedestrians, lane markings, vehicles and road signs. Unlike rule-based systems, the networks **learn from massive real-world driving data** collected from Tesla's fleet of millions of cars. The unit describes the decision part as **state-space search**: continuously evaluating possible actions (turn, stop, accelerate) and picking the safest, most efficient one. Fully autonomous driving is still under development and faces regulatory challenges.

### മലയാളത്തിൽ

Tesla-യുടെ ബലം അതിന്റെ fleet data ആണ്. ലോകമെമ്പാടും ഓടുന്ന ലക്ഷക്കണക്കിന് കാറുകൾ കാണുന്ന അപൂർവ്വ സംഭവങ്ങൾ (ഉദാ. റോഡിൽ വീണ മരക്കൊമ്പ്) ശേഖരിച്ച് neural network-നെ വീണ്ടും പരിശീലിപ്പിക്കുന്നു. ഓരോ നിമിഷവും "brake ചെയ്യണോ, lane മാറണോ, speed കൂട്ടണോ" എന്ന actions താരതമ്യം ചെയ്ത് ഏറ്റവും സുരക്ഷിതമായത് തിരഞ്ഞെടുക്കുന്നു. പക്ഷേ ഇപ്പോഴും driver ശ്രദ്ധിച്ചിരിക്കണം; ഇത് Level 2 ആണ്.

### EXAMPLE (DECISION AS SEARCH OVER ACTIONS)

State: own speed 20 m/s, car ahead 30 m away at 15 m/s, left lane clear. Actions: A1 keep speed, A2 brake gently, A3 change to the left lane. The cost is risk plus delay, with a low score being best. Which action is chosen?

A1: the gap shrinks by 5 m/s, so the car reaches the one ahead in  $30/5 = 6$  s. Risk is high: cost 9.

A2: gap stable, small delay: cost 3.

A3: lane clear, no delay, small lane-change risk: cost 2.

The planner picks A3, the lowest cost. If a car later appears in the left lane, A3's risk jumps and A2 wins. The evaluation repeats many times a second.

Techniques: deep learning · computer vision · sensor fusion · fleet learning · planning (search over actions)

Benefits (per the unit): autonomous driving, road safety, energy efficiency through optimized routes and driving patterns

Limits: still Level 2 (driver supervision required), regulatory approval, rare "edge cases", liability

**Exam tip:** Two points in the unit are out of date. (1) Sensors: Tesla removed radar from new cars from 2021 and ultrasonic sensors from 2022, and now relies on cameras only ("Tesla Vision"). Its "sensor fusion" today mainly means fusing the views of eight cameras. (2) Tesla's recent FSD versions (v12 onward, 2024) use end-to-end neural networks for planning rather than hand-coded search. In an IGNOU answer you can still describe decision-making as evaluating possible actions and choosing the safest; just don't claim it is Level 4 or 5.

## AI in customer service and chatbots

**Chatbot:** Software that uses NLP and machine learning to converse with users in natural language and answer their queries automatically, 24/7.

**Chatbot:** NLP-യും machine learning-ഉം ഉപയോഗിച്ച് users-മായി natural language-ൽ സംസാരിക്കുകയും അവരുടെ queries-ന് 24/7 സ്വയമേവ മറുപടി നൽകുകയും ചെയ്യുന്ന software.

**Sentiment analysis:** Detecting from a customer's text whether they are angry, confused or satisfied.

**Sentiment analysis:** ഒരു customer-ന്റെ text-ൽ നിന്ന് അവർ ദേഷ്യത്തിലാണോ, ആശയക്കുഴപ്പത്തിലാണോ, സംതൃപ്തരാണോ എന്ന് കണ്ടെത്തൽ.

Good customer service builds trust and satisfaction. Earlier it was provided by human executives over phone, email or in person. **Chatbots and AI virtual assistants** now give **24/7 support, reduce costs and improve satisfaction**.

#### മലയാളത്തിൽ

അർദ്ധരാത്രി train ticket-ന്റെ refund status അറിയണമെങ്കിൽ ഇനി call centre തുറക്കാൻ കാത്തിരിക്കേണ്ട; chatbot ഉടനെ മറുപടി തരും. ദേഷ്യത്തോടെ എഴുതുന്ന customer-നെ sentiment analysis തിരിച്ചറിഞ്ഞ് ഒരു മനുഷ്യ executive-ന് കൈമാറും. Malayalam ഉൾപ്പെടെ പല ഭാഷകളിലും ഇപ്പോൾ chatbots സംസാരിക്കും.

| Technique (12.5.1)                | Role in customer service                                                          |
|-----------------------------------|-----------------------------------------------------------------------------------|
| Natural Language Processing (NLP) | Understand the customer's message and reply in human language                     |
| Machine learning                  | Learn from feedback and past chats; recommend better responses                    |
| Sentiment analysis                | Detect whether the customer is angry, confused or satisfied, and adjust the reply |
| Personalized recommendation       | Suggest products or services from browsing history and preferences                |

Applications (12.5.2): 24/7 availability across time zones · thousands of queries at once at peak hours · onboarding and registration · delivery status, refunds, order changes · troubleshooting before escalating to a human · multilingual support

#### EXAMPLE (SENTIMENT ANALYSIS)

A simple lexicon gives: late -1, broken -2, worst -3, thanks +1, good +2. Score two messages. M1: "Delivery was late and the product is broken, worst service." M2: "Got it, good packing, thanks."

M1 = -1 - 2 - 3 = -6: strongly negative, so apologise, offer a replacement and escalate to a human agent.

M2 = +2 + 1 = +3: positive, so thank the customer and perhaps ask for a review. Real systems use trained models that also handle negation ("not good") and sarcasm, where word lists fail.

#### EXAMPLE (INDIAN AND KERALA CHATBOTS)

Name some deployed assistants.

IRCTC's **AskDisha** answers railway booking queries. SBI's **SIA** and HDFC Bank's **EVA** handle banking queries. The Government of India's **Bhashini** platform provides translation and speech tools for Indian languages, including Malayalam, that chatbots can use to serve people in their own language. In 2019 Kerala Police introduced **KP-Bot**, a humanoid robot at its headquarters in Thiruvananthapuram, to receive visitors and direct them at the front office, an early example of an AI front-desk assistant in public service.



**Exam tip:** For "risks of relying heavily on AI for complaint resolution" (Check Your Progress 2, Q3) write: loss of empathy and personalization, misreading complex queries, sarcasm or emotion, privacy issues, algorithmic bias, system errors, over-automation, and no escalation path for sensitive cases. The fix is a human-in-the-loop hand-off.

## Case study: Amazon Alexa

**Definition:** A voice assistant is AI software that uses speech recognition, NLP and ML to understand spoken commands, identify the user's intent and respond or act on it.

Speech recognition, NLP, ML എന്നിവ ഉപയോഗിച്ച് സംസാരിച്ച commands മനസ്സിലാക്കി, user-ന്റെ intent തിരിച്ചറിഞ്ഞ്, മറുപടി നൽകുകയോ പ്രവർത്തിക്കുകയോ ചെയ്യുന്ന AI software ആണ് voice assistant.

**Alexa** is Amazon's voice-based AI assistant and one of the most widely used virtual assistants. It uses **speech recognition, NLP and ML** to understand spoken commands, work out the user's intent and respond naturally. Users can check prices, place orders, track deliveries and manage subscriptions by voice. The unit says Alexa identifies intent using **rule-based state-space search**, matches it to a response and executes the action. Its ML algorithms learn from millions of interactions to become more accurate and personal. Companies use Alexa as a virtual customer-care executive; smart-home device makers, for example, use it to help customers set up and control products.

### മലയാളത്തിൽ

"Alexa, എന്റെ order എവിടെ എത്തി?" എന്ന് ചോദിക്കുമ്പോൾ ആദ്യം ശബ്ദം text ആക്കുന്നു (speech recognition). പിന്നെ NLP "ഇത് TrackOrder എന്ന intent ആണ്" എന്ന് മനസ്സിലാക്കുന്നു, order system-ൽ നോക്കി ഉത്തരം ശബ്ദമായി തിരിച്ചു പറയുന്നു. കൂടുതൽ ഉപയോഗിക്കുതോറും അത് നിങ്ങളുടെ ശീലങ്ങൾ പഠിക്കും.

Voice-assistant pipeline:

Wake word ("Alexa") → Automatic speech recognition (audio → text) → NLU (intent + slots) → Dialogue manager / skill (decide and act) → Response → Text-to-speech (text → audio)

Benefits: faster response · 24×7 availability · better satisfaction · lower overheads

### EXAMPLE

Trace the utterance "Alexa, reorder the dog food I bought last month."

1. Wake word detected on the device. 2. ASR gives the text "reorder the dog food I bought last month". 3. NLU: intent = ReorderItem; slots: item = "dog food", time = "last month". 4. The dialogue manager searches the order history, finds the product and asks "Reorder Brand X 10 kg for ₹2,450?" 5. The user says "yes", the order is placed and confirmed by voice. If the user's answer does not match any expected option, the assistant re-prompts or hands over to support.

### EXAMPLE (STATE-SPACE VIEW)

Show the dialogue as moving through states.

S0 (idle) → [wake word] → S1 (listening) → [intent recognized] → S2 (awaiting confirmation) → ["yes"] → S3 (order placed) → S0. A "no" at S2 leads to S4 (ask which item). This is the sense in which the unit calls it a search through states: from the current state, the system chooses the transition that best serves the recognized intent.

**Exam tip:** The unit calls Alexa's intent identification "rule-based state-space search". In practice Alexa recognizes intents mainly with ML-based natural-language-understanding models (intent classification and slot filling); rules and state machines mostly manage the dialogue flow. A safe exam line: "Alexa uses speech recognition and NLP/ML to identify intent, and a dialogue manager to choose the response and action."

## AI in education: personalized learning

**Definition:** Personalized learning adapts the content, pace and learning style of teaching to each student's needs, instead of a one-size-fits-all approach.

One-size-fits-all രീതിക്ക് പകരം, ഓരോ student-ന്റെയും ആവശ്യങ്ങൾക്കനുസരിച്ച് content, pace, learning style എന്നിവ ക്രമീകരിക്കുന്നതാണ് personalized learning.

Traditional teaching is one-size-fits-all, but learners differ in ability, preferred style and speed. **Personalized learning** adapts the content, pace and learning style to each student's needs. If a student struggles with algebra, the AI gives extra exercises, step-by-step hints or videos on that topic. If another finds it easy, it gives harder problems to keep them challenged.

#### മലയാളത്തിൽ

ഒരു ക്ലാസ്സിൽ 40 കുട്ടികൾ; ടീച്ചർക്ക് ഓരോരുത്തരുടെയും വേഗത്തിനനുസരിച്ച് പഠിപ്പിക്കാൻ സമയമില്ല. Adaptive learning system ഓരോ കുട്ടിക്കും ഒരു private tuition master പോലെയാണ്. തെറ്റുന്ന topic-ൽ കൂടുതൽ practice-ഉം hints-ഉം, എളുപ്പമുള്ളിടത്ത് കടുപ്പമുള്ള ചോദ്യങ്ങളും തരും.

| Feature                         | Meaning                                                                 | Example                                     |
|---------------------------------|-------------------------------------------------------------------------|---------------------------------------------|
| Real-time difficulty adjustment | Questions get easier or harder as the student answers                   | Khan Academy, DreamBox                      |
| Learning analytics              | Scores, speed and mistakes analysed to find weak areas                  | Teacher dashboards                          |
| Recommendations                 | Lessons, exercises or videos chosen by weak area or in increasing order | Coursera course recommendations             |
| Gamification                    | Points, badges, levels reward progress                                  | Learning games                              |
| Adaptive content                | Digital textbooks / video lessons adapt to the learner                  | Knewton; BYJU'S adaptive videos and quizzes |

#### EXAMPLE (AN ADAPTIVE RULE)

A system uses: accuracy  $< 60\%$   $\rightarrow$  easier level plus hints and a video;  $60-85\%$   $\rightarrow$  same level;  $> 85\%$   $\rightarrow$  next level. Anu scores 4/10 on quadratic equations, Rahul 9/10. What happens?

Anu:  $40\% < 60\%$ , so she gets a video on factorization, then easier questions with step-by-step hints. Rahul:  $90\% > 85\%$ , so he moves on to word problems on quadratics. Both keep learning at their own level, and the teacher's dashboard shows "factorization" as Anu's weak area.

#### EXAMPLE (KERALA)

An AI "teacher" in a Kerala classroom.

In March 2024 KTCT Higher Secondary School, Kallambalam (Thiruvananthapuram) introduced **Iris**, a generative-AI humanoid robot teacher developed with Makerlabs Edutech under the school's Atal Tinkering Lab. It answers students' questions in English, Malayalam and Hindi. It supports the human teachers; it does not replace them.

**Exam tip:** The unit calls Knewton a Pearson product. Knewton did partner with Pearson to power adaptive digital courses, but it was an independent company and was acquired by **Wiley** in 2019. Writing "Knewton (used in Pearson's courses)" is safe. The unit also writes "BYJU'S used AI", in the past tense; the company has since faced serious financial trouble.

## Intelligent Tutoring Systems (ITS)

**Definition:** An Intelligent Tutoring System (ITS) is a computer program that uses AI to give one-to-one tutoring like a human teacher, with hints, corrections and real-time feedback.

Hints, corrections, real-time feedback എന്നിവയോടെ, ഒരു മനുഷ്യ അധ്യാപകനെപ്പോലെ one-to-one tutoring നൽകാൻ AI ഉപയോഗിക്കുന്ന computer program ആണ് Intelligent Tutoring System (ITS).

An **Intelligent Tutoring System** is a computer program that provides **one-to-one tutoring** like a human teacher. It uses AI to understand the student's response, give hints, correct mistakes and give feedback in real time. It has four components.

### മലയാളത്തിൽ

ഒരു നല്ല tuition teacher-ന് നാല് കാര്യങ്ങൾ അറിയാം: subject (domain model), കുട്ടിക്ക് എന്തറിയാം എന്തറിയില്ല (student model), അടുത്തതായി എന്ത് പഠിപ്പിക്കണം (tutoring model), എങ്ങനെ സംസാരിക്കണം (user interface). ITS-ലും ഇതേ നാല് ഭാഗങ്ങളാണ്.

| Component                    | Stores / does                              | In a maths ITS                                              |
|------------------------------|--------------------------------------------|-------------------------------------------------------------|
| Domain model                 | Knowledge of the subject (rules, facts)    | Rules of algebra: how to solve $2x + 3 = 11$                |
| Student model                | Student's pace, weaknesses, misconceptions | "Often forgets to change the sign when moving a term"       |
| Tutoring (pedagogical) model | Decides the next lesson, hint or problem   | "Give a hint on transposing terms, then two practice items" |
| User interface               | Interaction by text, voice or visuals      | Step-by-step input boxes, spoken hints                      |

### EXAMPLE

A student solves  $2x + 3 = 11$  and writes " $2x = 11 + 3, x = 7$ ". Show how the ITS components respond.

Domain model: the correct step is  $2x = 11 - 3 = 8$ , so  $x = 4$ . It spots that the student added 3 instead of subtracting.

Student model: records the misconception "sign error on transposition" (the third time this week).

Tutoring model: gives the hint "When +3 moves to the other side, what happens to its sign?", then chooses two similar problems.

Interface: shows the hint next to the wrong line. The student corrects to  $x = 4$  and gets instant feedback.

Real-world ITS: Carnegie Learning's Cognitive Tutor (mathematics) · AutoTutor (science and technology through natural-language dialogue) · Duolingo (personalized language learning)

**Exam tip:** "Explain the components of an ITS with a diagram" is a likely long question.

Draw the four boxes with the student model at the centre: the domain model feeds knowledge, the tutoring model reads the student model to choose actions, and the interface connects to the learner.

## Case study: Duolingo

**Adaptive learning:** An AI approach that tracks each learner's performance and adjusts the difficulty and pace of lessons to it.

**Adaptive learning:** ഓരോ learner-ന്റെയും performance നിരീക്ഷിച്ച് അതിനനുസരിച്ച് lessons-ന്റെ difficulty-യും pace-ഉം ക്രമീകരിക്കുന്ന AI രീതി.

**Gamification:** Using game elements such as streaks, points and leaderboards to keep learners motivated.

**Gamification:** Learners-ന്റെ താൽപര്യം നിലനിർത്താൻ streaks, points, leaderboards പോലെയുള്ള game elements ഉപയോഗിക്കൽ.

**Duolingo** is a language-learning platform that uses AI to give millions of users a personalized, adaptive experience. It records each user's **response time, accuracy and mistakes**, predicts their proficiency, and adjusts exercise difficulty and the pace of new lessons. The unit views this as **state-space search**: evaluate the learner's current state and choose the next lesson that maximizes progress. **Rules** keep grammar and vocabulary in a logical sequence, **ML** finds weak topics for focused practice, **NLP** checks pronunciation and written answers instantly, and **gamification** (streaks, points, leaderboards) keeps users motivated with adaptive rewards.

#### മലയാളത്തിൽ

Duolingo-യിൽ നിങ്ങൾ ഒരു വാക്ക് ഇന്നലെ ശരിയായി പറഞ്ഞാലും കുറച്ച ദിവസം കഴിഞ്ഞാൽ മറന്നുപോകാം. എപ്പോൾ മറക്കാൻ സാധ്യതയുണ്ടെന്ന് AI പ്രവചിച്ച് അതിന് തൊട്ടുമുമ്പ് ആ വാക്ക് വീണ്ടും ചോദിക്കും. Streak മുറിച്ചാതിരിക്കാൻ ദിവസവും app തുറക്കുന്നത് gamification-ന്റെ ഫലമാണ്.

| AI technique      | What it does in Duolingo                                            |
|-------------------|---------------------------------------------------------------------|
| Rule-based system | Presents grammar and vocabulary in a logical order                  |
| Machine learning  | Predicts proficiency; finds weak topics; schedules revision         |
| NLP / speech      | Evaluates pronunciation and typed answers; instant feedback         |
| State-space view  | Current learner state → choose the next lesson to maximize progress |
| Gamification      | Streaks, points, leaderboards, adaptive rewards                     |

#### EXAMPLE (PREDICTING FORGETTING)

Duolingo researchers published a "half-life regression" model (2016) in which the chance of recalling a word is  $p = 2^{-\Delta/h}$ , where  $\Delta$  = days since last practice and  $h$  = the word's memory half-life for that learner. With  $h = 4$  days, find  $p$  after 2, 4 and 8 days.

$$\Delta = 2: p = 2^{-0.5} \approx 0.71. \Delta = 4: p = 2^{-1} = 0.5. \Delta = 8: p = 2^{-2} = 0.25.$$

The app schedules the word for practice before  $p$  drops too low, around day 4 here. Each correct review increases  $h$ , so the word comes back less often. That is personalization through ML.

#### EXAMPLE (NEXT-STATE CHOICE)

A learner's state: Food vocabulary 95% mastered, Past tense 55%, Greetings 90% but last practised 20 days ago. Which lesson next?

Past tense is the weakest skill (55%), so ML picks focused past-tense practice. Greetings is strong but may be fading, so one short review is added. Food at 95% can wait. The rule-based sequence then allows the next unit only after past tense crosses the required level.

**Exam tip:** Structure a case-study answer as problem (one-size-fits-all teaching) → data collected (response time, accuracy, mistakes) → techniques (rules, ML, NLP, gamification) → outcome (personalized, interactive, continuous learning). Since 2023 Duolingo also uses large language models (GPT-4, in "Duolingo Max") for explanations and role-play conversation.

## Comparing the five domains; common challenges

Across all domains, AI processes huge volumes of data, learns patterns and performs complex tasks quickly and accurately, and it gives data-driven insights for decisions. The common challenges are **data privacy, ethical issues (bias, accountability) and the need for human oversight**. The unit concludes that AI is reshaping industries and how humans interact with machines.

### മലയാളത്തിൽ

അഞ്ച് മേഖലകളിലും കഥ ഒന്നുതന്നെ: ധാരാളം data, അതിൽ നിന്ന് pattern പഠിക്കുന്ന ML, ഭാഷയ്ക്ക് NLP, കാഴ്ചയ്ക്ക് computer vision. വെല്ലുവിളികളും ഒന്നുതന്നെ: privacy, bias, തെറ്റിന്റെ ഉത്തരവാദിത്വം, മനുഷ്യന്റെ മേൽനോട്ടം. AI സഹായിയാണ്; അന്തിമ തീരുമാനം മനുഷ്യന്റേതാണ്.

| Domain           | Main tasks                                     | Key techniques                                              | Case study              | Main benefit                                        | Main challenge                                        |
|------------------|------------------------------------------------|-------------------------------------------------------------|-------------------------|-----------------------------------------------------|-------------------------------------------------------|
| Healthcare       | Diagnosis, treatment, drug discovery, imaging  | ML, CNN/ computer vision, NLP                               | IBM Watson for Oncology | Earlier, more accurate diagnosis; personalized care | Privacy, dataset bias, accountability                 |
| Finance          | Fraud detection, algo-trading, risk assessment | ML, anomaly detection, NLP, DL                              | JPMorgan COiN           | Security, speed, lower cost                         | Explainability, fairness in credit, market risk       |
| Transportation   | Autonomous vehicles, traffic optimization      | Computer vision, sensor fusion, ML, planning/search         | Tesla Autopilot/ FSD    | Safety, less congestion and fuel use                | Safety, cost, infrastructure, trust, regulation       |
| Customer service | Chatbots, virtual assistants                   | NLP, speech recognition, sentiment analysis, recommendation | Amazon Alexa            | 24/7 support, scale, lower cost                     | Lack of empathy, misunderstanding, privacy            |
| Education        | Personalized learning, ITS, automated grading  | ML, rules, NLP, learning analytics                          | Duolingo                | Learning at each student's own pace                 | Over-reliance, weaker critical thinking, data privacy |

#### EXAMPLE

Name the main AI technique for each task: (a) reading legal contracts, (b) spotting a pedestrian, (c) detecting an angry customer, (d) flagging an unusual card payment, (e) picking a student's next exercise.

(a) NLP. (b) Computer vision (a CNN on camera frames, fused with LiDAR/radar). (c) Sentiment analysis, a branch of NLP. (d) ML anomaly detection. (e) Student model plus tutoring model (ML plus rules), i.e. an adaptive learning system.



### EXAMPLE

How should a teacher's role change in an AI-supported classroom (Check Your Progress 3, Q3)?

From content-deliverer to facilitator, mentor and ethical guide: interpret the AI's learning analytics, personalize support, build critical thinking, check that AI output is accurate and unbiased, and give emotional support, creativity and human contact, which AI cannot.

**Exam tip:** For a long "applications of AI" question, a table like the one above followed by one paragraph per domain (task, technique, example, one challenge) scores well and is quick to write.

## Quick recap

- AI is needed because traditional rule-based and manual methods cannot handle huge, complex, fast-changing data or make timely decisions.
- Healthcare AI supports diagnosis, treatment planning, drug discovery and medical imaging; it assists doctors and never replaces them.
- Diagnosis examples: LYNA (metastatic breast cancer), ECG analysis, brain scans for Alzheimer's/Parkinson's, mental-health assessment.
- Treatment: precision medicine, robot-assisted surgery, drug discovery (AlphaFold), wearables for chronic diseases.
- IBM Watson for Oncology (with MSKCC) uses NLP + ML and ranks options as recommended / for consideration / not recommended; it struggled with local protocols.
- Healthcare concerns: privacy, dataset bias, human final decision, accountability for errors.
- Finance: ML fraud detection adapts where fixed rules fail; algo-trading uses prediction, NLP sentiment and high-speed execution; risk assessment uses alternative data.
- JPMorgan COiN (Contract Intelligence): NLP + ML contract review cut 360,000 hours a year to seconds.
- Autonomous vehicles: sense → perceive → localize → decide → control; sensor fusion gives complementary strengths and redundancy.
- SAE levels run from 0 (no automation) to 5 (full automation); Tesla's systems are Level 2 and Waymo's robotaxis Level 4.
- Traffic optimization: adaptive signals (Surtrac: about 25% less travel time, 40% less waiting), predictive analysis, smart navigation, connected vehicles, bus scheduling.
- Customer service uses NLP, ML, sentiment analysis and recommendation to give 24/7, scalable, multilingual support; complex cases need a human hand-off.
- Alexa: speech recognition → intent → action → spoken reply, and it improves by learning from interactions.
- Education: personalized learning adapts content, pace and style; an ITS has domain, student and tutoring models plus a user interface; Duolingo combines rules, ML, NLP and gamification.

## Check yourself

### Q. 1. What are the challenges of using AI in healthcare?

(i) Data privacy: medical data is highly sensitive and misuse harms patients. (ii) Lack of quality, representative data: biased datasets give wrong results for under-represented groups. (iii) High implementation cost. (iv) Need for expert supervision: AI assists, and the doctor makes the final decision. (v) Accountability: who is responsible for a wrong AI diagnosis? (vi) Adapting to local protocols and fast-changing medical knowledge, as seen with IBM Watson for Oncology.

### Q. 2. How can AI ensure security in the domain of finance?

AI detects fraud by monitoring millions of transactions in real time and flagging anomalies (e.g. one card used in two countries within minutes). It analyses logins to catch abnormal behaviour such as access from an unusual device. It scans insurance claims for false claims and detects internal risks such as cyberattacks or staff misconduct. It helps protect customer data. Unlike fixed rules, ML models keep learning new fraud patterns; JPMorgan Chase, for example, uses AI for real-time transaction monitoring.

### Q. 3. How does bias in medical training datasets affect AI-driven clinical decisions? Suggest solutions.

Effect: a model trained mostly on one group (say, light-skinned or male patients) is less accurate for others. This leads to missed or wrong diagnoses and unequal treatment recommendations, widens healthcare disparities and reduces trust in AI. Solutions: diverse and representative datasets, continuous bias auditing (report accuracy per group), transparent and explainable algorithms, inclusive clinical validation on local populations, and multidisciplinary teams (doctors, data scientists, ethicists) overseeing the system.

### Q. 4. How do sensor fusion techniques (LiDAR, radar, cameras) improve decision-making in autonomous vehicles, and what limitations still exist?

Sensor fusion combines complementary sensors: cameras read colour, signs and lanes; LiDAR gives a precise 3D distance map; radar measures distance and speed even in rain and fog. Together they give a more accurate and reliable picture of the surroundings, which improves object detection and distance estimation, and redundancy means one failed sensor does not blind the car. Better perception leads to safer decisions about braking, lane changes and speed. Limitations: high computing demand, sensor calibration, sensitivity to weather (heavy rain or fog affects cameras and LiDAR), high cost (especially LiDAR), and difficulty with rare or unpredictable road situations.

**Q. 5. What is the role of sentiment analysis in AI-based customer service?**

Sentiment analysis (an NLP technique) detects the customer's emotion from their text or voice: angry, confused or satisfied. The system then adjusts its reply: it apologises and escalates an angry customer to a human, explains more simply to a confused one, and thanks a satisfied one. Across many conversations it also measures overall satisfaction and highlights recurring complaints. The result is more accurate responses and higher customer satisfaction.

**Q. 6. Discuss the potential risks of relying heavily on AI for customer complaint resolution.**

Loss of human empathy and personalization, which frustrates customers. Misinterpretation of complex queries, sarcasm or emotional tone, leading to inappropriate replies. Data-privacy risks from storing conversations. Algorithmic bias that treats some customers unfairly. System errors and over-automation, where the customer gets stuck in a loop with no way to reach a person. Sensitive cases (fraud, bereavement, medical issues) need human judgment. Mitigation: a clear hand-off to human agents, monitoring and privacy safeguards.

**Q. 7. Explain how AI-based adaptive learning systems improve student performance.**

They analyse each student's performance data (scores, speed, mistakes) to find strengths, weaknesses and knowledge gaps. Then they adjust difficulty, recommend targeted practice and give immediate feedback, so mistakes are corrected quickly and concepts are reinforced. Students learn at their own pace, which reduces frustration and builds confidence, and gamification keeps them engaged. Examples: Khan Academy and DreamBox adjust question difficulty in real time, and Duolingo schedules revision before a word is forgotten. The results are better retention and achievement than one-size-fits-all teaching.

**Q. 8. What is an Intelligent Tutoring System? Describe its components with an example.**

An ITS is a computer program that gives one-to-one tutoring like a human teacher, using AI to understand responses, give hints, correct mistakes and give real-time feedback. Components: the **domain model** (subject knowledge, e.g. algebra rules); the **student model** (the learner's pace, weaknesses, misconceptions); the **tutoring model** (decides the next lesson or hint); and the **user interface** (text, voice or visual interaction). Example: when a student writes  $2x + 3 = 11 \Rightarrow 2x = 14$ , the domain model detects the sign error, the student model records it, the tutoring model gives a hint on transposing terms, and the interface shows it. Real ITS: Carnegie Learning's Cognitive Tutor, AutoTutor, Duolingo.

**Q. 9. Explain the SAE levels of driving automation. At which level do most current commercial systems operate?**

SAE defines six levels. 0: no automation. 1: driver assistance (steering or speed). 2: partial automation (steering and speed, but the driver supervises constantly). 3: conditional automation (the car drives in limited conditions and the driver takes over on request). 4: high automation (no driver needed within a defined area and conditions). 5: full automation (anywhere, any conditions). Most cars sold today, including Tesla Autopilot/FSD, are Level 2. A few Level 3 systems exist (Mercedes-Benz Drive Pilot), and robotaxis such as Waymo operate at Level 4 in limited areas. No Level 5 system exists yet. (The unit groups Tesla and Cruise as "Level 2 or 3"; Tesla is Level 2, and Cruise ran Level 4 robotaxis until GM ended the programme in 2024.)

**Q. 10. Write a short note on the JPMorgan Chase COiN platform.**

COiN (Contract Intelligence) is JPMorgan Chase's AI platform for reviewing legal and financial documents, especially commercial loan agreements. Using NLP and ML it reads contracts and identifies important clauses, risks and compliance requirements. It reduced review work of about 360,000 employee hours a year to a few seconds, which saves cost and time and improves consistency and accuracy by removing fatigue-related human error. JPMorgan also uses AI for real-time fraud detection and transaction monitoring; these models learn new fraud patterns that rule-based systems miss, which improves customer trust and reduces financial risk.

---

Based on the IGNOU unit texts. Check definitions against the PDFs before the exam.